

Modelos e Técnicas Computacionais para Análise de Requisitos de Segurança

Safety e Security: da Engenharia de Requisitos ao Modelo
STAMP e à Técnica STPA

Notas de aula organizadas a partir da disciplina de
Modelos e Técnicas Computacionais para Análise de Requisitos de
Segurança
(Safety e Security)

Compiladas e organizadas por
Jeann Victor

Prefácio

Este livro nasceu a partir das transcrições das aulas da disciplina de *Modelos e Técnicas Computacionais para Análise de Requisitos de Segurança (Safety e Security)*. O objetivo é transformar o conteúdo apresentado em aula em um material de consulta organizado, direto e agradável de reler sempre que houver necessidade de relembrar os conceitos discutidos ao longo do curso.

A estrutura do livro segue a mesma sequência lógica adotada durante as aulas: parte-se dos fundamentos de engenharia de requisitos segundo o SWEBOK e da engenharia de sistemas segundo o INCOSE, avança-se para os conceitos básicos de sistemas e controle, passa-se pelo estudo de caso do desastre de Bhopal, discutem-se os modelos tradicionais de causalidade de acidentes e seus limites diante da complexidade dos sistemas atuais, e por fim apresenta-se o modelo STAMP e a técnica STPA para análise de hazards e prevenção de acidentes em sistemas complexos.

Sempre que pertinente, os conceitos apresentados pelo professor foram complementados com explicações adicionais, de modo a facilitar a compreensão. Ainda assim, a base principal e a linha condutora de todo o conteúdo é o material e a fala do professor responsável pela disciplina.

Boa leitura e bons estudos.

Sumário

Prefácio	i
I Engenharia de Requisitos e Engenharia de Sistemas	1
1 SWEBOK: Requisitos de Software (Parte 1) — Fundamentos e Processo	2
1.1 Por que Requisitos Importam	2
1.2 Quem Define os Requisitos	3
1.3 Categorias de Requisitos	3
1.4 Atividades de Requisitos de Software	4
1.5 Elicitação de Requisitos	4
2 SWEBOK: Requisitos de Software (Parte 2) — Elicitação e Análise	6
2.1 Qualidade de Requisitos Individuais e da Coleção	6
2.2 Qualidade de Serviço, Custo e Análise Formal	6
2.3 Especificação de Requisitos: Linguagem Natural	7
2.4 Especificação Baseada em Critérios de Aceitação	8
2.5 Modelos Gráficos e Formas de Versionamento	8
3 SWEBOK: Requisitos de Software (Parte 3) — Especificação e Validação	10
3.1 Validação de Requisitos	10
3.2 Gestão de Requisitos	11
3.3 Considerações Práticas	11
3.4 Ferramentas de Requisitos	12
4 Introdução à Engenharia de Sistemas e a Certificação INCOSE	14
4.1 O que é um Sistema e o que é Engenharia de Sistemas	14
4.2 O INCOSE e as Carreiras em Engenharia de Sistemas	15
4.3 As Certificações do INCOSE	15
4.4 O Systems Engineering Handbook	15
5 Engenharia de Requisitos no Contexto de Engenharia de Sistemas (Parte 1)	17
5.1 De Necessidades a Requisitos	17
5.2 A Hierarquia de Necessidades e Requisitos na Organização	17
5.3 Conceitos de Ciclo de Vida	18
5.4 Classificação e Justificativa de Requisitos	18
5.5 Propriedades Emergentes	19

6 Engenharia de Requisitos no Contexto de Engenharia de Sistemas (Parte 2)	21
6.1 Hierarquia, Elicitação e Elaboração de Requisitos	21
6.2 Verificação, Validação e Testes ao Longo do Ciclo de Vida	22
6.3 Gestão de Requisitos e Ferramentas	22
6.4 Técnicas de Engenharia de Requisitos	22
 II Conceitos Básicos de Engenharia de Sistemas	 24
7 Design Conceitual	25
7.1 O Ciclo de Vida de Desenvolvimento de Sistemas	25
7.2 Lacunas de Capacidade (Capability Gaps)	25
7.3 As Cinco Atividades do Design Conceitual	26
 8 O que é um Sistema?	 28
8.1 Definição de Sistema e o Papel do Objetivo	28
8.2 Sistemas de Engenharia versus Sistemas Naturais	28
8.3 A Definição Recursiva de Sistema	29
8.4 Estado, Propriedades, Ambiente e Fronteira	29
 9 O que é Engenharia de Sistemas?	 31
9.1 Definição e Origem da Engenharia de Sistemas	31
9.2 O Modelo V e a Ilusão da Linearidade	31
9.3 Documentação, Especificação e Fronteira do Sistema	32
9.4 Rastreabilidade	32
 10 O Conceito de Controle e Técnicas de Design para Safety	 34
10.1 O Conceito de Controle na Engenharia de Sistemas	34
10.2 Controles em Nível de Design	34
10.2.1 Técnicas de Design Intrinsecamente Seguro	35
10.2.2 Técnicas de Design Passivo	35
10.2.3 Técnicas de Design Ativo	35
10.3 Comparativo entre as Técnicas	36
 11 O Laço de Controle (Control Loop)	 37
11.1 Origem e Definição	37
11.2 Os Componentes do Control Loop	37
11.3 Feedback Control e Feedforward Control	38
11.4 Aplicação além da Engenharia	39
 12 Teoria de Sistemas versus Teoria da Complexidade	 40
12.1 Teoria de Sistemas	40
12.2 Teoria da Complexidade	40
12.3 Semelhanças e Diferenças	40

III	Estudo de Caso: O Desastre de Bhopal	42
13	O Desastre de Bhopal: Contexto Histórico	43
13.1	A Substância e o Perigo Envolvido	43
13.2	Os Dispositivos de Segurança da Usina	43
13.3	Uma Estrutura de Segurança Aparentemente Robusta	44
14	O Desastre de Bhopal: a Sequência de Eventos	46
14.1	A Cronologia dos Dezenove Eventos	46
14.2	Identificando Causas a Partir dos Eventos	47
14.3	As Perguntas que os Eventos Não Respondem	47
14.4	A Lição do Nível 1	48
15	O Desastre de Bhopal: Condições Inseguras e Falhas de Projeto	49
15.1	Os Dispositivos de Defesa em Profundidade Sob a Lupa	49
15.2	O Tanque de Armazenamento e os Instrumentos	50
15.3	Falhas de Procedimento: Slip Blind, Supervisão e Alarmes	50
15.4	Uma Nova Perspectiva Sobre a Culpa	51
16	O Desastre de Bhopal: Fatores Sistêmicos e Lições Aprendidas	52
16.1	Pressão Econômica e Corte de Custos	52
16.2	Erosão da Manutenção, Educação e Supervisão Técnica	53
16.3	Autoestima, Moral e Sinais de Alerta Ignorados	53
16.4	Fatores Sistêmicos Além de Bhopal	54
IV	Modelos de Causalidade de Acidentes	56
17	Safety versus Security: Definições Fundamentais	57
17.1	O Conceito de Acidente	57
17.2	Definindo Safety	57
17.3	A Diferença entre Safety e Security	58
18	O Modelo de Cadeia de Eventos	59
18.1	O que é um Modelo de Causalidade	59
18.2	A Estrutura do Modelo de Cadeia de Eventos	59
18.3	Barreiras entre Eventos	60
18.4	Encarnações do Modelo	60
18.5	Os Limites do Modelo	61
19	A Sedução pela Causa Raiz	62
19.1	A Arbitrariedade da Causa Raiz	62
19.2	A Sedução pela Causa Raiz	62
19.3	O Viés em Direção ao Operador	63
19.4	Lição Aprendida	63

20 Tipos de Acidentes e os Limites dos Modelos Tradicionais	65
20.1 Dois Tipos de Acidentes	65
20.2 A Complexidade dos Sistemas Atuais	66
20.3 Exemplos de Acidentes sem Falha de Componentes	66
20.4 Por Que as Técnicas Tradicionais Não Bastam	67
21 Confiabilidade, Software e a Integração Homem-Máquina: Lições Aprendidas	69
21.1 Safety Não é o Mesmo que Confiabilidade	69
21.2 Como o Software Revolucionou a Engenharia	69
21.3 Integrando Pessoas, Software e Hardware	70
21.4 Lições Aprendidas do Modelo de Cadeia de Eventos	71
 V O Modelo STAMP	 73
22 Mudança de Paradigma: Safety como Problema de Controle	74
22.1 A Necessidade de Novos Modelos e Técnicas	74
22.2 Três Formas de Lidar com Complexidade	74
22.2.1 Decomposição Analítica	74
22.2.2 Estatística	75
22.2.3 Teoria de Sistemas	75
22.3 A Mudança de Paradigma	76
22.4 Controlando Safety por Meio de Restrições	76
23 Tratando Safety com o Laço de Controle	78
23.1 Revisão: Controle, Controlador e Control Loop	78
23.2 Explicando Acidentes com o Control Loop	79
23.2.1 Sonda Polar de Marte	79
23.2.2 Reversor de Empuxo em Varsóvia	79
23.2.3 Reversor de Empuxo em Moscou	79
23.2.4 Fogo Amigo em Teste de Míssil	79
24 Introdução ao Modelo STAMP	81
24.1 O que é STAMP	81
24.2 O que Está Incluído no Escopo de STAMP	81
24.3 Pensamento Orientado a Eventos versus Systems Thinking	82
24.4 Acidentes que Exigem Causalidade Além da Linear	82
25 Análise de Hazards e Acidentes com STAMP	84
25.1 Retomando o Control Loop	84
25.2 O Papel Central das Control Actions	84
25.3 STPA para Controladores Humanos	85
25.4 Implicações para a Análise	86

26 Estrutura Hierárquica para Controle de Safety	87
26.1 Sistemas como Estruturas Hierárquicas	87
26.2 A Estrutura Sociotécnica de Leveson	87
26.3 Objetivos do Projeto de uma Estrutura de Controle	88
26.4 Complexidade e o Cuidado com o Caminho Bottom-Up	88
27 Técnicas STAMP: Eficácia e Aplicação na Indústria	90
27.1 Método, Técnica ou Ferramenta?	90
27.2 O Catálogo de Técnicas sobre STAMP	90
27.3 O Modelo e Suas Técnicas Funcionam? Evidências de Retorno de In- vestimento	91
27.4 Estudos de Caso em Sistemas Reais	91
27.5 Indústrias que Utilizam STAMP e Suas Técnicas	92
 VI A Técnica STPA	 94
28 Visão Geral da Técnica STPA	95
28.1 Os Quatro Passos da STPA	95
28.2 Passo 1: Definir o Objetivo da Análise	95
28.3 Passo 2: Modelar a Estrutura de Controle	96
28.4 Passo 3: Identificar as Ações de Controle Inseguras	96
28.5 Passo 4: Identificar os Cenários de Perdas	96
28.6 Requisitos e Rastreabilidade ao Longo dos Passos	96
29 Passo 1 da STPA: Definir o Objetivo da Análise	98
29.1 Visão Geral do Passo 1	98
29.2 Identificação das Perdas Inaceitáveis	98
29.3 Identificação dos Hazards de Nível de Sistema	99
29.3.1 Critérios para Definição de Hazards	100
29.3.2 Erros Comuns e Dicas Práticas	100
29.4 Definição das Restrições de Nível de Sistema	100
29.5 Refinamento Opcional dos Hazards em Sub-hazards	101
30 Passo 2 da STPA: Modelar a Estrutura de Controle	103
30.1 O Que É uma Estrutura de Controle	103
30.2 Equívocos Comuns na Modelagem	104
30.3 Como Modelar a Estrutura de Controle	104
30.4 Questões Frequentes sobre a Estrutura de Controle	105
30.5 Revisando a Estrutura de Controle	106
31 Passo 3 da STPA: Identificar Ações de Controle Inseguras (UCAs)	107
31.1 Definição e Ferramenta Tabular	107
31.2 Dúvidas Comuns sobre UCAs	108
31.3 UCAs com Controladores Humanos	109
31.4 Prevenindo Erros Comuns	109
31.5 Restrições de Controlador	109

32 Passo 4 da STPA: Identificar Cenários de Perdas	111
32.1 O Conceito de Cenário de Perda	111
32.2 Cenários que Levam a Ações de Controle Inseguras	111
32.2.1 Comportamento Inseguro do Controlador	111
32.2.2 Feedback ou Informação Inadequados	112
32.3 Cenários de Execução Inadequada da Ação de Controle	112
32.3.1 Fatores no Caminho de Controle	112
32.3.2 Fatores no Processo Controlado	113
32.4 Prevenindo Erros na Identificação de Cenários	113
32.5 Entradas, Saídas e Rastreabilidade	113
32.6 O Uso das Saídas da STPA	113
32.7 Considerações sobre a Técnica	114
Considerações Finais	115

Parte I

Engenharia de Requisitos e Engenharia de Sistemas

Capítulo 1

SWEBOK: Requisitos de Software (Parte 1) — Fundamentos e Processo

O SWEBOK (*Software Engineering Body of Knowledge*) é o guia que delimita a área de engenharia de software a partir de áreas de conhecimento, indo de processo de software a teste, qualidade e requisitos. Atualmente em sua quarta edição (em elaboração, com a terceira ainda em uso corrente), o SWEBOK dedica uma área de conhecimento inteira a Requisitos de Software, foco central desta disciplina. Este capítulo apresenta os fundamentos dessa área: o que é um requisito, por que a documentação de requisitos é crítica e como o SWEBOK organiza analiticamente o tema.

1.1 Por que Requisitos Importam

Requisitos podem ser vistos por duas perspectivas complementares: como expressão daquilo que é necessário para que um produto de software resolva um problema do mundo real, e como o conjunto de atividades necessárias para desenvolver e manter esses requisitos ao longo do projeto. Ambas as perspectivas — produto e projeto — estão presentes na área de conhecimento do SWEBOK.

Requisitos mal levantados produzem um efeito cascata: um requisito equivocado leva a decisões de design e implementação também equivocadas, e quanto mais tarde o erro é percebido, mais caro é corrigi-lo. Dois problemas centrais explicam a maior parte dos fracassos nessa área. O primeiro é a **incompletude**: o requisito existe no negócio, mas ninguém o percebeu, documentou ou comunicou à equipe de desenvolvimento, de modo que o software simplesmente não o atende. O segundo é a **ambiguidade**: o requisito é comunicado de forma que permite mais de uma interpretação, levando desenvolvedores a implementações divergentes. Um exemplo simples: se um requisito usa o termo "estudante" e outro usa "aluno" para o mesmo conceito, desenvolvedores independentes podem criar duas classes onde deveria haver apenas uma. Um defeito (bug), nesse contexto, nada mais é do que a implementação que desvia da intenção original de um requisito — a diferença entre o que o software deveria fazer e o que ele realmente faz.

Requisito de Software

Um requisito é uma condição ou capacidade que um usuário precisa para resolver um problema ou atingir um objetivo, ou uma condição ou capacidade que deve estar presente em um sistema (ou componente) para satisfazer um usuário, contrato, padrão ou outro documento formalmente imposto. O SWEBOK estende

essa definição para incluir também os *requisitos de projeto*, associados a restrições de tempo, escopo e custo impostas ao desenvolvimento.

1.2 Quem Define os Requisitos

Requisitos têm múltiplas fontes: clientes, usuários, autoridades regulatórias (como a ANAC, no caso da aviação civil), a própria organização e o projeto, que impõe restrições de prazo e orçamento. É importante notar a diferença entre papéis: o *cliente* (customer) costuma ser a empresa que adquire o software; dentro dela, uma pessoa específica tem autoridade para supervisionar o produto; e o *usuário* é quem efetivamente opera o sistema. Essas três figuras nem sempre coincidem.

1.3 Categorias de Requisitos

O SWEBOK organiza requisitos de software em uma hierarquia. No topo estão os requisitos de software, subdivididos em **requisitos de projeto** (ou de processo, associados a tempo, escopo e custo, tratados em outras áreas do SWEBOK como Gestão e Processo de Engenharia de Software) e **requisitos de produto**.

Os requisitos de produto se dividem em **funcionais** e **não funcionais**. Requisitos funcionais correspondem aos algoritmos e regras de negócio — o que o sistema deve fazer, independentemente de tecnologia (por exemplo, "realizar matrícula" em um sistema acadêmico). Requisitos não funcionais dizem respeito a qualidade e tecnologia, subdividindo-se em **restrições tecnológicas** (linguagem de programação, banco de dados, plataforma, escalabilidade) e **qualidade de serviço** (usabilidade, manutenibilidade, confiabilidade, disponibilidade, desempenho). Uma heurística útil: praticamente todo termo terminado em "-idade" indica um requisito não funcional. As normas ISO/IEC 25010, 25012 e 25019 detalham exaustivamente esses atributos de qualidade — e são particularmente relevantes para esta disciplina, já que Safety e Security são, eles próprios, requisitos não funcionais.

Uma analogia útil para diferenciar as duas categorias é o "filtro da tecnologia perfeita": imagine um supercomputador sem limitação de memória, processamento ou custo. Tudo que ainda precisaria ser especificado nesse cenário ideal — as regras de negócio, os algoritmos — é requisito funcional. A própria máquina perfeita (desempenho ilimitado, sem falhas) é o requisito não funcional.

Por que categorizar? Porque a categorização facilita a gestão via "dividir para conquistar": clientes de negócio entendem melhor requisitos funcionais (regras de negócio), enquanto desenvolvedores lidam melhor com requisitos de tecnologia. Cada stakeholder ataca a área em que tem mais conhecimento.

Outra distinção relevante é entre **requisitos de sistema** e **requisitos de software**. Segundo o INCOSE, sistema é uma combinação de elementos — hardware, software, firmware, pessoas, informação, técnicas — que interagem de forma organizada para atingir objetivos. O software é apenas parte de um sistema mais amplo (pense em um veículo autônomo). Alguns requisitos de software são **derivados** de requisitos de sistema: são impostos pelo time de desenvolvimento porque são importantes para a ar-

quietude e a estabilidade da solução, ainda que não tragam valor perceptível imediato para o stakeholder de negócio — um conceito recorrente na indústria aeronáutica.

1.4 Atividades de Requisitos de Software

O SWEBOK organiza as atividades de requisitos em duas grandes frentes: **desenvolvimento de requisitos** e **gestão de requisitos**. O desenvolvimento de requisitos compreende quatro etapas: *elicitação* (a investigação inicial, o primeiro levantamento), *análise* (o aprofundamento e detalhamento), *especificação* (a documentação formal) e *validação* (a verificação de que os requisitos atendem à real necessidade do cliente). Já a gestão de requisitos cuida do controle de alterações ao longo do tempo, incluindo atividades como o *scrubbing* — “escovar” os requisitos para deixá-los claros e concisos — e a manutenção de rastreabilidade entre o que foi implementado e o que foi especificado.

Uma síntese útil: desenvolver requisitos é chegar a um acordo sobre o que o software deve ser; gerenciar requisitos é manter esse acordo ao longo do ciclo de vida.

Vale notar que, ao contrário do modelo em cascata — no qual os requisitos são levantados por completo antes de qualquer outra fase —, tanto o RUP (Rational Unified Process, hoje da IBM) quanto os métodos ágeis tratam requisitos de forma iterativa e incremental. No RUP, a maior parte dos requisitos de alto nível é levantada nas fases de *Inception* e *Elaboration*, mas o detalhamento continua ao longo do projeto. Em métodos ágeis, a técnica mais usada são as histórias de usuário, que não constituem uma especificação formal, mas um convite à comunicação contínua entre equipe e cliente; requisitos são detalhados somente quando prestes a ser implementados, conforme a priorização do cliente.

1.5 Elicitação de Requisitos

A elicitação — também chamada captura, descoberta ou aquisição de requisitos — é a investigação inicial que busca trazer à tona os requisitos candidatos de um software. Seu principal benefício é minimizar a incompletude, ainda que nunca seja possível garantir que todos os requisitos foram levantados.

As fontes de elicitação são, essencialmente, os **stakeholders**: qualquer pessoa, grupo ou organização que tenha interesse ou influência sobre o projeto — clientes, usuários, especialistas de domínio, órgãos regulatórios, desenvolvedores.

Existem diversas técnicas de elicitação. As mais tradicionais são entrevistas, reuniões e sessões de *brainstorming*. Workshops estruturados como JAD e JRP reúnem múltiplos stakeholders sob a condução de um moderador experiente para levantar o maior número possível de requisitos, reduzindo redundância e ambiguidade. Outras técnicas incluem análise de protocolo, grupos focais, questionários e *surveys*, prototipação (de baixa ou alta fidelidade), *User Story Mapping* (em métodos ágeis) e *Journey Map* (na área de UX). É possível também elicitar requisitos analisando versões anteriores do próprio software ou de concorrentes (benchmarking competitivo), examinando bancos de dados de defeitos já identificados, pesquisando especificações disponíveis na literatura, observando o negócio em operação, atuando como aprendiz na própria rotina do usuário, analisando cenários de uso, decompondo capacidades em épicas e

depois em histórias de usuário, ou consultando normas ISO/IEC e regulações governamentais aplicáveis ao domínio.

Síntese do Capítulo

- O SWEBOK organiza a engenharia de software em áreas de conhecimento, sendo Requisitos de Software uma das mais centrais para esta disciplina.
- Um requisito é uma capacidade necessária ao usuário ou exigida do sistema; incompletude e ambiguidade são as duas causas mais comuns de fracasso na documentação de requisitos.
- Requisitos se dividem em funcionais (algoritmos, regras de negócio) e não funcionais (qualidade e restrições tecnológicas), além da distinção entre requisitos de produto e de projeto.
- Requisitos de software fazem parte de um sistema mais amplo; requisitos derivados nascem de necessidades de arquitetura, não diretamente do negócio.
- O desenvolvimento de requisitos (elicitación, análise, especificação, validação) e a gestão de requisitos (controle de alterações, scrubbing, rastreabilidade) formam as duas grandes frentes de trabalho.
- A elicitação é a etapa inicial de levantamento, apoiada por múltiplas técnicas e fontes, e seu objetivo é minimizar — sem jamais eliminar por completo — a incompletude dos requisitos.

Capítulo 2

SWEBOK: Requisitos de Software (Parte 2) — Elicitação e Análise

Uma vez elicitados, os requisitos precisam ser aprofundados: essa é a atividade de **análise de requisitos**. Este capítulo trata das propriedades desejáveis de requisitos individuais e de coleções de requisitos, do trade-off entre qualidade e custo, da análise formal e da resolução de conflitos — para então avançar à **especificação de requisitos**, isto é, às técnicas de documentação propriamente ditas.

2.1 Qualidade de Requisitos Individuais e da Coleção

Cada requisito levantado deveria satisfazer certas propriedades. Deve ser **não ambíguo**, interpretável de uma única maneira; **testável de forma quantificável**, para que se possa verificar se a implementação o atende; e deve agregar **valor** real ao cliente — um requisito pelo qual o cliente esteja disposto a pagar. Deve também representar de fato as necessidades dos stakeholders, usar o vocabulário desses stakeholders (evitando termos técnicos como "singleton" ou "lazy loading" ao entrevistar especialistas de negócio) e ser compartilhável e verificável por diferentes interessados.

Quando se considera a **coleção** de requisitos como um todo, outras propriedades entram em jogo: ela deve ser a mais **completa** possível (embora nunca haja garantia absoluta de completude), **consistente internamente** (sem requisitos que se contradigam) e **consistente externamente** (sem conflito com legislações ou normas aplicáveis). A coleção deve ainda ser **viável** dentro das restrições de orçamento, prazo e equipe — daí a importância de decompor requisitos em partes menores e independentes, permitindo que o cliente priorize apenas o que realmente precisa.

Uma armadilha comum é capturar, na etapa de elicitação, uma solução em vez do problema real. A técnica dos **Cinco Porquês** (*Five Whys*), emprestada da gestão de riscos, ajuda a verificar a real importância de um requisito: pergunta-se repetidamente "por quê" até chegar a uma causa raiz. Na prática, raramente é preciso chegar à quinta pergunta — normalmente na segunda ou terceira já fica claro se o requisito é essencial (por exemplo, quando a resposta indica perda de vantagem competitiva caso não seja implementado).

2.2 Qualidade de Serviço, Custo e Análise Formal

Requisitos não funcionais de qualidade de serviço envolvem um trade-off entre nível de performance e valor agregado ao cliente. Existe um "ponto de nível de performance

mais favorável”: além dele, investir mais em performance não traz mais valor perceptível ao cliente — é desperdício de recurso. Do lado oposto, níveis muito baixos de performance também não agregam valor algum. Complementarmente, ao se considerar também o custo de implementação, existe um ponto de melhor relação custo-benefício: a partir dele, pequenos ganhos de performance implicam aumentos desproporcionais de custo, tornando o investimento adicional pouco vantajoso.

Uma escola de pensamento distinta trabalha com **análise formal de requisitos**: documentar requisitos usando linguagens formais (como a linguagem Z), o que permite reduzir ambiguidade e, em alguns casos, provar matematicamente a corretude do requisito. O custo é a curva de aprendizado exigida e a dificuldade de leitura para não especialistas — por isso essa abordagem é tratada com mais profundidade em outra área do SWEBOK, dedicada a métodos formais.

Por fim, como múltiplos stakeholders têm interesses distintos, conflitos de requisitos são praticamente inevitáveis em qualquer projeto. Cabe ao analista de negócio ou engenheiro de requisitos negociar soluções. Uma técnica útil é o **Product Family Development**: o design do sistema acomoda, de um lado, os requisitos que não variam (o consenso) e, de outro, prevê variabilidade para os requisitos conflitantes — de maneira análoga a “programar para uma interface” em orientação a objetos, permitindo trocar implementações via polimorfismo.

2.3 Especificação de Requisitos: Linguagem Natural

A **especificação de requisitos** é a etapa de documentação, cujo objetivo é preservar o conhecimento levantado e facilitar sua comunicação com os interessados no projeto. O SWEBOK descreve diversas técnicas.

A forma mais tradicional é a **linguagem natural não estruturada**, do tipo “o sistema deverá fazer isso— muito comum, por exemplo, em sistemas embarcados aeronáuticos. Já as **regras de negócio** definem ou restringem algum aspecto do comportamento do sistema (por exemplo: “um estudante não pode se matricular em disciplinas do próximo semestre se houver taxa não paga— típica de instituições privadas).

A **linguagem natural estruturada** adiciona um template que reduz ambiguidade. O padrão mais simples é “ator realiza ação”, geralmente disparada por um evento e sujeita a condições. Os **casos de uso** seguem esse formato, documentando ator principal, gatilho, fluxo principal e alternativo. As **histórias de usuário**, típicas de métodos ágeis, seguem o template “enquanto [ator], eu quero [capacidade], de forma que [benefício]— a técnica mais usada atualmente para identificação de requisitos, embora praticamente ausente do setor aeronáutico, onde predominam listas de features e casos de uso.

2.4 Especificação Baseada em Critérios de Aceitação

Uma abordagem particularmente valiosa liga requisitos a **casos de teste**: um caso de teste do tipo "dada uma entrada X, esperamos que o sistema produza Y" é, na prática, um requisito reescrito. Essa proximidade justifica duas técnicas:

O **ATDD** (*Acceptance Test-Driven Development*) é uma variação do TDD (*Test-Driven Development*, técnica do Extreme Programming) em nível mais alto: em vez de guiar a implementação de um método, guia a implementação de uma história de usuário inteira. O processo tem três passos: seleciona-se a história de usuário; o time (desenvolvedores, especialistas de domínio, qualidade) define os casos de teste de aceitação em conjunto; roda-se o teste (falha esperada, "barra vermelha"); implementa-se até passar ("barra verde"); refatora-se; repete-se.

O **BDD** (*Behavior-Driven Development*) usa a sintaxe **Gherkin** — *Given, When, Then* (dado, quando, então) — para escrever cenários de aceitação a partir de uma história de usuário. Por exemplo: dado que o saldo é \$500 e o cartão é válido, quando o correntista solicita \$100, então o caixa deve liberar \$100 e o saldo deve cair para \$400. Frameworks como o Cucumber automatizam esses cenários diretamente. A grande vantagem do BDD é reduzir ambiguidade e tornar o requisito imediatamente testável — e essa técnica tem aplicação prática também em verificação de requisitos de segurança, tema de pesquisas recentes na área.

2.5 Modelos Gráficos e Formas de Versionamento

Requisitos também podem ser especificados via **modelos gráficos**, como o UML (para engenharia de software) e o SysML (para engenharia de sistemas, ambos mantidos pela OMG). Esses modelos se dividem em **estruturais** (diagrama de classe, modelo entidade-relacionamento) e **comportamentais** (caso de uso, diagrama de sequência, diagrama de atividade). Historicamente, técnicas de análise estruturada (não orientada a objetos), como o DFD (Diagrama de Fluxo de Dados), cumpriram papel semelhante antes da consolidação da orientação a objetos.

Quanto mais formal o modelo, menor a ambiguidade, mais fácil a tradução para código (por vezes automática) e mais simples a derivação de casos de teste — ao custo de maior esforço para produzir e interpretar a especificação. Para dar estrutura sem chegar ao formalismo pleno, é comum atribuir atributos a cada requisito: identificador único, descrição, fonte, tipo (funcional/não funcional), dependências e conflitos.

Por fim, a especificação pode ser **incremental** (cada versão documenta apenas as diferenças em relação à anterior, gerando menor volume) ou **compreensiva** (cada versão é completa e autocontida, facilitando a consulta). Na prática, é comum um modelo híbrido, com incrementos menores (1.1, 1.2...) consolidados periodicamente em versões completas (2.0, 3.0...).

Síntese do Capítulo

- Requisitos individuais devem ser não ambíguos, testáveis, valiosos e fiéis ao vocabulário dos stakeholders; a coleção deve ser completa, consistente e viável.
- A técnica dos Cinco Porquês ajuda a validar se um requisito é realmente necessário.
- Existe um ponto ótimo de custo-benefício entre performance e valor entregue; além dele, investir mais deixa de compensar.
- Especificação em linguagem natural pode ser não estruturada, estruturada (casos de uso, histórias de usuário) ou baseada em critérios de aceitação (ATDD, BDD/Gherkin), esta última aproximando requisitos de casos de teste automatizáveis.
- Modelos gráficos (UML, SysML) e diferentes graus de formalismo oferecem trade-offs entre precisão e facilidade de comunicação.
- Especificações podem ser incrementais, compreensivas ou híbridas, dependendo da cultura e das necessidades do projeto.

Capítulo 3

SWEBOK: Requisitos de Software (Parte 3) — Especificação e Validação

Encerrando a área de conhecimento Requisitos de Software do SWEBOK, este capítulo trata da **validação de requisitos**, das **atividades de gestão de requisitos**, de considerações práticas do processo e das **ferramentas** de apoio a todo esse ciclo.

3.1 Validação de Requisitos

Uma distinção fundamental abre este tema: **verificação** é assegurar-se de que você está fazendo a coisa de maneira correta; **validação** é assegurar-se de que você fez a coisa correta. Validação de requisitos, portanto, é garantir que os requisitos documentados realmente representam as necessidades dos stakeholders — avaliando se são relevantes, se estão bem declarados, se são compreensíveis, consistentes, completos e conformes a padrões da área.

Validação de Requisitos

Validação de requisitos é o processo de assegurar que os requisitos levantados e documentados de fato representam as reais necessidades dos stakeholders, respondendo a perguntas como: os requisitos são relevantes? Estão bem escritos e documentados? São compreensíveis, consistentes e completos? Estão em conformidade com os padrões aplicáveis ao domínio?

O SWEBOK descreve três métodos principais de validação. As **revisões de requisitos** são a técnica mais comum: revisores experientes examinam o documento de especificação em busca de omissões, requisitos mal declarados, suposições inválidas, falta de clareza ou desvios de práticas aceitas (como legislação aplicável). Quanto mais perspectivas e papéis diferentes participarem da revisão — clientes, usuários, engenheiros de requisitos, arquitetos —, melhor tende a ser o resultado.

A **simulação e execução** complementa a revisão quando o documento é longo demais para uma leitura minuciosa por stakeholders não técnicos. Consiste em percorrer um cenário (por exemplo, o cenário de um caso de uso) e verificar, junto ao stakeholder, se aquela narrativa corresponde à real necessidade do negócio.

A **prototipação** é útil quando os requisitos não podem ser validados apenas de forma textual — especialmente em interfaces gráficas, onde o cliente interage com um protótipo (wireframe) e fornece feedback direto. Um cuidado importante: protótipos de alta fidelidade, por parecerem quase o produto final, tendem a intimidar o cliente e

reduzir o feedback recebido; protótipos de baixa fidelidade, justamente por parecerem rascunhos, costumam gerar mais contribuições úteis.

3.2 Gestão de Requisitos

Enquanto o desenvolvimento de requisitos busca um acordo sobre o que o software deve fazer, a **gestão de requisitos** mantém esse acordo ao longo do tempo. Ela se organiza em três tópicos.

O **requirements scrubbing** ("escovação de requisitos") é a prática de examinar cuidadosamente a especificação para eliminar requisitos complexos, redundantes ou que não agregam valor, mantendo apenas o menor subconjunto necessário para atender às necessidades dos stakeholders — reduzindo assim complexidade, custo e cronograma. Em processos baseados em planejamento (cascata), o scrubbing normalmente antecede a validação formal; em métodos ágeis, ele já está embutido na Reunião de Planejamento de Sprint do Scrum, quando o time refina e prioriza os itens do Product Backlog.

O **controle de alterações de requisitos** é indispensável porque requisitos mudam continuamente. Em processos de ciclo de vida em cascata, esse controle costuma ser explícito e rígido: a especificação de requisitos muitas vezes vira contrato com o cliente, e qualquer alteração exige análise formal de impacto antes de ser aceita, rejeitada ou adiada. Em métodos ágeis, o controle é mais implícito: a solicitação de mudança simplesmente retorna ao Product Backlog para ser repriorizada, refletindo o espírito do subtítulo do livro clássico de Kent Beck sobre Extreme Programming — "Embrace the Change" (acolha as mudanças).

O **scope matching** (casamento de escopo) garante que o conjunto de requisitos não ultrapasse as restrições de custo, cronograma ou equipe disponível. Se o escopo excede a capacidade, as opções são reduzir o escopo (priorizando o que traz mais valor) ou aumentar a capacidade (mais tempo ou mais pessoas) — esta última com uma ressalva importante: a Lei de Brooks afirma que adicionar pessoas a um projeto já atrasado tende a atrasá-lo ainda mais, pois o tempo das pessoas mais produtivas é consumido treinando as recém-chegadas. Em processos baseados em plano, o scope matching é feito antes da validação; em métodos ágeis, ele é automático, já que o time só aceita no Sprint o volume de histórias compatível com sua velocidade histórica (medida, por exemplo, em pontos de história via Planning Poker).

3.3 Considerações Práticas

O processo de requisitos é, por natureza, **iterativo**: a elicitação inicial tende a ser "em largura" (uma visão ampla do sistema), enquanto análise e especificação aprofundam "em profundidade" os tópicos prioritários — um padrão evidente em métodos ágeis, onde requisitos só são detalhados quando próximos de serem implementados.

A **priorização de requisitos** orienta o que deve ser detalhado e implementado primeiro. Um critério isolado de "valor para o cliente" não é suficiente: o Modelo Kano, por exemplo, mostra que também é preciso considerar o quanto a ausência de um requisito frustraria o cliente. O Processo Unificado, por sua vez, prioriza frequentemente por risco arquitetural, e não apenas por valor percebido. Formas de comunicar a pri-

orização vão de escalas simples (“deveria ter”, “gostaria de ter”) a valores numéricos ou à própria posição no Product Backlog.

A **rastreabilidade de requisitos** permite navegar de um requisito até as decisões de design, teste e implementação que o satisfazem — e o caminho inverso, de um trecho de código até o requisito que o justifica. Isso é essencial tanto para garantir que nada foi implementado sem necessidade quanto para realizar análise de impacto diante de qualquer alteração.

Por fim, a **volatilidade e estabilidade** de requisitos reconhece que alguns requisitos raramente mudam (regras de negócio consolidadas), enquanto outros mudam com frequência (por exemplo, por força de nova legislação). Técnicas de design que desacoplam interface de implementação — como “programar para uma interface— tornam o software mais resiliente a essas mudanças.

3.4 Ferramentas de Requisitos

O SWEBOK descreve três categorias de ferramentas de apoio. As **ferramentas de gestão de requisitos** tornam-se indispensáveis em projetos grandes e complexos — planilhas ou documentos de texto não sustentam o volume de atributos, rastreabilidade e controle de alterações necessário (a indústria aeroespacial, por exemplo, recorre a ferramentas robustas como o IBM Rational DOORS). As **ferramentas de modelagem de requisitos** apoiam a documentação gráfica (diagramas UML de caso de uso, por exemplo) e ajudam a garantir a corretude sintática dos modelos. Já as **ferramentas de geração de casos de teste** permitem derivar testes automatizados diretamente dos requisitos — como frameworks BDD (a exemplo do Cucumber), que traduzem cenários Gherkin em código de teste, ou ferramentas que geram casos de teste a partir de máquinas de estado. Essa integração entre requisitos e geração automática de testes é, inclusive, tema de pesquisas atuais aplicadas à verificação de requisitos de segurança.

Síntese do Capítulo

- Verificação é “fazer certo a coisa”; validação é “fazer a coisa certa— e a validação de requisitos garante que eles refletem as reais necessidades dos stakeholders.
- Revisões, simulação/execução e prototipação são os três métodos clássicos de validação, cada um mais adequado a um contexto de complexidade e formato de documentação.
- Gestão de requisitos compreende scrubbing (eliminar o desnecessário), controle de alterações e scope matching (ajustar escopo às restrições de custo, prazo e equipe).
- A Lei de Brooks alerta que adicionar pessoas a um projeto atrasado tende a atrasá-lo ainda mais.
- Priorização, rastreabilidade e gestão de volatilidade são considerações práticas essenciais ao longo de todo o ciclo de vida dos requisitos.

-
- Ferramentas de gestão, modelagem e geração de casos de teste tornam-se indispensáveis à medida que a complexidade do sistema cresce.

Capítulo 4

Introdução à Engenharia de Sistemas e a Certificação INCOSE

Este capítulo apresenta os fundamentos da engenharia de sistemas: sua definição, seu modelo de ciclo de vida mais conhecido (o modelo V), o papel do INCOSE como órgão normatizador da área e o caminho de certificação profissional que ela oferece. O objetivo é situar o leitor no panorama da disciplina antes de aprofundar, nos capítulos seguintes, a engenharia de requisitos nesse contexto mais amplo.

4.1 O que é um Sistema e o que é Engenharia de Sistemas

De acordo com o INCOSE, um sistema é um arranjo de partes ou elementos que, coletivamente, exibem comportamento ou significado que os constituintes individuais não exibem isoladamente — o sistema é mais do que a soma de suas partes.

A norma ISO/IEC 15288:2023, que governa a área, define engenharia de sistemas como uma abordagem integrativa e transdisciplinar que habilita a realização, o uso e a aposentadoria bem-sucedidos de sistemas de engenharia, empregando conceitos e princípios de sistemas e métodos de gestão, tecnológicos e científicos. Essa mesma definição é adotada pelo *Systems Engineering Handbook*, livro de referência da área e base da prova de certificação ASEP, discutida adiante.

Engenharia de Sistemas

Engenharia de sistemas é uma abordagem integrativa, multidisciplinar e transdisciplinar que habilita a realização, o uso e a aposentadoria de sistemas complexos, combinando conceitos e princípios de sistemas com métodos de gestão, tecnológicos e científicos. É interdisciplinar por natureza, pois sistemas sociotécnicos complexos envolvem software, hardware, pessoas e instituições.

O modelo de ciclo de vida mais utilizado na área é o modelo V (nome derivado de seu formato gráfico), descrito em detalhe no *Systems Engineering Handbook*. Trata-se de uma diretriz a ser instanciada para o sistema particular em desenvolvimento, não uma receita rígida. A própria disciplina de modelos e técnicas para análise de requisitos de *safety* e *security* utiliza uma instancição do modelo V no *STPA Handbook*, que parte do desenvolvimento conceitual até questões de operação, manutenção e evolução dos sistemas.

4.2 O INCOSE e as Carreiras em Engenharia de Sistemas

O INCOSE (*International Council on Systems Engineering*) é o conselho internacional responsável por ditar as normas da área. A entidade promove simpósios anuais para discutir o estado da arte e o futuro da engenharia de sistemas, além de planejamento estratégico de longo prazo — seu documento de visão para 2035 apresenta objetivos, indicadores e metas para a área.

Por ser interdisciplinar, a área demanda profissionais de diversas formações: engenharia mecânica, elétrica, química, ambiental e, de particular interesse para esta disciplina, engenharia de segurança (*safety*). O próprio site do INCOSE destaca que o engenheiro de sistemas atua como a principal interface entre gestão, clientes, fornecedores e engenheiros especializados nas diversas áreas do desenvolvimento de sistemas — exigindo, portanto, profissionais com visão holística de todo o ciclo de vida.

4.3 As Certificações do INCOSE

O INCOSE oferece três níveis de certificação profissional: **ASEP** (*Associate Systems Engineering Professional*), prova de entrada que não exige experiência prévia; **CSEP** (*Certified Systems Engineering Professional*); e **ESEP** (*Expert Systems Engineering Professional*) — estas duas últimas condicionadas à comprovação de experiência na área.

Para profissionais de Ciência da Computação, o caminho natural de entrada é o ASEP, tanto por não exigir experiência quanto por contar com um guia de estudo dedicado, o *Systems Engineering Handbook* (5ª edição, 2023). Para obtê-la, o candidato deve ser membro registrado do INCOSE, inscrever-se mediante taxa e, a partir daí, tem 12 meses e até três tentativas para realizar o exame — cerca de 120 questões de múltipla escolha em 120 minutos (um minuto por questão em média). Algumas questões extras não contam para a pontuação e servem apenas para validar itens de provas futuras.

4.4 O Systems Engineering Handbook

O *Systems Engineering Handbook* é o guia de conhecimento (*Body of Knowledge*) oficial da área, orientado a atividades e processos do ciclo de vida de desenvolvimento de sistemas — referência básica para quem estuda para o ASEP. Sua estrutura cobre: introdução à engenharia de sistemas; os processos técnicos que compõem o modelo V (fortemente baseados na ISO/IEC 15288 e organizados segundo a lógica IPO — *Input, Process, Output* —, mesma lógica usada pelo PMBOK e por métodos de análise de *hazards* como o PHA); métodos e ciclo de vida, incluindo as subseções sobre engenharia de *safety* e de *security* de sistemas, centrais para esta disciplina; customizações e aplicações em diversos domínios, como *Agile* e *Lean Systems Engineering* — este último influenciou práticas ágeis de software como o Kanban; relacionamento com engenharia de software, hardware, gestão de projetos e pesquisa operacional; estudos de caso,

incluindo *cybersecurity* e inteligência artificial em veículos autônomos; e apêndices com glossário de termos da área.

Síntese do Capítulo

- Sistema é um arranjo de partes que, coletivamente, exibe comportamento que os componentes isolados não exibem; engenharia de sistemas é a abordagem integrativa e transdisciplinar que viabiliza sua realização, uso e aposentadoria.
- O modelo V é o modelo de ciclo de vida mais utilizado na área e será instanciado, mais adiante, pelo STPA Handbook.
- O INCOSE é o conselho internacional que normatiza a área, promove eventos e planejamento estratégico de longo prazo (visão 2035).
- A engenharia de sistemas é interdisciplinar por natureza e demanda profissionais de múltiplas formações, incluindo engenharia de *safety*.
- O INCOSE certifica profissionais em três níveis — ASEP, CSEP e ESEP — , sendo o ASEP a porta de entrada acessível a quem não possui experiência prévia.
- O *Systems Engineering Handbook* é o guia de estudo oficial para o ASEP, estruturado em processos segundo a lógica Input-Process-Output.

Capítulo 5

Engenharia de Requisitos no Contexto de Engenharia de Sistemas (Parte 1)

Este capítulo desloca o foco da engenharia de software para o campo mais amplo da engenharia de sistemas, que engloba software, pessoas, hardware, instituições e demais entidades associadas a sistemas sociotécnicos complexos. O objetivo é mostrar como os conceitos de necessidade e requisito têm origem nesse campo mais abrangente e como se relacionam com os diferentes níveis organizacionais de um empreendimento.

5.1 De Necessidades a Requisitos

Antes de existir o conceito de requisito, existe o conceito de necessidade. Necessidades são capacidades declaradas na linguagem da gestão de negócio ou dos stakeholders, em nível de operação de negócio — não de tecnologia. Requisitos são derivados das necessidades, mas são declarações mais formais e estruturadas, passíveis de validação. É por isso que a definição usual de requisito em engenharia de software fala em "capacidade": um requisito é uma capacidade que o sistema deverá atender para que o usuário se sinta satisfeito — noção que vem diretamente da engenharia de sistemas. Pode existir mais de um requisito associado a uma mesma necessidade; a transformação de uma na outra ocorre por meio de atividades de análise de requisitos, de negócio ou de missão, desempenhadas por profissionais que fazem a ponte entre a linguagem de negócio e a técnica.

Necessidade vs. Requisito

Necessidades são capacidades declaradas na linguagem de negócio dos stakeholders, em nível de operação — descrevem o que o problema resolve para alguém. Requisitos são derivados das necessidades, porém formalizados e estruturados de modo a serem validáveis e implementáveis por uma equipe de desenvolvimento. Uma mesma necessidade pode gerar múltiplos requisitos.

5.2 A Hierarquia de Necessidades e Requisitos na Organização

É útil visualizar essa transformação ao longo dos níveis organizacionais, do empreendimento global até o sistema e seus subsistemas. Em cada nível há uma visão de necessidades (domínio do problema — o que o sistema faz e para quem) e uma visão

de requisitos correspondente (domínio da solução — como isso é feito, em software ou hardware).

Análise de missão e de negócio transformam necessidades de gestão de negócio em requisitos de negócio, documentados na *Business Requirements Specification* (BRS). A análise de requisitos transforma necessidades de stakeholders em requisitos de stakeholders (*Stakeholder Requirement Specification*, STRS). Necessidades no escopo do sistema tornam-se requisitos de sistema (*System Requirement Specification* — SRS no Processo Unificado, SYRS na engenharia de sistemas, o mesmo tipo de artefato), e necessidades de subsistema geram uma *Subsystem Requirement Specification*. Na prática, é comum usar ferramentas de gestão de requisitos em vez de documentos de texto isolados.

5.3 Conceitos de Ciclo de Vida

Assim como o desenvolvimento de software trabalha com modelos de ciclo de vida (cascata, iterativo-incremental, espiral), a engenharia de sistemas organiza seu ciclo de vida em torno de conceitos-chave:

- **OpsCon/CONOPS** — o CONOPS é o documento de conceito de operação; o OpsCon é mais amplo e descreve o que o sistema fará e por que, sob a perspectiva do usuário.
- **Aquisição** — como o sistema será obtido (interna ou externamente), envolvendo stakeholders, contratos, design e verificação.
- **Deployment** — validação, entrega e colocação em operação, análogo à implantação contínua em software.
- **Suporte** — infraestrutura e mão de obra para manter o sistema após a entrega.
- **Aposentadoria (*retirement*)** — remoção de operação, incluindo descarte de materiais perigosos (ex.: o plugin Flash, aposentado pela Adobe).

5.4 Classificação e Justificativa de Requisitos

Como na engenharia de software, requisitos se classificam em funcionais (o que o sistema deve fazer) e não funcionais (qualidade, desempenho e restrição — disponibilidade, escalabilidade, usabilidade). Restrições como "o relatório deve ser emitido em até 30 segundos" ou "deve ser desenvolvido em Java" costumam ser requisitos não funcionais, embora, a depender do contexto, também possam ser tratadas como funcionais.

Requisitos de negócio e de stakeholders são menos formais que requisitos de sistema, já prontos para desenvolvimento. Todo requisito deve ter um *rationale* — a justificativa de sua existência — sob pena de se desperdiçar recursos implementando algo desnecessário. Além de funcional/não funcional, é comum classificar requisitos com outros adjetivos: operacionais, de stakeholder, ambientais, de interface, de órgãos

reguladores, de *safety* e de *security* — estes dois últimos, não funcionais, são o foco desta disciplina.

Vale destacar papéis centrais: **usuário** (ou ator), quem utiliza o sistema; **cliente**, a organização que paga pelo desenvolvimento, distinta do **adquirente** (*acquirer*); **desenvolvedor**, quem transforma requisitos em sistema; **contratante** (*contractor*), organização responsável pelo desenvolvimento mediante contrato; e **stakeholders**, todas as partes com interesse no sistema.

Diante disso, engenharia de requisitos, no contexto de engenharia de sistemas, é o conjunto de atividades, métodos e processos que transformam necessidades em requisitos validáveis e implementáveis, produzindo um sistema que atenda às necessidades de seus clientes. Ela existe porque é preciso garantir que os requisitos não sejam ambíguos — ambiguidade pode gerar, em operação, perdas inaceitáveis, incluindo danos ambientais, financeiros ou perda de vidas — e porque é preciso negociar custo-benefício entre requisitos conflitantes e controlar alterações de escopo ao longo do ciclo de vida.

5.5 Propriedades Emergentes

Um conceito importante para esta disciplina é o de propriedade emergente: aquela que surge exclusivamente da interação entre os componentes de um sistema, e não pode ser observada em nenhum componente isolado — o todo é maior que a soma das partes, ao contrário da lógica de “dividir para conquistar” comum em computação.

Um exemplo didático é a bicicleta: guidão, quadro, freio e selim, isoladamente, não pedalam — é na interação entre esses componentes e o ciclista que emerge “estar em movimento”. Requisitos não funcionais — manutenibilidade, confiabilidade, usabilidade, *safety* e *security* — são exemplos típicos: só existem a partir da integração dos elementos do sistema, incluindo sua interação com o ambiente.

Por isso, propriedades emergentes são definidas de cima para baixo (*top-down*): não é possível obtê-las especificando requisitos de elementos individuais isoladamente. Um sistema completo pode ainda exibir propriedades emergentes não previstas no design — por vezes indesejáveis.

Síntese do Capítulo

- Necessidade é uma capacidade declarada na linguagem de negócio; requisito é sua formalização, validável e implementável.
- A transformação de necessidade em requisito ocorre em vários níveis organizacionais — negócio, stakeholders, sistema, subsistema —, cada um com seu documento característico (BRS, STRS, SYRS, Subsystem Requirement Specification).
- Conceitos de ciclo de vida como OpsCon/CONOPS, aquisição, *deployment*, suporte e aposentadoria estruturam a vida de um sistema do início ao fim.
- Requisitos se classificam em funcionais e não funcionais, e todo requisito deve ter uma justificativa (*rationale*) clara para sua existência.

- Papéis como usuário, cliente, adquirente, desenvolvedor, contratante e stakeholder têm significados distintos dentro do ciclo de vida de engenharia de sistemas.
- Propriedades emergentes — como *safety* e *security* — surgem apenas da interação entre componentes, nunca de elementos isolados, exigindo especificação *top-down*.

Capítulo 6

Engenharia de Requisitos no Contexto de Engenharia de Sistemas (Parte 2)

Dando sequência à discussão sobre necessidades e requisitos, este capítulo trata da elicitación e elaboração de requisitos, da hierarquia e rastreabilidade de requisitos, das técnicas e ferramentas de apoio, e dos conceitos de verificação, validação e gestão de mudanças.

6.1 Hierarquia, Elicitación e Elaboração de Requisitos

Uma forma consagrada de visualizar requisitos é como hierarquia: no topo, requisitos genéricos, no nível da missão do sistema; à medida que se desce, requisitos cada vez mais granulares, prontos para desenvolvimento. Essa decomposição ocorre por duas atividades. A **elicitación** é a coleta inicial dos requisitos, mais em largura do que em profundidade. A **elaboração** envolve análise e detalhamento — por decomposição ou derivação — até o nível hierárquico mais fino. Requisitos derivados não trazem valor aparente para os stakeholders de negócio, mas os projetistas sabem serem essenciais ao sucesso do produto. Como em software, análise de requisitos não é atividade para profissionais inexperientes: exige conhecimento do negócio, do domínio e de aspectos técnicos de desenvolvimento — trabalho de engenheiros de requisitos e analistas de negócio.

Um exemplo hipotético ilustra a decomposição: a missão de um "Centro Médico ACME" — fornecer serviços médicos em ambiente seguro e confortável, obtendo lucro — decompõe-se em fornecer serviços médicos, ambiente seguro, ambiente confortável e obter lucro. Funções adicionais, como infraestrutura e gestão do centro médico, não derivam diretamente da missão, mas são necessárias do ponto de vista de design — são requisitos derivados, muitas vezes identificados só por profissionais experientes.

Hierarquia de Requisitos e Rastreabilidade

Requisitos formam uma hierarquia que vai do nível de missão (alto nível, abstrato) até requisitos de granularidade fina, prontos para implementação. Decompor requisitos de forma hierárquica — numerando-os como 1, 1.1, 1.1.1 — fornece automaticamente **rastreabilidade**: sabe-se que um requisito de nível inferior deriva de um superior, e uma alteração em um requisito de alto nível exige análise de impacto sobre tudo que dele deriva.

Há diversas formas de identificar necessidades e requisitos: *workshops*, *brainstorming*, entrevistas, questionários, casos de uso, cenários de operação, simulação, protótipos, observação da operação de sistemas existentes, revisão de documentação técnica e análise competitiva (*benchmarking*) frente a soluções de concorrentes.

6.2 Verificação, Validação e Testes ao Longo do Ciclo de Vida

Verificação e validação são centrais também na engenharia de sistemas. **Verificação** é construir certo a coisa: confirmar, durante o desenvolvimento, que o sistema está de acordo com a especificação de requisitos. **Validação** é construir a coisa certa: confirmar, após a entrega, que o produto atende às necessidades originais dos stakeholders. A distinção é atribuída a Barry Boehm, associado também ao modelo de ciclo de vida espiral.

Uma lição consolidada nas últimas décadas é que testes não devem ficar restritos ao final do desenvolvimento: quanto antes um defeito é identificado, mais barato é corrigi-lo — princípio incorporado pelos métodos ágeis, por exemplo através do TDD (*Test Driven Development*), em que o teste é escrito antes da solução. Deve haver testes contínuos: durante o desenvolvimento, na aceitação e na operação.

6.3 Gestão de Requisitos e Ferramentas

Requisitos mudam constantemente — estima-se que mais de 50% dos requisitos de um sistema sejam modificados antes de entrarem em operação, por razões que vão de mudanças regulatórias a simples mudança de ideia dos stakeholders. O subtítulo do livro clássico de Kent Beck sobre *Extreme Programming*, "*Embrace the Change*", já reconhecia isso: alteração de requisito costuma ser oportunidade de atender melhor o cliente.

Gestão de requisitos é o processo pelo qual as alterações são gerenciadas ao longo do ciclo de vida do sistema. Em sistemas complexos, essa gestão não é viável manualmente: é praticamente impossível manter rastreabilidade sem ferramentas de software. Métodos ágeis, como o Scrum, tratam a questão de forma mais informal, priorizando requisitos a cada *Sprint* — adequado a projetos de menor porte. Sistemas sociotécnicos complexos demandam documentação formal e ferramentas dedicadas, que apoiam navegação pela especificação, geração de relatórios, importação/exportação em múltiplos formatos, análise de impacto de alterações e alocação funcional entre sistema, subsistemas e instalações físicas.

6.4 Técnicas de Engenharia de Requisitos

Diversas técnicas apoiam a elicitación e o detalhamento de requisitos. Na engenharia de software, destacam-se o modelo de caso de uso da UML (diagramas, cenários e formulários), diagramas de comportamento (atividade, sequência, estado) e, historicamente, ferramentas da análise estruturada, como o DFD e o diagrama entidade-relacionamento.

Na engenharia de sistemas, é bastante usado o **Requirements Breakdown Structure** (RBS) — decomposição de requisitos inspirada no *Work Breakdown Structure* (WBS) da gestão de projetos. A lógica é a da hierarquia já discutida: partir de um requisito de alto nível e subdividi-lo em níveis sucessivos, obtendo rastreabilidade automática — o mesmo princípio da decomposição do *Product Backlog* do Scrum em temas, épicos, histórias e tarefas. Um exemplo clássico ilustra a técnica com um alarme residencial: do requisito "segurança doméstica", decompõe-se em impedir entrada não autorizada, detectar intrusos e classificar a entrada — cada um refinável em subitens, como ajuste de sensibilidade para reduzir alarmes falsos.

Outra técnica é o **FFBD** (*Functional Flow Block Diagrams*), que representa graficamente não só a decomposição hierárquica, mas também o fluxo entre requisitos — por exemplo, mostrando que um requisito pode ser alcançado a partir de dois outros, e detalhando-se em seguida em subitens. Essa representação explicita hierarquia e dependências funcionais, particularmente valiosa em sistemas de segurança.

Por fim, a rastreabilidade é indispensável: permite saber de onde um requisito se originou, sua relação com outros requisitos e quais artefatos de design dele derivam. Rastreabilidade para trás ajuda a identificar o *scope creep* — o crescimento desordenado do escopo do projeto: se uma funcionalidade implementada não pode ser rastreada até nenhum requisito, é sinal de que algo desnecessário está sendo construído.

Síntese do Capítulo

- Requisitos formam uma hierarquia, do nível de missão até o nível de implementação, construída pelas atividades de elicitação (coleta) e elaboração (análise e detalhamento).
- Verificação é construir certo a coisa (conformidade com a especificação); validação é construir a coisa certa (atendimento às necessidades reais dos stakeholders).
- Testes devem ocorrer ao longo de todo o ciclo de vida, não apenas ao final — quanto antes um defeito é encontrado, mais barato é corrigi-lo.
- Gestão de requisitos é indispensável porque requisitos mudam constantemente (mais de 50% antes de entrarem em operação), exigindo ferramentas de software para rastreabilidade e análise de impacto.
- Técnicas como o Requirements Breakdown Structure (RBS) e o Functional Flow Block Diagram (FFBD) apoiam a decomposição hierárquica e o mapeamento do fluxo entre requisitos.
- Rastreabilidade — para frente e para trás — é essencial para análise de impacto de mudanças e para identificar *scope creep*.

Parte II

Conceitos Básicos de Engenharia de Sistemas

Capítulo 7

Design Conceitual

Este capítulo situa o design conceitual dentro do ciclo de vida de desenvolvimento de sistemas e mostra como necessidades organizacionais se transformam nos primeiros requisitos formais de um sistema.

7.1 O Ciclo de Vida de Desenvolvimento de Sistemas

Todo sistema percorre um ciclo de vida que vai da identificação de uma necessidade inicial até sua eventual aposentadoria. Em alto nível, esse ciclo pode ser dividido em quatro grandes fases: pré-aquisição (ou pré-desenvolvimento), aquisição/desenvolvimento, utilização (o sistema em operação) e aposentadoria.

As fases de aquisição/desenvolvimento e de utilização, por sua vez, se subdividem em etapas menores. A fase de aquisição normalmente compreende: design conceitual, design preliminar, design detalhado e desenvolvimento, além da etapa de produção/construção. Já a fase de utilização se divide em suporte do sistema e uso operacional. Este capítulo trata da primeira dessas etapas — o design conceitual, também chamado de design lógico ou design do domínio do problema.

De forma resumida, no design conceitual são articuladas as necessidades que devem ser analisadas e transformadas em requisitos, documentados no *System Requirements Specification* (SRS). Aplica-se aqui o princípio de *flow down*: parte-se de uma necessidade, chega-se a requisitos de alto nível, que são progressivamente refinados em requisitos de granularidade mais fina, até estarem prontos para implementação por um time de desenvolvimento.

7.2 Lacunas de Capacidade (Capability Gaps)

Para entender de onde vêm os requisitos que alimentam o design conceitual, é preciso compreender o conceito de *capability gap* (lacuna de capacidade). Toda organização — grande ou pequena, pública ou privada — realiza planejamento estratégico de longo prazo, definindo objetivos, indicadores e metas. Ao fazer esse planejamento, é comum identificar lacunas na capacidade organizacional: desejos de longo prazo que não podem ser atingidos de imediato porque existem problemas ou limitações na operação atual.

Lacuna de Capacidade

Uma lacuna de capacidade (*capability gap*) é a discrepância entre as capacidades atuais de um sistema ou organização e as capacidades desejadas necessárias para atingir um objetivo estratégico. Enquanto essa lacuna não é resolvida, a organização deixa de explorar oportunidades e fica exposta a ameaças e à concorrência.

O conceito se aplica em diferentes escalas. Em nível pessoal: alguém pode desejar surfar, mas ter lacunas de capacidade como não possuir prancha, não ter suporte no carro para transportá-la e não ter a habilidade necessária. Em nível de negócio: uma empresa pode querer explorar um nicho de mercado percebido, mas não ter ainda a capacidade de fazê-lo. Em nível governamental: um governo pode desejar aperfeiçoar os serviços de saúde e transporte, mas esbarrar em lacunas que primeiro precisam ser resolvidas. Em nível militar: pode-se desejar manter vigilância constante nas fronteiras, mas faltar a capacidade tecnológica ou operacional para tanto.

Quando uma lacuna de capacidade é considerada suficientemente importante para justificar ações concretas, ela passa a ser tratada como um objetivo estratégico — e é exatamente nesse ponto, quando uma necessidade se associa a uma lacuna crítica e priorizada, que se inicia o ciclo de vida de engenharia de sistemas, com a etapa de design conceitual. Como normalmente várias lacunas de capacidade coexistem em uma organização, é necessário priorizá-las, atacando primeiro aquelas mais viáveis dentro de uma janela de tempo razoável.

7.3 As Cinco Atividades do Design Conceitual

Os livros-texto que fundamentam esta disciplina (*Introduction to Systems Engineering* e *Systems Engineering Practice*) detalham o design conceitual em cinco atividades principais, numeradas de C1 a C5, cada uma com atores responsáveis, artefatos de entrada e artefatos de saída.

1. **Definir necessidades e requisitos de negócio:** produz o *Business Needs and Requirements*, documentado no *Preliminary Life Cycle Concept Document* e no *Business Requirements Specification*.
2. **Definir necessidades e requisitos dos stakeholders:** produz o *Stakeholders Needs and Requirements*, também documentado no *Life Cycle Concept Document* e em uma especificação de requisitos de stakeholders.
3. **Definir requisitos do sistema:** aqui são identificados os requisitos funcionais (associados a regras de negócio e algoritmos — o que o sistema deve fazer) e os requisitos não funcionais (associados a qualidade de serviço), documentados no SRS.
4. **Conduzir a síntese em nível de sistema:** gera decisões arquiteturais de alto nível e integra o sistema, alimentando o próprio SRS.
5. **Revisar o design do sistema:** valida se necessidades, requisitos e síntese estão alinhados, produzindo o *functional baseline* — o marco funcional inicial do sistema.

Vale destacar a relação entre síntese e design: ambos envolvem propor soluções que atendam aos requisitos do sistema, mas a síntese foca em decisões arquiteturais de alto nível e integração, enquanto o design é um conceito mais amplo, que também inclui especificação detalhada, implementação e validação dos elementos individuais do sistema. Pode-se dizer que a síntese é um passo-chave dentro do processo maior de design.

Ao concluir essas cinco atividades e obter o *functional baseline*, o projeto avança para a próxima etapa da fase de aquisição: o design preliminar.

Síntese do Capítulo

- O ciclo de vida de um sistema vai da necessidade inicial à aposentadoria, passando por pré-aquisição, aquisição/desenvolvimento, utilização e aposentadoria.
- O design conceitual é a primeira etapa da fase de aquisição/desenvolvimento, também chamado de design lógico ou design do domínio do problema.
- Uma lacuna de capacidade (*capability gap*) é a diferença entre a capacidade atual e a capacidade desejada de uma organização ou sistema.
- Uma necessidade associada a uma lacuna de capacidade priorizada dá início ao ciclo de vida de engenharia de sistemas.
- O design conceitual se organiza em cinco atividades (C1 a C5): requisitos de negócio, requisitos de stakeholders, requisitos de sistema, síntese em nível de sistema e revisão de design.
- A etapa termina com o *functional baseline*, ponto de partida para o design preliminar.

Capítulo 8

O que é um Sistema?

Este capítulo apresenta o conceito mais básico da técnica STPA (*System-Theoretic Process Analysis*): o conceito de sistema. Compreendê-lo bem fornece o arcabouço teórico necessário para os demais conceitos usados ao longo do curso, incluindo o modelo de causalidade de acidentes STAMP.

8.1 Definição de Sistema e o Papel do Objetivo

Um sistema é um conjunto de componentes que trabalham em cooperação, como um todo, para realizar algum objetivo comum. A definição de objetivo é, portanto, indissociável da definição de sistema — e um mesmo sistema pode ter objetivos diferentes dependendo de quem o observa.

Sistema

Um sistema é um conjunto de componentes que interagem entre si para atingir um objetivo comum. O sistema é uma abstração — um modelo, uma simplificação da realidade — concebido a partir da perspectiva de um observador, e essa perspectiva determina quais propriedades do sistema são consideradas relevantes.

Um exemplo ilustra bem essa variação de perspectiva: o sistema aeroporto. Para o passageiro, seu objetivo é fornecer transporte aéreo. Para o poder público, gerar receita e fomentar a atividade econômica na região. Para a companhia aérea, transportar passageiros e cargas. Para os negócios instalados no terminal — restaurantes, livrarias —, atrair clientes para consumo. O mesmo sistema físico é interpretado de formas distintas conforme o stakeholder.

Por isso, quando o curso trata de sistemas sociotécnicos complexos — que envolvem software, hardware, instituições e pessoas, e que podem levar anos e milhares de pessoas para se desenvolver —, a documentação e a especificação tornam-se essenciais. Elas garantem que todos os envolvidos compartilhem um mesmo modelo mental do sistema, incluindo seus objetivos.

8.2 Sistemas de Engenharia versus Sistemas Naturais

Existe uma distinção importante entre sistemas projetados pelo ser humano (sistemas de engenharia) e sistemas naturais (por exemplo, biológicos). Nos sistemas de engenharia, o objetivo costuma ser especificado antes da construção do sistema — ainda que observadores diferentes possam interpretá-lo de maneiras distintas. Já nos sistemas

naturais, o comportamento não é definido a priori: observa-se o comportamento do sistema e, a partir dele, infere-se um propósito aparente. Este curso trata exclusivamente de sistemas de engenharia.

Duas suposições sustentam essa abordagem: primeiro, que os objetivos dos sistemas podem ser definidos; segundo, que o sistema é decomponível — seus componentes, assim como as relações diretas e indiretas entre eles, podem ser estudados separadamente.

8.3 A Definição Recursiva de Sistema

Um aspecto central da definição de sistema é seu caráter recursivo: um sistema pode ser visto como componente de um sistema maior, ou pode ser subdividido em subsistemas menores. Por exemplo, o sistema aeroporto pode ser decomposto em subsistemas menores, e esses subsistemas, por sua vez, podem ser vistos como parte de um sistema ainda maior.

Alguns autores da literatura preferem tratar esse encadeamento como um conceito distinto, chamado "sistema de sistemas". Neste curso, opta-se por não fazer essa distinção: trabalha-se sempre com o termo único "sistema", aplicando sua definição recursiva conforme o nível de abstração adotado pelo observador.

8.4 Estado, Propriedades, Ambiente e Fronteira

Todo sistema possui um **estado**, definido pelo conjunto de variáveis relevantes e seus valores em um dado momento no tempo. Essas variáveis representam as **propriedades** do sistema, e o conjunto de propriedades consideradas relevantes também varia de acordo com o observador. No exemplo do aeroporto, propriedades relevantes para o transporte aéreo poderiam incluir o número de passageiros em um portão de embarque ou a localização e situação das aeronaves (embarcando, taxiando, decolando).

O sistema interage com aquilo que está ao seu redor, chamado de **ambiente**: um conjunto de componentes que também possui estado e propriedades, mas que não faz parte do sistema em análise — embora seu comportamento possa afetar o sistema. A linha que separa o que está dentro do sistema do que está no ambiente é chamada de **fronteira** (ou limite) do sistema. Essa fronteira não precisa ser física; pode ser um corte conceitual definido pelo observador para delimitar o que é de seu interesse analisar ou projetar.

Sistemas que trocam entradas e saídas com o ambiente são chamados **sistemas abertos**; sistemas que não interagem com o mundo externo são **sistemas fechados** — como um sistema termodinâmico isolado, sem troca de calor. Este curso trata exclusivamente de sistemas abertos, já que sistemas sociotécnicos de interesse sempre interagem com seu ambiente.

Síntese do Capítulo

- Sistema é um conjunto de componentes que cooperam para atingir um objetivo comum; o objetivo varia conforme o observador (exemplo: aeroporto visto por

passageiro, governo, companhia aérea e comércio).

- Documentação e especificação são essenciais para manter uma visão compartilhada dos objetivos em sistemas complexos.
- Sistemas de engenharia têm objetivo definido a priori; sistemas naturais têm comportamento observado a posteriori. O curso trata apenas de sistemas de engenharia.
- A definição de sistema é recursiva: um sistema pode ser subsistema de outro maior ou composto por subsistemas menores; este curso não usa o termo "sistema de sistemas".
- Sistemas têm estado (propriedades e valores em um momento no tempo) e uma fronteira que os separa do ambiente.
- O curso trabalha exclusivamente com sistemas abertos, que trocam entradas e saídas com o ambiente.

Capítulo 9

O que é Engenharia de Sistemas?

Tendo estabelecido o conceito de sistema — sempre atrelado a objetivos que variam conforme a perspectiva do stakeholder —, este capítulo apresenta a engenharia de sistemas propriamente dita, o modelo de processo mais utilizado na área (o modelo V) e o conceito de rastreabilidade.

9.1 Definição e Origem da Engenharia de Sistemas

A engenharia de sistemas é a tentativa de impor estrutura tanto ao design quanto à construção de sistemas, com o objetivo de aperfeiçoar os resultados do esforço de engenharia. A área cresceu significativamente após a Segunda Guerra Mundial, quando sistemas sociotécnicos cada vez mais complexos — especialmente na área de defesa e mísseis — passaram a ser construídos. A ausência de um processo estruturado para desenvolver e operar esses sistemas levou a atrasos frequentes, qualidade inadequada e estouro de orçamento — um paralelo direto com o que ocorreu, décadas depois, na conhecida "crise do software".

9.2 O Modelo V e a Ilusão da Linearidade

O modelo de ciclo de vida mais básico e conhecido da engenharia de sistemas é o **modelo V**, que orienta a concepção, definição, projeto, implementação, operação, manutenção e evolução de sistemas — sejam eles compostos por software, hardware, instituições ou pessoas.

Uma ressalva importante: a representação gráfica do modelo V, com suas fases dispostas sequencialmente, pode sugerir um processo linear e em cascata. Essa leitura é enganosa. Em projetos que envolvem milhares de pessoas ao longo de anos, diversas atividades ocorrem em paralelo, e vários modelos de ciclo de vida se comunicam simultaneamente — daí o uso comum de abordagens iterativas e incrementais. O próprio *INCOSE Systems Engineering Handbook* destaca que as fases do ciclo de vida não implicam que o projeto siga um conjunto predeterminado de atividades em sequência estrita.

Dois exemplos tornam isso concreto. A engenharia de requisitos — foco central desta disciplina, no que se refere a requisitos de *safety* e *security* — não ocorre apenas em uma etapa inicial: requisitos são coletados e refinados ao longo de todo o ciclo de vida, e é comum que a fase de design gere novos requisitos à medida que se adquire mais conhecimento sobre o sistema. Da mesma forma, os testes, embora representados apenas no braço ascendente do V, na prática começam já na fase de concepção e se estendem por todo o desenvolvimento.

Além disso, o modelo V não deve ser seguido "ao pé da letra": é um arcabouço a ser customizado conforme as necessidades e objetivos de cada projeto, inclusive quanto aos nomes de suas fases — que variam de livro para livro e de organização para organização.

Modelo V

O modelo V é o modelo de processo mais básico da engenharia de sistemas, representando de forma pedagógica um sequenciamento de fases (concepção, requisitos, arquitetura, design, implementação, teste, operação). Apesar da representação sequencial, as atividades dessas fases ocorrem amplamente em paralelo ao longo do ciclo de vida real de um sistema.

9.3 Documentação, Especificação e Fronteira do Sistema

Por lidar com sistemas sociotécnicos complexos — que envolvem milhares de pessoas e muitos anos de desenvolvimento —, o modelo V é fortemente orientado a documentação. As especificações começam a ser produzidas já nas fases preliminares (concepção e definição) e são refinadas ao longo de todo o ciclo de vida. Documentação e especificação garantem que todos os envolvidos compartilhem o mesmo entendimento sobre o sistema, evitando ambiguidades que, em projetos de milhares de páginas, seriam inevitáveis.

Toda especificação de sistema deve conter, no mínimo: os objetivos do sistema; as propriedades de interesse (lembrando que todo sistema possui estado, isto é, um conjunto de propriedades e valores em um dado momento); as restrições de operação; e a fronteira do sistema.

A fronteira do sistema define o que está sob controle dos projetistas (dentro do sistema) e o que é assumido como fora de seu controle, pertencente ao ambiente — por exemplo, a direção do vento não é controlada pelo ser humano e normalmente fica fora do escopo de análise.

9.4 Rastreabilidade

Em sistemas complexos, cujas documentações e especificações somam milhares de páginas, um requisito definido em um ponto do ciclo de vida se manifesta em múltiplos artefatos posteriores — o sistema em operação é, em última instância, a realização de requisitos definidos muito antes. Por isso, a **rastreabilidade** é essencial: ela permite identificar, diante de uma alteração em qualquer parte da documentação, quais outras partes seriam impactadas, evitando inconsistências entre os artefatos do sistema.

A rastreabilidade será central na técnica STPA, estudada mais adiante: as ações de controle inseguras são rastreadas até as condições de perigo (*hazards*) que buscam evitar, e os cenários de acidente identificados no quarto passo da técnica são rastreados até as ações de controle inseguras definidas no terceiro passo. Rastreabilidade também permite justificar decisões de design, documentando os motivos que levaram um projetista a adotar determinada solução.

Síntese do Capítulo

- Engenharia de sistemas busca estruturar o design e a construção de sistemas para melhorar os resultados de engenharia; cresceu após a Segunda Guerra Mundial.
- O modelo V é o modelo de processo mais básico da área, cobrindo desde a concepção até a operação e manutenção de sistemas.
- Apesar de parecer sequencial, o modelo V é executado com forte paralelismo entre fases — requisitos e testes, por exemplo, ocorrem ao longo de todo o ciclo de vida.
- O modelo V é um arcabouço customizável, não uma prescrição rígida; nomes de fases variam entre autores e organizações.
- Toda especificação de sistema deve definir objetivos, propriedades, restrições de operação e a fronteira do sistema.
- Rastreabilidade é fundamental para gerenciar o impacto de mudanças e justificar decisões de design em sistemas complexos; será essencial na técnica STPA.

Capítulo 10

O Conceito de Controle e Técnicas de Design para Safety

Este capítulo apresenta o conceito de controle na engenharia de sistemas — fundamento da teoria de sistemas subjacente a todo o curso — e detalha as principais técnicas de design usadas para controlar *safety*.

10.1 O Conceito de Controle na Engenharia de Sistemas

O controle é essencial tanto na concepção quanto na operação de um sistema. Em teoria, um sistema poderia operar sem qualquer controle sobre seu comportamento, mas seria muito difícil garantir que seus objetivos fossem satisfeitos sem a ocorrência de eventos adversos. O controle é o mecanismo que busca assegurar que a operação do sistema permaneça dentro de limites pré-estabelecidos, evitando acidentes.

Controle

Controle é o mecanismo pelo qual se garante que o comportamento e a operação de um sistema permaneçam dentro de restrições pré-estabelecidas, prevenindo a ocorrência de eventos adversos e acidentes. Na engenharia de sistemas, controle não se refere apenas a comando hierárquico ou autoritário (como ordens militares); refere-se, em sentido amplo, a controles de design, controles de processo e controles sociais.

Vale destacar que, ao longo do tempo, sistemas em operação tendem a migrar para estados de maior risco: controles se afrouxam, procedimentos se relaxam, e sem vigilância contínua a probabilidade de acidentes e perdas inaceitáveis aumenta. Isso é especialmente relevante em sistemas sociotécnicos complexos — sistemas mais simples, como uma ponte, dificilmente exigem controle ativo comparável.

10.2 Controles em Nível de Design

Restrições de comportamento e os controles que as sustentam evitam que um sistema atinja seus objetivos "a qualquer preço" — por exemplo, uma indústria química que produz seus produtos, mas causa danos ambientais ou humanos no processo. Um dos tipos de controle mais relevantes para este curso é o controle em nível de design: funcionalidades incorporadas ao próprio sistema, desde o início do ciclo de vida de desenvolvimento, capazes de promover autocontrole sem necessariamente aumentar complexidade ou custo.

Existem três estratégias gerais para aumentar o nível de segurança de um sistema por meio de controles de design:

1. eliminar completamente as condições de perigo (*hazards*) — a opção mais desejável, pois sem hazard não há acidente decorrente dele;
2. tornar as condições de perigo raras e fáceis de gerenciar caso ocorram;
3. reagir à ocorrência do hazard, minimizando ou eliminando os danos resultantes.

Sistemas sociotécnicos complexos frequentemente exigem as três estratégias combinadas, já que eliminar completamente todos os hazards nem sempre é viável.

10.2.1 Técnicas de Design Intrinsecamente Seguro

São o ideal a se buscar: o próprio design impede fisicamente a ocorrência do hazard. Exemplos: um sistema projetado para não liberar energia suficiente para causar uma explosão; um processo industrial que substitui substâncias tóxicas por alternativas não tóxicas disponíveis no mercado; comunicação física em vez de comunicação em rede, quando a perda de informação sensível é um risco inaceitável. Nesses casos, o hazard simplesmente não pode ocorrer.

10.2.2 Técnicas de Design Passivo

Não exigem um controlador humano ou eletrônico para serem ativadas — funcionam automaticamente, frequentemente baseadas em princípios físicos como a gravidade, e estão associadas ao conceito de *fail-safe* (o sistema falha para um estado seguro). Exemplos incluem: tapetes sensíveis à pressão que desligam maquinário perigoso quando alguém se aproxima; cortinas de luz (feixes que, ao serem interrompidos, desligam o equipamento); limitadores de velocidade instalados em ônibus escolares; válvulas de alívio de pressão que abrem automaticamente ao atingir um limite; os antigos semáforos ferroviários mecânicos, que caíam para a posição "fechado" por gravidade caso o cabo de sustentação se rompesse; e o *deadman switch*, usado por exemplo em cortadores de grama e serras elétricas, que desliga o equipamento assim que o operador solta o botão de pressão (protegendo contra mal súbito do operador).

10.2.3 Técnicas de Design Ativo

Exigem a identificação de um estado de perigo do sistema para então corrigi-lo, envolvendo tipicamente seres humanos e computadores em um ciclo de monitoramento e decisão (o *control loop*, detalhado no próximo capítulo). Vários problemas podem ocorrer nesse processo: falha em detectar que o sistema está em condição de perigo; detecção do alarme sem que nenhuma ação seja tomada; ou ação tomada fora do prazo necessário para evitar o acidente (por exemplo, ao detectar uma temperatura muito acima do limite tolerado).

10.3 Comparativo entre as Técnicas

Técnicas passivas são geralmente mais simples de implementar, mais confiáveis e menos dependentes de monitoramento contínuo — quando algo falha, a resposta já está embutida fisicamente no sistema. Em contrapartida, oferecem menos liberdade de design e nem sempre são viáveis para eliminar todos os hazards de sistemas sociotécnicos complexos. Técnicas ativas, por sua vez, são mais flexíveis, mas dependem de um *control loop* confiável — envolvendo sensores, decisão e atuação — para funcionar corretamente.

Síntese do Capítulo

- Controle, na engenharia de sistemas, garante que o comportamento do sistema permaneça dentro de restrições pré-estabelecidas, evitando acidentes — não se limita a comando hierárquico.
- Sistemas tendem a migrar para estados de maior risco ao longo do tempo se não houver controle ativo.
- Há três estratégias de controle de design para safety: eliminar o hazard, tornar o hazard raro e gerenciável, ou reagir para minimizar danos.
- Técnicas intrinsecamente seguras eliminam o hazard pela própria natureza do design.
- Técnicas passivas (fail-safe) agem automaticamente sem controlador, baseadas em princípios físicos (tapetes de pressão, válvulas de alívio, deadman switch).
- Técnicas ativas exigem detecção e decisão (control loop), sendo mais flexíveis, porém mais suscetíveis a falhas de detecção, omissão ou atraso na resposta.

Capítulo 11

O Laço de Controle (Control Loop)

O capítulo anterior mostrou que as técnicas de controle ativo em nível de design dependem de um *control loop*: um laço de retroalimentação que observa o estado de um processo e decide quais ações de controle enviar para mantê-lo em estado seguro. Este capítulo detalha seus componentes, seus modos de operação e sua aplicabilidade além da engenharia.

11.1 Origem e Definição

O conceito de *control loop* vem da teoria de sistemas — mais precisamente da cibernética, área que estuda como sistemas de controle comandam, direcionam e regulam o comportamento de outros processos e dispositivos. O tema é extensamente tratado no livro *Engineering a Safer World*, de Nancy Leveson, referência central desta disciplina. Control loops estão presentes desde sistemas simples, como o termostato de aquecimento residencial, até sistemas industriais complexos de controle de processo.

Control Loop

Um control loop (laço de controle) é um ciclo de retroalimentação em que um controlador observa o estado de um processo controlado (via feedback), forma um modelo mental desse estado (modelo de processo) e, com base em um algoritmo de controle, decide e emite ações de controle (control actions) para manter o processo dentro de restrições de operação pré-estabelecidas.

11.2 Os Componentes do Control Loop

O **controlador** tem como objetivo manter o comportamento do sistema dentro de limites de segurança pré-estabelecidos, satisfazendo restrições de operação. Ele age emitindo **control actions** (ações de controle), que na engenharia são implementadas por **atuadores** — dispositivos que aplicam o comando no momento e na intensidade corretos sobre o processo controlado.

Para decidir qual control action emitir, o controlador precisa saber o estado atual do processo. Essa informação vem do **feedback**, normalmente implementado por meio de **sensores**, que capturam o estado do processo controlado e o retornam ao controlador.

Dentro do controlador existem ainda dois elementos internos essenciais: o **modelo de processo** — a visão (necessariamente incompleta) que o controlador tem do estado real do processo, construída a partir do feedback — e o **algoritmo de controle**

— o procedimento ou conjunto de regras que, aplicado sobre o modelo de processo, determina qual ação de controle deve ser emitida.

O **processo controlado** é o sistema ou processo cujo estado é afetado pelas control actions emitidas pelo controlador. É importante notar que ele também pode ser afetado por outros fatores além do controlador: entradas do ambiente, distúrbios externos e até outros controladores atuando simultaneamente sobre o mesmo processo. Quando dois controladores independentes emitem ações conflitantes sobre o mesmo processo, o sistema pode ser levado a um estado inseguro que, em conjunção com fatores ambientais, pode resultar em acidente — entendido aqui como qualquer perda inaceitável para os stakeholders: perda de vidas, ferimentos, prejuízo econômico, dano ambiental ou perda de reputação.

O ciclo se repete indefinidamente: uma control action muda o estado do processo; os sensores capturam esse novo estado; o modelo de processo é atualizado; um novo algoritmo de decisão pode gerar uma nova control action; e assim por diante.

Um exemplo simples ilustra o funcionamento completo: o termostato residencial. O sensor mede a temperatura ambiente (por exemplo, 72°F). O controlador — humano ou eletrônico — usa essa leitura para atualizar seu modelo de processo, combinando-a com outras informações (temperatura externa, flutuações recentes). Com base nisso, o algoritmo de controle decide aumentar, diminuir ou manter a temperatura, emitindo a control action correspondente (os botões de seta do termostato). Essa ação altera o estado do processo controlado (a temperatura do ambiente), que é novamente capturado pelo sensor, fechando o laço.

11.3 Feedback Control e Feedforward Control

O control loop opera em dois modos, geralmente combinados.

O **feedback control** baseia-se apenas no estado atual do processo: o controlador observa a situação presente e decide com base nela. O exemplo clássico é dirigir observando o velocímetro — a decisão de acelerar ou frear se baseia na velocidade atual e em condições do ambiente, como clima e pista.

O **feedforward control** combina o estado atual com uma previsão do futuro. O exemplo do motorista experiente ilustra bem: percebendo que uma ladeira íngreme se aproxima, ele acelera antes de chegar nela — não porque a velocidade atual esteja errada, mas porque antecipa a perda de velocidade que a subida causará, buscando manter uma velocidade média constante. Os dois modos normalmente se combinam: o motorista pode antecipar a aceleração (feedforward) e, já na ladeira, ajustar finamente com base na velocidade observada naquele momento (feedback).

Vale mencionar, ainda que brevemente, o conceito de *leading indicators* — indicadores usados em técnicas de análise de risco para antecipar a migração de um sistema rumo a um estado de alto risco. Esse conceito se relaciona às chamadas *shaping actions*: controles de design que buscam manter o processo controlado dentro de um estado seguro, restringindo sua migração para estados de risco. O tema será retomado com mais formalidade ao tratarmos de STPA.

11.4 Aplicação além da Engenharia

Por derivar da teoria de sistemas, o conceito de control loop não se limita a sistemas técnicos: é amplamente utilizado também para descrever sistemas organizacionais e sociais, sendo um conceito recorrente nas ciências sociais e nas teorias de gestão.

Síntese do Capítulo

- O control loop é o mecanismo central das técnicas de controle ativo em nível de design, originado da teoria de sistemas (cibernética).
- Seus componentes principais são: controlador, control actions, atuadores, feedback, sensores, modelo de processo, algoritmo de controle e processo controlado.
- O processo controlado pode ser afetado não só pelas control actions, mas por distúrbios ambientais e por outros controladores — inclusive de forma conflitante.
- O termostato ilustra o ciclo completo: sensor mede o estado, controlador atualiza seu modelo mental e decide, control action altera o processo, e o ciclo se repete.
- Feedback control decide a partir do estado presente; feedforward control incorpora previsão do futuro; ambos costumam ser usados em conjunto.
- O control loop também é aplicado fora da engenharia, em sistemas organizacionais e sociais.

Capítulo 12

Teoria de Sistemas versus Teoria da Complexidade

O modelo de causalidade de acidentes estudado neste livro, assim como as técnicas construídas sobre ele, tem fundamento na teoria de sistemas. Existe, porém, uma teoria vizinha e frequentemente confundida com ela: a teoria da complexidade. Este capítulo apresenta as duas abordagens, destaca suas semelhanças e explica por que a teoria de sistemas é a base escolhida para a análise de requisitos de segurança.

12.1 Teoria de Sistemas

A teoria geral dos sistemas tem como um de seus fundadores o biólogo austríaco Ludwig von Bertalanffy. Outro autor importante, Norbert Wiener, estendeu essas ideias para a matemática e a engenharia sob o nome de cibernética. Apesar disso, o termo consagrado atualmente é "teoria geral dos sistemas", em referência a Bertalanffy.

No uso cotidiano, dois conceitos aparecem associados a essa teoria:

Teoria de Sistemas e Systems Thinking

Teoria de sistemas é o conjunto de princípios usado para entender o comportamento de sistemas complexos, sejam eles naturais ou projetados por engenheiros. **Systems thinking** (pensamento sistêmico) é a prática de aplicar esses princípios: é o termo que descreve o que as pessoas fazem quando raciocinam a partir da teoria de sistemas.

12.2 Teoria da Complexidade

A teoria da complexidade nasceu e cresceu a partir dos próprios conceitos da teoria de sistemas, somados a discussões que ganharam força na década de 1960. Ela está historicamente associada aos pesquisadores do Instituto Santa Fé, no Novo México (Estados Unidos).

12.3 Semelhanças e Diferenças

As duas teorias compartilham uma característica central: ambas analisam o comportamento do sistema como um todo, de forma holística, em vez de decompô-lo em componentes isolados. Os componentes básicos da teoria de sistemas – emergência, hierarquia, comunicação e controle – também estão presentes na teoria da complexidade. Da mesma forma, conceitos já discutidos neste curso, como os laços de controle

(*feedback* e *feedforward*), adaptabilidade, interações não lineares e restrições, aparecem em ambas as abordagens.

As diferenças, no entanto, são relevantes. A teoria de sistemas assume que todo sistema possui propriedades e comportamentos emergentes. Já a teoria da complexidade classifica os sistemas em quatro tipos – simples, complicados, complexos e caóticos –, cada um com um grau distinto de emergência.

Essa distinção leva a uma diferença de aplicação: a teoria de sistemas é mais adequada a sistemas projetados por humanos, cujo funcionamento interno é conhecido a priori. A teoria da complexidade, por sua vez, é mais adequada a sistemas naturais cujo design não conhecemos de antemão, como sistemas meteorológicos e sociológicos.

Como o foco deste curso é o modelo de causalidade de acidentes e as técnicas construídas sobre seu arcabouço – aplicadas a sistemas projetados por engenharia –, a base teórica adotada é a teoria de sistemas, e não a teoria da complexidade.

Síntese do Capítulo

- A teoria geral dos sistemas tem origem em Bertalanffy; Wiener a estendeu à matemática e à engenharia com o nome de cibernética.
- Teoria de sistemas é o conjunto de princípios sobre sistemas complexos; *systems thinking* é a aplicação prática desses princípios.
- A teoria da complexidade nasceu dos conceitos da teoria de sistemas, com forte associação ao Instituto Santa Fé.
- Ambas compartilham a visão holística e conceitos como emergência, hierarquia, comunicação, controle, *feedback/feedforward* e adaptabilidade.
- Diferem no tratamento da emergência (a complexidade distingue sistemas simples, complicados, complexos e caóticos) e no domínio de aplicação: sistemas projetados (teoria de sistemas) versus sistemas naturais (teoria da complexidade). Este curso adota a teoria de sistemas.

Parte III

Estudo de Caso: O Desastre de Bhopal

Capítulo 13

O Desastre de Bhopal: Contexto Histórico

Em dezembro de 1984, na cidade indiana de Bhopal, ocorreu o que é considerado o pior acidente industrial da história. Milhares de pessoas morreram imediatamente como consequência do vazamento de um gás tóxico, e as sequelas se estenderam por anos, afetando inclusive gerações posteriores. Este capítulo introduz o contexto técnico e organizacional da usina onde o desastre aconteceu, servindo de base para a análise detalhada das causas que será conduzida nos capítulos seguintes.

13.1 A Substância e o Perigo Envolvido

O produto químico liberado no acidente foi o isocianato de metila (MIC), amplamente utilizado na fabricação de pesticidas, plásticos, vernizes e espumas. O MIC apresenta um risco particular: ao entrar em contato com água, reage produzindo grande quantidade de calor. Esse calor é suficiente para queimar qualquer tecido úmido do corpo humano — olhos, garganta, pulmões e demais mucosas — o que explica a letalidade do vazamento.

A planta responsável pela produção era operada pela Union Carbide, empresa que adotava, ao menos formalmente, requisitos rigorosos de redução de risco. Entre esses requisitos estava o uso de sistemas de proteção redundantes, conhecidos como defesa em profundidade.

Defesa em Profundidade

A defesa em profundidade consiste em dispor múltiplas camadas de proteção independentes, de modo que, se um dispositivo de segurança falhar, outro atue para conter ou minimizar a falha. O racional é estatístico: multiplicando-se as probabilidades de falha de cada camada, a probabilidade conjunta de todas falharem simultaneamente tende a ser ínfima. Essa abordagem é amplamente empregada em indústrias nucleares, químicas, petrolíferas e de gás. Como o caso de Bhopal demonstra, entretanto, a eficácia da defesa em profundidade depende de que cada camada esteja de fato operante e bem projetada — pressupostos que, na prática, podem ser sistematicamente violados.

13.2 Os Dispositivos de Segurança da Usina

A usina de Bhopal contava com uma sequência de dispositivos de defesa em profundidade, dispostos para atuar em cascata caso o MIC escapasse do armazenamento.

O armazenamento era feito em três tanques de aço inoxidável duplamente protegidos, cada um com capacidade de 60 toneladas. O manual de operação determinava que os tanques jamais deveriam ser preenchidos além da metade de sua capacidade máxima, e um tanque adicional permanecia vazio, em stand-by, destinado a receber o excesso de produto em caso de emergência, como uma reação química que gerasse expansão de gás. O MIC era mantido sob atmosfera estática de nitrogênio comprimido, entre 2 e 10 PSI acima da pressão atmosférica, com armazenamento limitado a doze meses e inspeções periódicas obrigatórias.

Caso houvesse vazamento, entrava em ação o purificador de gás (*vent gas scrubber*), projetado para neutralizar o MIC escapado usando soda cáustica, com capacidade de tratar até oito toneladas de gás por hora em operação máxima. Se o scrubber não conseguisse processar todo o gás, o excedente deveria ser queimado na torre de chamas, instalada em altura elevada para tornar o gás inofensivo antes de atingir o solo. Um resquício remanescente seria neutralizado pela cortina de água, capaz de atingir de 40 a 50 pés (13 a 15 metros) de altura — ou até 115 pés (35 metros) com mangueiras adicionais.

Complementava o sistema uma unidade de refrigeração, responsável por manter o MIC próximo de 0°C, com alarme configurado para disparar caso a temperatura atingisse 11°C. Também eram exigidos equipamentos de proteção individual — roupas de borracha e respiradores — para qualquer operação próxima aos tanques, além de sirenes destinadas a alertar trabalhadores e a comunidade vizinha em caso de vazamento incontrolável.

13.3 Uma Estrutura de Segurança Aparentemente Robusta

No papel, a usina de Bhopal apresentava um sistema de defesa em profundidade completo: tanques redundantes, purificador de gás, torre de chamas, cortina de água, sistema de refrigeração, alarmes e equipamentos de proteção individual. Essa aparente robustez é justamente o que torna o desastre didaticamente relevante: um acidente de tal magnitude não ocorreu por ausência de projeto de segurança, mas por uma cadeia de falhas — de manutenção, de gestão e de cultura organizacional — que comprometeu, uma a uma, cada camada de proteção prevista. Os capítulos seguintes reconstroem essa cadeia, começando pelos eventos imediatos que desencadearam o vazamento, passando pelas condições estruturais da usina, até chegar aos fatores sistêmicos que tornaram o desastre praticamente inevitável.

Síntese do Capítulo

- O desastre de Bhopal (dezembro de 1984) é considerado o pior acidente industrial da história, com milhares de mortes imediatas e efeitos multigeracionais.
- A substância liberada, isocianato de metila (MIC), reage com água gerando calor intenso, capaz de queimar tecidos úmidos do corpo humano.

- A usina, operada pela Union Carbide, adotava o princípio de defesa em profundidade: múltiplas camadas de proteção redundantes.
- As camadas previstas incluíam tanques de armazenamento controlado, purificador de gás (scrubber), torre de chamas, cortina de água, sistema de refrigeração, alarmes e equipamentos de proteção individual.
- A robustez teórica do projeto de segurança contrasta com o resultado real, o que motiva a análise aprofundada das causas nos capítulos seguintes.

Capítulo 14

O Desastre de Bhopal: a Sequência de Eventos

Para analisar acidentes industriais complexos, é comum aplicar um modelo hierárquico de causalidade organizado em níveis. No nível mais básico, identifica-se a cadeia de eventos que antecede o acidente. Em níveis superiores, analisam-se as condições que propiciaram esses eventos e, por fim, os fatores sistêmicos da organização como um todo. Este capítulo trata do primeiro nível: a reconstrução cronológica dos dezenove eventos que culminaram no vazamento de isocianato de metila (MIC) em Bhopal.

Antes da cronologia, vale apresentar um dispositivo de projeto central para o caso: o *slip blind*, uma placa metálica sólida inserida manualmente entre flanges de tubulação para isolar fisicamente um trecho em manutenção do restante do sistema. Diferentemente de uma válvula fechada — que pode vaziar ou ser aberta inadvertidamente —, o *slip blind* garante isolamento total. Sua ausência é um dos pontos centrais da cronologia a seguir.

14.1 A Cronologia dos Dezenove Eventos

1. Em 2 de dezembro de 1984, um trabalhador recém-contratado foi designado para lavar tubulações e filtros entupidos na usina, conectados aos tanques de MIC. As tubulações estavam isoladas apenas por uma válvula fechada; nem o trabalhador fechou adequadamente a válvula, nem foi inserido o *slip blind* que garantiria isolamento total.
2. **Eventos 4 a 6.** À noite, por volta das 23h, os medidores indicaram pressão de 10 PSI nos tanques (acima da faixa recomendada de 2 a 3 PSI, mas ainda dentro do limite tolerável) e temperatura de cerca de 20°C, quando o ideal seria próximo de 0°C.
3. **Eventos 7 e 8.** Os medidores de pressão e temperatura foram ignorados pelos trabalhadores, que não confiavam em sua precisão. Os gestores instruíam os operadores a identificar vazamentos pela irritação nos olhos. Às 2h30, foi detectado vazamento de líquido nas tubulações e leve irritação ocular, mas vazamentos de válvula eram considerados rotineiros e não geraram alarme.
4. **Eventos 9 e 10.** Trabalhadores identificaram um gotejamento contínuo fora da unidade de MIC e relataram ao supervisor de plantão, que decidiu adiar a verificação. Já na madrugada de 3 de dezembro, o operador da sala de controle notou que o tanque 610 registrava 40 PSI de pressão e 25°C de temperatura.

5. **Eventos 11 a 13.** À meia-noite e 45, um forte estrondo foi ouvido e o concreto ao redor do tanque rachou. A temperatura interna chegou a 400°C, rompendo as válvulas de alívio de pressão (dispositivos passivos de projeto), o que permitiu a fuga do gás em direção ao scrubber.
6. **Eventos 14 e 15.** O vazamento persistiu até as 2h30, com o MIC atingindo 108 pés de altura e um total estimado de 25 a 30 toneladas de gás liberado. Um operador desligou a linha em manutenção e acionou o scrubber, mas o medidor não indicou circulação de soda cáustica. Ele se recusou a verificar visualmente sem a companhia de um supervisor, e o supervisor também se recusou a acompanhá-lo por medo da toxicidade da substância.
7. **Eventos 16 a 18.** A válvula que conectava o tanque 610 ao tanque de backup vazio nunca foi aberta, pois o medidor de nível indicava que esse tanque já estava parcialmente cheio. O supervisor da área não encontrou máscara de oxigênio, tentou pular a cerca da usina em fuga e fraturou a perna; o supervisor da sala de controle permaneceu no local até a manhã seguinte, sem ferimentos.
8. **Evento 19.** A sirene de alerta só foi acionada à meia-noite e 50 — quando o gás já havia atingido o próprio purificador — e foi desligada cinco minutos depois, por política da empresa. Ninguém acionou a polícia; foi o pânico na cidade vizinha que levou a polícia a se dirigir à usina, sem obter informações úteis dos responsáveis.

14.2 Identificando Causas a Partir dos Eventos

Diante apenas dessa cronologia, o relatório oficial do acidente apontou como causa a entrada de água no tanque de MIC, e a gestão da Union Carbide atribuiu a responsabilidade ao operário recém-contratado que lavava as tubulações — chegando a alegar que só uma sabotagem deliberada explicaria um erro tão grave. O trabalhador foi preso e, pouco depois, liberado.

A causa raiz identificada no relatório foi a não inserção do slip blind durante a manutenção das tubulações.

14.3 As Perguntas que os Eventos Não Respondem

A simples cronologia de eventos, no entanto, deixa uma série de questões em aberto: por que o responsável pela limpeza não inseriu o slip blind? Por que os supervisores adiaram a verificação de um vazamento relatado? Por que todos os dispositivos de defesa em profundidade falharam simultaneamente — uma probabilidade estatisticamente próxima de zero se cada camada fosse de fato independente e funcional? Por que o operador não abriu a válvula de escape para o tanque de backup? Por que a sirene demorou tanto para soar e foi desligada minutos depois? Por que os trabalhadores não evacuaram a área conforme o treinamento previa? Por que o sistema de refrigeração não manteve a temperatura adequada?

14.4 A Lição do Nível 1

A análise apenas da cadeia de eventos tende a apontar culpados individuais, porque toda ação e omissão em uma cronologia envolve pessoas específicas. Mas o próprio exercício de narrar os dezenove eventos gera mais perguntas do que respostas: por que os dispositivos falharam, por que os procedimentos não foram seguidos, por que a organização permitia essas lacunas. A lição central deste nível de análise é que não é possível compreender por que um acidente ocorreu observando somente a cadeia de eventos — é necessário investigar as condições que tornaram esses eventos possíveis, o que será o objeto do próximo capítulo.

Síntese do Capítulo

- A cronologia de Bhopal reúne dezenove eventos, do início da manutenção sem slip blind (2 de dezembro) até a ativação tardia da sirene (madrugada de 3 de dezembro).
- Medidores de pressão e temperatura foram sistematicamente ignorados por desconfiança dos próprios trabalhadores.
- O relatório oficial atribuiu a causa ao erro do operário recém-contratado e a causa raiz à ausência do slip blind.
- A análise baseada apenas em eventos tende a individualizar a culpa, pois toda ação registrada envolve uma pessoa.
- Diversas perguntas ficam sem resposta quando se olha apenas para os eventos: por que os dispositivos de segurança falharam em cadeia, por que procedimentos não foram seguidos.
- Lição aprendida: a cadeia de eventos, isoladamente, é insuficiente para explicar as causas de um acidente.

Capítulo 15

O Desastre de Bhopal: Condições Inseguras e Falhas de Projeto

Depois de reconstruir a cadeia de eventos que antecedeu o vazamento de isocianato de metila (MIC), o passo seguinte no modelo hierárquico de causalidade é examinar as condições da usina que tornaram esses eventos possíveis. A pergunta que orienta este nível de análise é direta: como todos os mecanismos de defesa em profundidade — purificador de gás, torre de chamas, cortina de água, sistema de refrigeração, alarmes e instrumentos de monitoramento — puderam falhar ao mesmo tempo, se a probabilidade teórica de uma falha conjunta é praticamente nula?

15.1 Os Dispositivos de Defesa em Profundidade Sob a Lupa

O purificador de gás (*vent scrubber*) havia sido projetado para neutralizar apenas pequenas quantidades de gás em baixa pressão e temperatura. No acidente, a pressão do gás escapado excedeu em cerca de duas vezes e meia o limite de projeto, e a temperatura ultrapassava 80°C — muito além da capacidade do equipamento. Além disso, o scrubber estava desligado para manutenção no momento do vazamento.

A torre de chamas era inadequada para as 40 toneladas de gás liberadas, e mesmo que seu projeto suportasse tal volume, ela não poderia ter sido usada: a tubulação de conexão estava corroída e nunca havia sido substituída.

A cortina de água alcançava, por projeto, entre 40 e 50 pés (aproximadamente 15 metros) de altura, mas o vapor de MIC escapou a mais de 100 pés (30 metros). Diferentemente do que se poderia supor, essas três falhas — scrubber, torre de chamas e cortina de água — não decorreram de defeitos de componentes isolados, mas de erros de projeto e de manutenção deficiente, o que expõe uma fragilidade do próprio raciocínio probabilístico da defesa em profundidade: multiplicar probabilidades de falha independentes só faz sentido se as falhas realmente forem independentes e os equipamentos estiverem operando dentro do projetado.

O sistema de refrigeração deveria manter o MIC abaixo de 5°C e permanecer sempre ligado enquanto houvesse substância nos tanques, com alarme configurado para disparar a 11°C. Na prática, a unidade estava desligada, o MIC era rotineiramente mantido próximo de 20°C, e a gestão da usina havia recalibrado o alarme para disparar apenas próximo de 20°C — eliminando qualquer aviso antecipado do aquecimento da substância.

Condições versus Eventos

Enquanto os eventos descrevem o que aconteceu numa sequência temporal, as condições explicam por que esses eventos puderam ocorrer: são o estado de projeto, manutenção, calibração e cultura organizacional que precede e possibilita a cadeia de eventos. Uma mesma condição insegura — como um alarme recalibrado ou um dispositivo desligado por economia — pode estar na origem de múltiplos eventos aparentemente desconexos. Investigar apenas os eventos tende a apontar culpados individuais; investigar as condições revela falhas estruturais que antecedem e transcendem qualquer erro pontual de operador.

15.2 O Tanque de Armazenamento e os Instrumentos

O tanque 610 continha entre 40 e 50 toneladas de MIC, dentro de uma capacidade total de 60 toneladas — violando a regra de nunca ultrapassar metade da capacidade. O tanque de backup, que deveria estar praticamente vazio para absorver excedentes em emergência, continha menos de uma tonelada real, mas o medidor indicava 20% de ocupação (o equivalente a 10 ou 12 toneladas), evidenciando que os instrumentos de medição estavam descalibrados ou mal configurados. De modo mais geral, os dispositivos passivos de defesa em profundidade eram frequentemente desligados pela empresa para economizar recursos, e havia poucos alarmes e intertravamentos (*interlocks*) funcionando — justamente os mecanismos que poderiam ter alertado os operadores sobre condições anormais.

Era prática rotineira armazenar MIC no próprio tanque reservado para emergências, mesmo sem necessidade real. E quando o pânico já se instalava na cidade, moradores ignoraram os ônibus estacionados especificamente para evacuação de emergência.

15.3 Falhas de Procedimento: Slip Blind, Supervisão e Alarmes

O checklist de manutenção usado pelos operadores não continha nenhuma instrução para inserir o slip blind antes da limpeza de tubulações. O trabalhador recém-contratado responsável pela tarefa não verificou se a linha estava devidamente isolada — em parte porque não constava no procedimento, em parte porque lhe haviam dito que essa não era sua responsabilidade.

A operação de limpeza deveria ser supervisionada por um segundo operador, função que havia sido eliminada por motivos financeiros.

Os alarmes soavam com frequência — entre 20 e 30 vezes por semana — de modo que os trabalhadores haviam se habituado a tratá-los como falsos positivos. O sinal de emergência crítica era idêntico ao usado para outros propósitos, como treinamentos, sem distinção sonora. No dia do acidente, o alarme só foi disparado cerca de duas horas depois do início do vazamento, e foi desligado cinco minutos depois, por política da empresa.

Os equipamentos de emergência eram insuficientes — faltavam máscaras de oxigênio, fato descoberto apenas após o acidente — e os operadores não recebiam treinamento adequado para lidar com situações não rotineiras. Quando a polícia chegou à usina, atraída pelo pânico na cidade, o responsável pela planta inicialmente negou o vazamento e chegou a afirmar que o MIC não era perigoso.

15.4 Uma Nova Perspectiva Sobre a Culpa

Analisar apenas os eventos do capítulo anterior conduzia quase inevitavelmente à conclusão de que o operador recém-contratado era o culpado, pois é sempre uma pessoa que está fisicamente próxima de um evento. Ao examinar as condições da usina, contudo, essa conclusão se revela insuficiente: o procedimento de manutenção não exigia o slip blind, a supervisão fora eliminada por corte de custos, os medidores estavam descalibrados, os alarmes haviam sido dessensibilizados deliberadamente, e múltiplos dispositivos de defesa em profundidade continham erros de projeto que os tornavam ineficazes mesmo quando ligados. A visão do acidente muda substancialmente quando se passa de "quem errou" para "por que o sistema permitia que esse erro se tornasse catastrófico".

Ainda assim, restam perguntas: por que o sistema de refrigeração foi desligado? Por que ninguém supervisionava a manutenção nas tubulações? Por que a empresa tolerou repetidos erros de projeto em seus próprios dispositivos de segurança? Por que não havia um grupo dedicado exclusivamente à segurança operacional? Essas perguntas apontam para um nível ainda mais profundo de análise — os fatores sistêmicos da organização, tema do próximo capítulo.

Síntese do Capítulo

- Scrubber, torre de chamas e cortina de água falharam por erros de projeto e manutenção deficiente, não por falha aleatória de componentes.
- O sistema de refrigeração estava desligado e o alarme de temperatura havia sido recalibrado de 11°C para 20°C, eliminando o aviso antecipado.
- O tanque 610 excedia o limite de preenchimento permitido, e os medidores de nível do tanque de backup indicavam valores incorretos.
- O checklist de manutenção não previa a inserção do slip blind, e a supervisão da operação havia sido eliminada por corte de custos.
- Alarmes soavam com tanta frequência no cotidiano que perderam credibilidade, e o alarme de emergência real demorou duas horas para ser acionado.
- Analisar as condições da usina desloca a atribuição de culpa de um erro individual para falhas estruturais de projeto, manutenção e gestão.

Capítulo 16

O Desastre de Bhopal: Fatores Sistêmicos e Lições Aprendidas

Os capítulos anteriores reconstruíram o acidente de Bhopal em dois níveis: os eventos que compuseram a cadeia causal imediata e as condições inseguras da usina que tornaram essa cadeia possível. Resta o terceiro e mais profundo nível de análise: os fatores sistêmicos, isto é, o conjunto de causas inerentes ao sistema organizacional como um todo, e não a um componente ou indivíduo isolado. Diferentemente das condições, que costumam se ligar de forma quase linear a um evento específico — um alarme recalibrado que deixa de disparar a tempo, por exemplo —, os fatores sistêmicos envolvem causalidade indireta, distribuída entre pessoas, cultura organizacional, políticas e pressões econômicas. Por isso mesmo, são os fatores mais frequentemente omitidos nos relatórios de acidentes, e também os mais importantes: representam lições genéricas, aplicáveis a praticamente qualquer indústria, e são exatamente o que precisa ser corrigido para evitar que acidentes semelhantes se repitam.

16.1 Pressão Econômica e Corte de Custos

O pano de fundo financeiro é essencial para entender Bhopal. Em 1981, três anos antes do acidente, a demanda por isocianato de metila (MIC) caiu drasticamente, reduzindo a produção da usina para menos da metade de sua capacidade. A matriz da Union Carbide, nos Estados Unidos, pressionou a subsidiária indiana para cortar despesas, sem no entanto especificar como isso deveria ser feito. A gestão local respondeu cortando pela metade a equipe de manutenção, sob a justificativa de que estava apenas eliminando desperdícios, sem afetar a segurança.

O corte de custos teve dois efeitos silenciosos e cumulativos. Primeiro, os funcionários mais qualificados — muitos deles com formação técnica sólida e treinamento nos Estados Unidos — começaram a pedir demissão em busca de empresas onde a segurança fosse mais valorizada, sendo substituídos por profissionais menos experientes, como um engenheiro júnior no lugar de um especialista sênior. Segundo, decisões técnicas relevantes passaram a ser justificadas por economia: cinco meses antes do acidente, os gestores desligaram o sistema de refrigeração alegando que ele nunca havia funcionado adequadamente e que a economia gerada compensava o risco. Não houve consenso posterior sobre se a matriz americana chegou a aprovar formalmente essa decisão. Na mesma linha, o alarme de temperatura do tanque foi recalibrado de 11°C para 20°C, e o registro histórico de temperaturas do tanque foi descontinuado.

16.2 Erosão da Manutenção, Educação e Supervisão Técnica

Os procedimentos de manutenção foram severamente reduzidos, e a troca de turnos passou a depender da disponibilidade de substitutos: quando ninguém aparecia, o turno seguinte ficava sem qualquer supervisão. Paralelamente, uma decisão política do governo indiano determinou que apenas indianos poderiam trabalhar na usina, o que interrompeu o fluxo de treinamento intensivo nos Estados Unidos e as inspeções regulares de engenheiros norte-americanos. Por volta de 1982, por razões financeiras, a supervisão direta de segurança feita pela matriz foi encerrada, e depois desse ano nenhum consultor americano permaneceu residente em Bhopal. O treinamento para lidar com situações não rotineiras e emergências tornou-se praticamente inexistente.

Essa desqualificação progressiva também atingiu o perfil de recrutamento: na inauguração da usina, operadores e técnicos tinham ao menos dois anos de formação superior em química ou engenharia química, além de seis meses de treinamento intensivo. Com as perdas financeiras, esses padrões caíram drasticamente. Em 1983, um engenheiro químico pediu demissão em protesto contra o afrouxamento dos controles de segurança e foi substituído por um engenheiro elétrico — um exemplo emblemático de como a experiência técnica relevante foi sendo substituída por mão de obra genérica e menos preparada.

16.3 Autoestima, Moral e Sinais de Alerta Ignorados

A soma desses fatores corroeu a autoestima e a satisfação dos trabalhadores. A moral da equipe caiu, a insatisfação com o emprego aumentou, e não havia entre os funcionários a crença de que a gestão estivesse atenta aos detalhes que garantiam uma operação segura — ao contrário, a percepção era de que a empresa tomava medidas drásticas de corte sem considerar as consequências. Com isso, as questões de segurança foram sistematicamente relegadas a segundo plano diante das prioridades financeiras.

Esse ambiente organizacional favoreceu ainda a persistência de condições inseguras adicionais: o MIC armazenado apresentava contaminação por clorofórmio de quatro a cinco vezes acima do especificado, sem qualquer correção; os tanques não estavam devidamente lacrados; equipamentos básicos de proteção, como luvas e máscaras, raramente eram usados, em parte por causa do calor extremo da região; e as inspeções e auditorias internas eram raras e superficiais.

O aspecto mais grave, porém, foi a reiterada omissão diante de sinais de alerta explícitos. Uma auditoria realizada em 1982 — dois anos antes do acidente — já identificava boa parte dos problemas que viriam a se manifestar na tragédia: operação de tubulações sem uso de *slip blind*, válvulas vazando com risco de contaminação do tanque, falhas no *scrubber* e nos medidores de pressão e temperatura, capacidade insuficiente da cortina de água e a torre de chamas fora de operação no momento da auditoria. A recomendação de aumentar a capacidade da cortina de água nunca foi implementada. Apesar disso, o gerente da unidade declarou, à época, que todas as deficiências haviam sido corrigidas — o que se mostrou falso. Entre 1981 e 1984,

outros incidentes envolvendo MIC já haviam ocorrido: um trabalhador morreu em 1981 em um acidente cujo processo foi arquivado para minimizar a responsabilidade da empresa; um vazamento semelhante ocorreu um ano antes da tragédia final; um jornalista local alertou repetidamente a empresa sobre os riscos da usina; e ao menos uma autoridade governamental que tentou levantar essas questões acabou pressionada a pedir demissão. As autoridades locais, mesmo cientes dos alertas, não tomaram nenhuma providência.

Fatores Sistêmicos e a Diluição da Responsabilidade

Um hazard — como o contato do MIC com água — não é, por si só, suficiente para causar um acidente catastrófico: é preciso que ele se combine com o pior conjunto possível de condições ambientais e organizacionais simultâneas. Em Bhopal, essa combinação não foi acidental: cada condição de defesa comprometida (tanque de backup mal calibrado, scrubber inoperante, alarmes dessensibilizados) tinha origem em decisões sistêmicas — cortes de custo, erosão de treinamento, queda de moral, sinais de alerta ignorados — tomadas ao longo de anos, por múltiplas pessoas, em múltiplos níveis da organização. Fatores sistêmicos são, por definição, difíceis de atribuir a um único responsável; corrigi-los exige mudanças estruturais na cultura de segurança, nas políticas e na governança da organização, não a substituição de um componente ou a punição de um operador.

16.4 Fatores Sistêmicos Além de Bhopal

A questão central deste último exercício da sequência é se esses fatores — pressão de custos, queda na qualificação técnica, perda de autoestima dos trabalhadores, sinais de alerta ignorados — são particulares ao caso de Bhopal ou se aparecem em outras indústrias. A resposta é clara: nenhum desses fatores é exclusivo de Bhopal. Eles se repetem, com variações, em acidentes industriais de naturezas muito diferentes, o que os torna a correção mais valiosa disponível para quem deseja atacar causas em vez de sintomas. Entre eles, o hábito de ignorar sinais de alerta é particularmente universal: em Bhopal esses sinais foram numerosos e inequívocos — auditoria formal, mortes anteriores, alertas da imprensa e de autoridades —, e ainda assim foram descartados, muitas vezes sob justificativas financeiras ou de competitividade que jamais deveriam prevalecer diante do risco à vida humana.

Síntese do Capítulo

- Fatores sistêmicos são causas globais do sistema organizacional, de causalidade indireta e difícil atribuição individual, frequentemente omitidos em relatórios de acidentes apesar de serem os mais importantes de corrigir.
- A pressão por corte de custos após a queda de demanda pelo MIC levou à redução da equipe de manutenção, à saída de profissionais qualificados e ao desligamento do sistema de refrigeração por economia.

- O fim da supervisão técnica americana após 1982 e a queda nos padrões de recrutamento e treinamento esvaziaram a capacitação da equipe local para lidar com emergências.
- A queda de moral e autoestima dos trabalhadores refletia a percepção de que a gestão priorizava cortes financeiros em detrimento da segurança.
- Sinais de alerta claros — auditoria de 1982, mortes anteriores, alertas da imprensa e de autoridades — foram sistematicamente ignorados nos anos que antecederam o acidente.
- Nenhum desses fatores sistêmicos é exclusivo de Bhopal: eles se repetem em outras indústrias e representam a lição mais genérica e mais valiosa de todo o caso.

Parte IV

Modelos de Causalidade de Acidentes

Capítulo 17

Safety versus Security: Definições Fundamentais

Antes de estudar os modelos de causalidade de acidentes que serão apresentados nos próximos capítulos, é necessário fixar com precisão o que se entende por *safety* neste livro — e por que essa palavra não é sinônimo de *security*, ainda que o português traduza ambas como “segurança”.

17.1 O Conceito de Acidente

Toda definição de *safety* parte de um conceito anterior: o de acidente. Um acidente é uma perda inaceitável — qualquer evento não planejado ou indesejável que resulta em dano. Essa definição é intencionalmente ampla e abrange ferimentos e mortes de pessoas, danos à propriedade, danos ambientais, perda de uma missão, impactos negativos ao negócio, atrasos no lançamento de um produto, complicações jurídicas, entre outras consequências.

Ao longo deste livro, os termos acidente, *mishap* e *loss* (perda) são tratados como sinônimos. Cada um é mais comum em determinada indústria — *mishap*, por exemplo, é amplamente usado no meio militar —, mas todos designam a mesma ideia central. A definição de acidente adotada aqui cobre tanto perdas causadas de forma não intencional quanto perdas causadas de forma proposital, o que é importante porque essa mesma definição servirá de base tanto para *safety* quanto para *security*.

17.2 Definindo Safety

Com o conceito de acidente estabelecido, a definição de *safety* pode ser enunciada de forma direta.

Safety

Safety é a ausência de perdas (*losses*). O conceito está associado aos objetivos de um sistema e às restrições impostas sobre como esse sistema pode atingir tais objetivos.

Restrições de sistema limitam a forma como os objetivos podem ser alcançados. Considere uma indústria química cujo objetivo é produzir determinados compostos: ela não pode fazer isso de qualquer maneira. Existem restrições — impostas por controles de engenharia, regulatórios e operacionais — que impedem, por exemplo, que a produção química polua o ambiente ou coloque em risco a vida das pessoas que

vivem ao redor da planta. Essas restrições operacionais são o que, na maior parte do tempo, associamos à palavra *safety*.

Ainda assim, há sistemas em que *safety* não é apenas uma restrição, mas o próprio objetivo do sistema — o controle de tráfego aéreo é um exemplo típico, em que garantir a separação segura entre aeronaves é a razão de existir do sistema, não uma limitação lateral imposta a ele.

17.3 A Diferença entre Safety e Security

O inglês distingue *safety* de *security*; o português usa uma única palavra, "segurança", para ambas. A diferença entre os dois conceitos não está no tipo de perda envolvida — a definição de acidente serve para os dois —, mas na natureza da ameaça que provoca essa perda.

Uma forma prática de diferenciá-los é observar como profissionais de cada área enxergam seu próprio papel. Especialistas em *safety* veem sua função como a de prevenir perdas decorrentes de ações não intencionais, cometidas por atores benevolentes — pessoas que não tinham a intenção de causar dano, mas o causaram por erro, falha ou circunstância. Especialistas em *security*, por sua vez, veem seu papel como o de prevenir perdas decorrentes de ações intencionais, cometidas por atores malevolentes — agentes que buscam deliberadamente causar dano ao sistema.

Essa distinção pelo intuito do agente causador é o critério mais útil para separar os dois campos: mesma perda, mesmo sistema, mas intenções radicalmente diferentes por trás da causa. Um mesmo incidente — a queda de um servidor, por exemplo — pode ser tratado como problema de *safety* se resultou de uma falha de hardware não intencional, ou como problema de *security* se resultou de um ataque deliberado. Reconhecer essa fronteira é essencial para que o analista escolha as técnicas de análise adequadas a cada caso, tema que será retomado nos capítulos seguintes deste livro.

Síntese do Capítulo

- Um acidente é uma perda inaceitável: qualquer evento não planejado que gera dano a pessoas, propriedade, ambiente, missão ou negócio.
- Acidente, *mishap* e *loss* são sinônimos usados neste livro, cada um mais comum em determinada indústria.
- *Safety* é definida como ausência de perdas, associada aos objetivos e às restrições de um sistema.
- Na maioria dos sistemas, *safety* corresponde a uma restrição operacional; em alguns, como o controle de tráfego aéreo, *safety* é o próprio objetivo do sistema.
- *Safety* trata de perdas por ações não intencionais de atores benevolentes; *security* trata de perdas por ações intencionais de atores malevolentes.
- A distinção entre os dois campos está no intuito por trás da causa da perda, não no tipo de perda em si.

Capítulo 18

O Modelo de Cadeia de Eventos

Com as definições de safety e security estabelecidas, este capítulo introduz o primeiro modelo de causalidade de acidentes que todo analista encontra na prática: o modelo de cadeia de eventos. Trata-se do modelo mais antigo e mais amplamente usado na história da engenharia de safety, e é a base subjacente de praticamente todas as técnicas tradicionais de análise de risco ainda empregadas hoje.

18.1 O que é um Modelo de Causalidade

Um modelo é uma abstração da realidade — não é a própria realidade. Modelos existem para que possamos lidar com um mundo complexo e confuso, filtrando as informações relevantes para um determinado problema e descartando o restante. Um modelo de causalidade de acidentes é um modelo que explica como um sistema funciona e que nos ajuda a prever como ele vai se comportar no futuro.

Um princípio importante deve acompanhar o leitor ao longo de todo este livro: não existe modelo certo ou modelo errado. Podemos apenas comparar a efetividade e a utilidade de diferentes modelos para um propósito específico. Por ser uma simplificação, todo modelo recorta do sistema apenas a parte de interesse para a análise em questão, descartando o que não é relevante para aquele problema específico.

18.2 A Estrutura do Modelo de Cadeia de Eventos

O modelo de cadeia de eventos tem mais de 200 anos de existência e é o modelo subjacente a diversas outras representações — com nomes e notações diferentes — que serão apresentadas adiante. Por estar quase sempre associado a eventos de falha, também é chamado de modelo de cadeia de eventos de falha.

Modelo de Cadeia de Eventos

O modelo de cadeia de eventos representa um acidente como uma sequência linear no tempo: o evento 1 causa o evento 2, que causa o evento 3, e assim sucessivamente até uma consequência final. A relação entre eventos consecutivos é de necessidade e suficiência — tipicamente, uma falha F2 ocorre se e somente se a falha F1 ocorreu antes dela. A característica central do modelo é assumir causalidade linear, como uma linha reta de eventos.

Dentro dessa cadeia, costuma-se identificar uma causa raiz (o evento inicial), eventos causais intermediários e um evento final, que corresponde à perda. O modelo agrada por sua simplicidade: um evento leva ao outro, até chegar a uma consequência, e o analista nunca se sente "perdido" ao segui-lo.

Um exemplo típico ilustra o funcionamento do modelo: a explosão de um tanque industrial. A umidade leva à corrosão do revestimento metálico; a corrosão enfraquece o metal; o metal enfraquecido, combinado com a pressão normal de operação, faz a pressão interna do tanque aumentar; o aumento de pressão trinca o metal, o que leva à ruptura do tanque; a ruptura libera ar comprimido e projéteis, que danificam equipamentos e ferem pessoas próximas.

18.3 Barreiras entre Eventos

Como o modelo assume que os acidentes são causados por uma sequência de eventos de falha, a estratégia natural de prevenção é eliminar os eventos ou inserir barreiras que impeçam que uma falha se propague para a próxima. No exemplo do tanque, cada elo da cadeia admite uma barreira específica: dessecantes reduzem a umidade; revestimentos de aço inoxidável evitam a corrosão; o design do tanque pode considerar sua localização em relação a pessoas e equipamentos; a pressão interna pode ser reduzida ao longo do tempo; diafragmas de proteção e malhas de contenção mitigam os efeitos de uma eventual ruptura; e a distância do tanque em relação a outras estruturas protege equipamentos e pessoas mesmo se o pior ocorrer.

Essa lógica de barreiras — impedir que um evento alimente o próximo — é o mecanismo de prevenção característico de todo modelo baseado em cadeia de eventos.

18.4 Encarnações do Modelo

Ao longo do século XX, diversos autores propuseram variações do modelo de cadeia de eventos, com notações e nomes distintos, mas mantendo a mesma suposição de base: causalidade linear.

O **modelo do Dominó**, proposto por Herbert William Heinrich em 1931, descreve uma sequência em que uma pessoa inserida em um ambiente comete uma falha, que leva a uma condição de perigo, que, combinada a más condições ambientais, resulta no acidente. Esse modelo ficou famoso, entre outras razões, por reforçar a tendência de atribuir a culpa ao erro humano.

O **modelo do Queijo Suíço**, de James Reason, reinterpreta a mesma lógica sob a perspectiva de defesa em profundidade: falhas latentes permanecem ocultas no sistema até que, alinhadas através de sucessivas camadas de defesa, resultem em um acidente inaceitável.

O **diagrama de Gravata Borboleta** (*bowtie*), criado na década de 1970 a partir da combinação de árvores de falha e árvores de eventos, recebeu esse nome pela semelhança visual com o acessório usado em cerimônias formais. No centro do diagrama está o evento crítico; à esquerda ficam as causas, associadas a árvores de falha e a barreiras proativas; à direita ficam as consequências, associadas a medidas mitigadoras e paliativas.

Dominó, Queijo Suíço e Gravata Borboleta usam nomes e notações gráficas diferentes, mas são, no fundo, o mesmo modelo de cadeia de eventos — todos assumem que acidentes decorrem de uma sequência linear de eventos de falha.

18.5 Os Limites do Modelo

Modelos de cadeia de eventos são fáceis de entender e continuam amplamente usados. A questão que se coloca não é se estão certos ou errados, mas se são simples demais para explicar acidentes nos sistemas complexos, sociotécnicos e ricos em software que caracterizam a engenharia atual. Se o objetivo é compreender e prevenir verdadeiramente as causas dos acidentes, é preciso ir além da cadeia de eventos e considerar fatores sistêmicos que essa estrutura linear não captura. É essa lacuna que motiva o estudo do modelo STAMP e das técnicas construídas sobre ele, como o STPA e o CAST, apresentadas em capítulos posteriores deste livro.

Síntese do Capítulo

- Um modelo de causalidade é uma abstração que simplifica a realidade para explicar e prever o comportamento de um sistema; não há modelo certo ou errado, apenas mais ou menos útil.
- O modelo de cadeia de eventos, com mais de 200 anos de existência, assume causalidade linear: um evento causa o próximo, até chegar a uma consequência final.
- A prevenção nesse modelo consiste em criar barreiras que impeçam a propagação de um evento para o seguinte, como ilustrado no exemplo da explosão de um tanque industrial.
- Dominó (Heinrich), Queijo Suíço (Reason) e Gravata Borboleta são encarnações distintas do mesmo modelo de cadeia de eventos, com notações e nomes diferentes.
- O modelo é simples e intuitivo, mas sua linearidade dificulta a captura de fatores sistêmicos relevantes em sistemas complexos e sociotécnicos.
- Essa limitação motiva a busca por modelos mais abrangentes, como o STAMP, tratado em capítulos posteriores.

Capítulo 19

A Sedução pela Causa Raiz

O capítulo anterior mostrou que o modelo de cadeia de eventos organiza um acidente como uma sequência linear, com um evento inicial — a chamada causa raiz — e um evento final, a perda. Este capítulo questiona a própria ideia de causa raiz e apresenta um conceito cunhado pelo professor John Carroll para descrever por que insistimos em buscá-la mesmo quando ela não explica o acidente de forma satisfatória.

19.1 A Arbitrariedade da Causa Raiz

Quando se usa o modelo de cadeia de eventos para explicar um acidente, é comum apontar como causa raiz o primeiro evento da cadeia. Mas essa escolha é arbitrária: é sempre possível inserir um evento anterior ao "primeiro" evento, retrocedendo a explicação indefinidamente. Definir onde a cadeia começa é uma decisão de quem analisa o acidente, não uma verdade objetiva embutida no sistema.

O desastre de Bhopal ilustra bem esse problema. O relatório oficial identificou como causa raiz a não inserção de um *slip blind* — um dispositivo de proteção da tubulação — no momento em que ela foi lavada para desentupimento. Essa conclusão recaiu sobre o operador responsável pela limpeza, que chegou a ser preso. No entanto, a mesma operação de lavagem deveria ser supervisionada por uma segunda pessoa, posição que havia sido eliminada da subsidiária da Union Carbide por corte de despesas. Nada impediria que o relatório apontasse como causa raiz a ausência dessa supervisão, e não a ação do operador. A escolha de qual evento é a "causa raiz" dentro de uma cadeia é, portanto, arbitrária.

19.2 A Sedução pela Causa Raiz

Sedução pela Causa Raiz

Termo cunhado pelo professor John Carroll para descrever a tendência de buscar e fixar uma causa raiz simples e conveniente para um acidente — em geral atribuída a um operador ou a uma falha de componente —, mesmo quando fatores políticos, culturais, educacionais e econômicos, que não têm relação linear direta com os eventos, tiveram papel decisivo na origem da perda.

Esse apego à causa raiz não é acidental: o conceito atrai porque é simples, porque dá a sensação de controle e porque parece equivaler a "descobrir o problema". Mas concentrar a análise na causa raiz é, na prática, focar nos sintomas, não no processo sistêmico que produziu esses sintomas. Fatores como um design de baixo custo ou pressões de produtividade impostas pela gestão de uma empresa raramente aparecem

como "eventos" dentro da cadeia — ainda que sejam, muitas vezes, os verdadeiros responsáveis pelo acidente.

19.3 O Viés em Direção ao Operador

Como o modelo de cadeia de eventos concentra a atenção nos eventos mais próximos da consequência final, ele tende naturalmente a apontar uma pessoa como causa — afinal, por trás de quase toda falha de componente há alguém envolvido na operação, manutenção ou decisão. Foi exatamente o que ocorreu em Bhopal: a causa raiz identificada recaiu sobre o operador que lavava as tubulações.

Esse padrão é conveniente para fins judiciais e para encontrar um culpado, mas é contraproducente para quem deseja evitar acidentes futuros. Sistemas complexos são projetados de tal forma que o erro humano é esperado; culpar indivíduos não reduz a recorrência de acidentes se as condições sistêmicas que levaram àquele comportamento permanecerem intactas. O foco deveria estar em entender por que a pessoa agiu daquela forma, e não apenas em registrar que ela "errou". Há inclusive um efeito perverso: quando o operador mais próximo do evento morre no próprio acidente, a atenção se desloca ainda mais facilmente para uma causa raiz abstrata, o que convém a quem prefere não investigar fatores organizacionais mais profundos — a referência popular ao personagem Homer Simpson, que atribui a culpa a quem quiser, resume bem essa tentação.

19.4 Lição Aprendida

O modelo de cadeia de eventos é útil, mas não é suficiente. Não é possível entender por que os acidentes ocorrem sem entender as condições que levaram aos eventos e, mais importante ainda, os fatores sistêmicos enraizados no sistema sociotécnico como um todo. A cadeia de eventos responde apenas pelos eventos diretos e imediatos; ela não captura os níveis mais profundos de causalidade que operam por trás desses eventos. É justamente para suprir essa lacuna que se torna necessário avançar para modelos que tratem explicitamente de múltiplos níveis de causalidade — tema retomado nos próximos capítulos.

Síntese do Capítulo

- Definir a "causa raiz" como o primeiro evento de uma cadeia é uma escolha arbitrária: sempre é possível inserir um evento anterior.
- O desastre de Bhopal mostra que a mesma cadeia de eventos permite atribuir a causa raiz tanto ao operador quanto à ausência de supervisão eliminada por corte de custos.
- A "sedução pela causa raiz" (John Carroll) descreve a preferência por explicações simples que ignoram fatores políticos, culturais e econômicos por trás do acidente.

- Focar na causa raiz é focar nos sintomas, não no processo sistêmico que os produziu.
- Culpar operadores próximos ao evento não reduz acidentes futuros, pois erro humano é esperado em sistemas complexos.
- O modelo de cadeia de eventos, embora útil, não é suficiente: é preciso entender condições e fatores sistêmicos além dos eventos diretos.

Capítulo 20

Tipos de Acidentes e os Limites dos Modelos Tradicionais

Até aqui, este livro tratou o acidente como resultado de uma cadeia de eventos que remonta a uma causa raiz. Este capítulo dá um passo além: mostra que os acidentes podem ser classificados em dois tipos distintos, e que apenas um deles é bem explicado pelas técnicas tradicionais de análise de riscos. O segundo tipo — cada vez mais comum em sistemas complexos e ricos em software — expõe os limites desses modelos e justifica a necessidade de abordagens mais recentes, como o STAMP e a técnica STPA, discutidas em capítulos posteriores.

20.1 Dois Tipos de Acidentes

Um sistema é um conjunto de componentes — hardware, software, instituições, pessoas — que interagem entre si para atingir determinado objetivo. A partir dessa definição, é possível classificar os acidentes em dois grandes tipos.

O primeiro tipo é o **acidente por falha de componente**. Nele, um ou mais componentes falham, tipicamente por uma falha aleatória que ocorre durante a operação do sistema e que segue alguma distribuição de probabilidade. Esse conceito de falha está diretamente associado à confiabilidade dos componentes: quanto mais confiável um componente, menor a chance de ele falhar aleatoriamente ao longo de sua vida útil.

O segundo tipo é o **acidente por interação entre componentes**. Aqui, não há necessariamente um componente específico que tenha falhado; o problema surge da própria interação entre os elementos do sistema. Esse tipo de acidente está associado ao grau de acoplamento e complexidade do design dos sistemas sociotécnicos atuais — sistemas em que hardware, software, instituições e pessoas interagem em redes cada vez mais densas. Quanto maior o acoplamento e a complexidade, maiores os riscos de problemas de engenharia de sistemas que não se reduzem a uma falha pontual.

A introdução do software nesses sistemas complexos exacerbou os problemas de interação. Antes, boa parte dos acidentes podia ser rastreada a falhas físicas de componentes; com sistemas cada vez mais *software-intensivos*, o centro de gravidade dos problemas migrou para o design do sistema — incluindo o design do próprio software — que introduziu uma complexidade praticamente ilimitada nos designs baseados em software.

20.2 A Complexidade dos Sistemas Atuais

Técnicas como FTA, ETA, HAZOP, ARP, Bowtie e FMEA foram criadas há 50 ou 70 anos e ainda são amplamente utilizadas. Todas compartilham uma mesma premissa: os acidentes são causados por falha de componentes, e por isso são construídas sobre o modelo de cadeia de eventos de falha.

O problema é que, a partir da década de 1960, houve uma explosão na introdução de controle computadorizado nos sistemas complexos, e o software passou a aumentar exponencialmente a complexidade dos designs. Embora essa transformação tenha se consolidado a partir de meados da década de 1970, as técnicas de análise de risco em uso permaneceram, em sua maioria, as mesmas criadas décadas antes — desenhadas para um mundo de sistemas eletromecânicos, não para sistemas sociotécnicos ricos em software.

20.3 Exemplos de Acidentes sem Falha de Componentes

Uma série de casos reais ilustra que, em sistemas complexos atuais, nem sempre é possível apontar um componente que falhou.

Em um teste da Marinha com mísseis, um software foi projetado para relançar automaticamente um míssil caso houvesse um obstáculo — como uma antena — entre o míssil e o alvo. Ao disparar o míssil de teste com a antena no meio do caminho, o software fez exatamente o que fora programado para fazer: substituiu o míssil. Nenhum componente falhou; o problema estava nos requisitos, que não previram aquele cenário de teste.

No acidente da sonda polar de Marte, o software precisava desligar os motores de descida assim que sensores nas pernas da sonda detectassem contato com o solo. Durante o desdobramento das pernas, ruídos nos sensores foram interpretados pelo software como sinal de pouso — quando na verdade a sonda ainda estava a cerca de 40 metros de altura. Os motores foram desligados prematuramente e a sonda se espantou. Hardware e software funcionaram como projetados; a falha foi de comunicação entre requisitos, que não previram esse ruído.

No acidente com um avião A320, o reversor de empuxo só pode ser acionado quando a aeronave está em solo, condição detectada pelo software a partir do peso e da rotação das rodas. Em uma pista muito molhada, as rodas hidroplanaram, o toque foi suave demais e o software não reconheceu o pouso. Os pilotos não conseguiram acionar o reversor de empuxo e, ao acelerarem para tentar reduzir a velocidade, o software interpretou a ação como aceleração normal. O avião saiu da pista.

No estado de Washington, um dispositivo antifurto instalado em carros de locadoras desligava o veículo automaticamente se o GPS detectasse movimento com o motor desligado — situação típica de roubo por guincho. O problema é que, ao transportar os carros em balsas, o mesmo padrão de movimento com motor desligado disparou o dispositivo, e os carros ficaram inutilizados ao desembarcar. Ninguém havia pensado nesse cenário durante o projeto: um risco desconhecido, e não um componente com defeito.

No acidente da companhia aérea Red Wings, em Moscou (2012), ventos cruzados fizeram a aeronave pousar de lado, com um toque tão suave que o peso sobre as rodas não foi suficiente para o software reconhecer o pouso. Os pilotos, vendo visualmente que já estavam em solo, tentaram acionar o reversor de empuxo, mas o sistema interpretou o comando como aceleração. A aeronave saiu ao final da pista.

Em todos esses casos, nenhum componente individual falhou: cada elemento operou de acordo com os requisitos com os quais foi projetado. Ainda assim, os acidentes ocorreram.

20.4 Por Que as Técnicas Tradicionais Não Bastam

Acidente por Interação de Componentes

Acidente que ocorre não por falha de um componente específico, mas pela combinação de interações entre componentes que, individualmente, atendem a seus requisitos. É típico de sistemas sociotécnicos complexos e ricos em software, nos quais propriedades como *safety* e *security* emergem da interação entre partes, e não do comportamento isolado de cada uma delas — as chamadas propriedades emergentes.

As técnicas tradicionais de análise de riscos não estão erradas; funcionaram bem na época e no contexto em que foram criadas, quando os sistemas eram predominantemente eletromecânicos e os acidentes se explicavam majoritariamente por falhas de componentes. O problema é aplicá-las, sem complemento, aos sistemas complexos e ricos em software de hoje, nos quais uma parcela relevante dos acidentes decorre de interações inseguras, e não de falhas.

A complexidade cria incógnitas que não podem ser capturadas observando apenas o comportamento do sistema como uma cadeia de eventos de falha. É necessário entender também as condições que antecederam os eventos e os fatores sistêmicos que os produziram. Esse é o motivo pelo qual as técnicas tradicionais precisam ser complementadas por modelos capazes de capturar problemas de design que envolvam interação entre hardware, software e pessoas — motivação central do modelo STAMP e da técnica STPA, apresentados adiante neste livro.

Síntese do Capítulo

- Acidentes se dividem em dois tipos: por falha de componente (associado à confiabilidade) e por interação entre componentes (associado a acoplamento e complexidade de design).
- Técnicas tradicionais (FTA, ETA, HAZOP, ARP, Bowtie, FMEA) têm 50 a 70 anos e assumem que acidentes resultam de falha de componentes.
- Software aumentou exponencialmente a complexidade dos sistemas a partir da década de 1970, sem que as técnicas de análise acompanhassem essa mudança.

- Casos reais (míssil da Marinha, sonda de Marte, A320, balsas de Washington, Red Wings) mostram acidentes em que nenhum componente falhou — todos operaram conforme seus requisitos.
- A complexidade dos sistemas atuais gera propriedades emergentes de *safety* e *security* que não se explicam pelo comportamento isolado de componentes.
- Técnicas antigas continuam úteis, mas precisam ser complementadas por modelos capazes de capturar problemas de design e interação, não apenas falhas.

Capítulo 21

Confiabilidade, Software e a Integração Homem-Máquina: Lições Aprendidas

O capítulo anterior mostrou, por meio de vários exemplos, que nem todo acidente resulta de um componente que falhou. Este capítulo fecha a discussão sobre o modelo de cadeia de eventos: separa os conceitos de *safety* e confiabilidade, explica por que o software mudou radicalmente a engenharia de sistemas, argumenta pela integração entre operadores, software e hardware, e sintetiza as lições aprendidas antes de avançar para o modelo STAMP.

21.1 Safety Não é o Mesmo que Confiabilidade

É comum confundir sistema seguro com sistema confiável, mas os dois conceitos não coincidem. Existem três cenários possíveis: (A) sistemas não confiáveis, porém seguros; (B) sistemas confiáveis, porém inseguros; e (C) sistemas não confiáveis e inseguros ao mesmo tempo.

O cenário (B) é o mais revelador: reúne exatamente os casos discutidos no capítulo anterior, em que acidentes ocorreram mesmo sem que nenhum componente falhasse. Ou seja, confiabilidade — a garantia de que os componentes funcionam conforme especificado — não é suficiente para garantir *safety*.

Isso tem consequência direta sobre o alcance das técnicas de análise de risco. As técnicas FMEA e FMECA trabalham bem com o cenário (A): identificam componentes não confiáveis que, ainda assim, não colocam o sistema em risco. Já FTA, HAZOP e STPA são capazes de capturar o cenário (C), de não confiabilidade combinada com insegurança. Mas apenas a STPA — construída sobre o modelo STAMP, foco deste curso — é capaz de identificar sistematicamente o cenário (B): acidentes que ocorrem mesmo com todos os componentes funcionando exatamente como projetados. Esse é o principal argumento para avançar rumo a um modelo de causalidade que vá além da cadeia de eventos e leve em conta condições e fatores sistêmicos, e não apenas falhas pontuais.

21.2 Como o Software Revolucionou a Engenharia

Software não é um componente físico: é uma abstração, um design. Uma mesma máquina de propósito geral se transforma em máquina de propósito específico dependendo do software instalado nela — uma plataforma de negociação de ações ou um

sistema de comércio eletrônico, por exemplo, com o mesmo hardware por trás. Software é, portanto, design puro, abstraído de sua realização física. E não se diz que um design “falha” no mesmo sentido em que um componente físico falha: quando um software age de forma inesperada, o mais correto é dizer que ele atendeu exatamente aos requisitos que lhe foram dados — só que esses requisitos estavam incompletos ou equivocados.

Essa distinção é mais que semântica. Ela desloca o problema: em vez de procurar o componente defeituoso, é preciso investigar por que os requisitos não capturaram determinada condição de operação.

O software trouxe três mudanças estruturais para a engenharia de sistemas. Primeiro, o software, em sentido estrito, não falha — falham os requisitos que ele implementa. Segundo, o software viabilizou uma complexidade de sistema quase ilimitada, muito além da capacidade humana de antecipar todas as interações diretas e indiretas entre componentes; torna-se impossível, inclusive, realizar testes exaustivos que cubram todos os erros possíveis em operação. Terceiro, o papel do software em acidentes está quase sempre ligado a problemas de requisitos — suposições incompletas ou erradas sobre a operação do sistema, condições ambientais não previstas ou situações de controle não tratadas.

Um ponto central é que *safety* depende do contexto, e não pode ser avaliada isoladamente. O mesmo software usado no foguete Ariane 4, considerado seguro, foi reaproveitado no Ariane 5 e contribuiu para a explosão da espaçonave — o contexto operacional havia mudado. Da mesma forma, uma faca de manteiga é segura ou perigosa dependendo de quem a maneja: uma criança ou um adulto. Métricas de performance que avaliam o software de forma isolada, como níveis de integridade de software, não substituem uma análise holística do sistema como um todo — hardware, software e pessoas interagindo.

21.3 Integrando Pessoas, Software e Hardware

A abordagem tradicional de *safety* tende a atribuir acidentes ao erro do operador mais próximo do evento. A lição aprendida costuma ser superficial: mais treinamento, mais atenção, mais regras e procedimentos. Esse padrão aparece, por exemplo, no relatório oficial do acidente da American Airlines em Cali, na Colômbia, que descreveu que o sistema de gestão de voo (FMS) “ficou confuso” durante a navegação. Software não fica confuso no sentido literal — mas a formulação do relatório denuncia como é difícil, mesmo para investigadores, separar o comportamento do software do comportamento do sistema sociotécnico como um todo.

Erro Humano como Sintoma

Em sistemas sociotécnicos complexos, o erro do operador não deve ser tratado como a causa de um acidente, mas como um sintoma de condições sistêmicas mais profundas — procedimentos inadequados, pressões de produção, complexidade que excede a capacidade cognitiva humana de antecipar todas as interações do sistema. Atacar o sintoma (culpar ou retreinar o operador) não elimina as condições que geraram o erro, e por isso não previne acidentes futuros.

Um sistema é um conjunto de componentes que interagem para atingir um objetivo, e esses componentes incluem fatores humanos, software e hardware analisados de forma acoplada — não isolada. Boa parte do que entendemos por *safety* só pode ser compreendida pela interação entre esses elementos, não pelo comportamento de cada parte individual. Reconhecer isso implica também reconhecer que, dada a complexidade dos sistemas atuais, é praticamente impossível eliminar totalmente o erro humano; a resposta não é culpar o operador, mas projetar o sistema — considerando procedimentos, pressões de produção e outros fatores sistêmicos — de modo a reduzir a chance de que aquele erro ocorra e a mitigar suas consequências quando ocorrer. Não é possível reduzir acidentes de forma efetiva sem essa integração entre operadores, software e hardware.

21.4 Lições Aprendidas do Modelo de Cadeia de Eventos

Os exercícios sobre o desastre de Bhopal e os exemplos discutidos ao longo destes capítulos convergem para um conjunto de lições que encerram a discussão sobre o modelo de cadeia de eventos.

Primeira lição: é preciso olhar além dos eventos para prevenir acidentes. O modelo de cadeia de eventos de falha não é suficiente para explicar os acidentes que ocorrem em sistemas complexos atuais; é necessário entender também as condições e os fatores sistêmicos que antecederam os eventos.

Segunda lição: os acidentes de sistemas complexos atuais nem sempre resultam de falha de componentes. Muitos estão associados a falhas de design, inclusive quando cada componente individual funcionou exatamente como especificado.

Terceira lição: o software contribui de forma diferente do hardware para os acidentes. O problema central, na maioria dos casos, não está no software em si, mas no levantamento e na definição de requisitos.

Quarta lição: não é possível atacar *safety* de forma efetiva analisando operadores, software e hardware isoladamente. É necessária uma visão holística de sistema, apoiada por modelos e técnicas que tratem o conjunto de forma integrada — o que motiva o estudo do modelo STAMP e da técnica STPA nos capítulos seguintes.

Síntese do Capítulo

- Safety e confiabilidade são conceitos distintos: existem sistemas não confiáveis porém seguros, confiáveis porém inseguros, e não confiáveis e inseguros ao mesmo tempo.
- Apenas a técnica STPA, entre as analisadas, identifica sistematicamente acidentes que ocorrem sem falha de componentes (confiável, porém inseguro).
- Software é design abstrato, não um componente físico: quando age de forma inesperada, o problema normalmente está nos requisitos, não em uma "falha" do software.

- Software viabilizou complexidade quase ilimitada nos sistemas, tornando testes exaustivos inviáveis e o comportamento do sistema dependente do contexto.
- Erro humano é sintoma de condições sistêmicas, não causa isolada; culpar o operador não previne acidentes futuros.
- Safety exige integração holística entre pessoas, software e hardware, e não a análise isolada de cada componente — motivação central para o modelo STAMP.

Parte V

O Modelo STAMP

Capítulo 22

Mudança de Paradigma: Safety como Problema de Controle

Este capítulo apresenta a mudança de paradigma central desta parte do livro: abandonar a visão de *safety* como problema de confiabilidade e falha de componentes, e passar a tratá-la como um problema de controle, no qual se impõem restrições de *safety* sobre o sistema analisado. Antes de chegar a essa mudança, discutimos por que os modelos tradicionais se tornaram insuficientes e revisamos três formas clássicas de lidar com complexidade — decomposição analítica, estatística e teoria de sistemas —, mostrando por que a última é a base teórica mais adequada para os sistemas complexos atuais.

22.1 A Necessidade de Novos Modelos e Técnicas

Os modelos e técnicas tradicionais de *safety* são construídos sobre o conceito de confiabilidade e sobre o modelo de cadeia de eventos. Esses modelos foram úteis no passado, mas os exemplos de acidentes discutidos ao longo do curso mostram que eles não são suficientes para explicar as causas de acidentes nos sistemas complexos de hoje.

Os novos problemas enfrentados pela engenharia de *safety* não se resumem a falhas de hardware. Eles envolvem também questões de software (em especial requisitos), fatores humanos, interações diretas e indiretas, *feedback loops* complexos entre componentes, problemas de design de sistema, de gestão, de regulação e políticas, fatores ambientais e situações inesperadas — além das interações complexas entre todos esses elementos. Diante desse cenário, é necessário um modelo e um conjunto de técnicas que não se limitem a tratar problemas de confiabilidade, mas que tratem *safety* como problema de controle, impondo restrições de *safety* sobre o sistema analisado.

22.2 Três Formas de Lidar com Complexidade

Antes de apresentar o novo modelo, vale posicionar três abordagens clássicas para lidar com complexidade: decomposição analítica, estatística e teoria de sistemas.

22.2.1 Decomposição Analítica

A decomposição analítica segue o paradigma de dividir para conquistar: um sistema complexo é dividido em partes menores — por critério físico/funcional ou por comportamento (cadeia de eventos) —, cada parte é analisada individualmente, e os resultados são depois combinados. Nessa visão, o todo é igual à soma das partes.

A separação física/funcional assume que os componentes interagem de forma direta, permitindo analisá-los isoladamente e depois recompor o entendimento do todo. Já a separação por cadeia de eventos organiza o sistema como uma sucessão de eventos no tempo, cada um causado pelo evento anterior — um efeito dominó que termina em uma perda inaceitável. Esse é o modelo clássico por trás das técnicas tradicionais de prevenção de acidentes e análise de *hazards*, como FMEA, FMECA, FTA e HAZOP. Um exemplo de decomposição funcional é o utilizado pela norma ARP 4761 (diretrizes de avaliação de *safety* para aeronaves), que decompõe uma aeronave em funções — controle de fluxo, potência, orientação, controle no solo, ambiente de cabine — e cada função em subfunções sucessivas.

A decomposição analítica só funciona sob uma suposição crítica: a de que dividir o sistema em partes não distorce a propriedade observada no todo. Isso exige que os componentes operem de forma independente e que as interações entre eles sejam diretas e lineares — sem *feedback loops* complexos ou interações não lineares. Para os sistemas complexos, altamente acoplados, automatizados e baseados em software que construímos hoje, essa suposição não se sustenta: a separação em partes menores frequentemente distorce a propriedade emergente que se deseja entender. É exatamente por isso que precisamos de uma nova base teórica.

22.2.2 Estatística

A segunda abordagem clássica é a estatística. Um modo útil de situar seu papel vem de uma obra clássica sobre teoria de sistemas, que divide o problema de complexidade em três regiões: *simplicidade organizada* (regida pelas leis da mecânica), *complexidade não organizada* (com aleatoriedade suficiente para o uso de estatística) e *complexidade organizada* (muito complexa para análise detalhada, mas pouco aleatória para uso de estatística).

Existe uma relação entre grau de aleatoriedade e grau de acoplamento dos componentes: sistemas mais simples, com menor acoplamento, podem ser tratados por decomposição analítica e pelas técnicas tradicionais de *safety*. Sistemas com maior aleatoriedade podem, em tese, ser tratados estatisticamente. Mas os sistemas atuais — que envolvem fatores humanos, design de sistema, software, legislação e pessoas — se encaixam na região de complexidade organizada, onde nem a decomposição analítica nem a estatística são suficientes.

22.2.3 Teoria de Sistemas

A terceira forma de lidar com complexidade — e a base de todo o restante deste livro — é a teoria de sistemas. Ela foi desenvolvida justamente para sistemas de complexidade organizada, nos quais a separação em subsistemas distorce o resultado observado: aqui, o todo é mais do que a soma das partes.

Teoria de Sistemas e Propriedades Emergentes

A teoria de sistemas trata sistemas cuja subdivisão em componentes menores distorce as propriedades de interesse. Diferentemente da decomposição analítica, ela assume que o todo é maior que a soma das partes, e que certas propriedades

— as *propriedades emergentes* — só existem no nível do sistema como um todo, surgindo das interações diretas e indiretas (incluindo *feedback loops*) entre seus componentes.

Para quem vem da engenharia de software, propriedades emergentes correspondem aos requisitos não funcionais: escalabilidade, usabilidade, disponibilidade, *safety*, *security* — todas as qualidades do sistema que não podem ser atribuídas a um único componente isolado. A teoria de sistemas não depende de grande volume de dados históricos, o que a torna especialmente adequada para sistemas novos, ainda em fase de projeto — diferentemente da estatística, que depende de dados acumulados. Historicamente, a teoria de sistemas foi empregada pela primeira vez em sistemas de mísseis balísticos nas décadas de 1950 e 1960.

A teoria de sistemas foca no sistema como um todo, e não em suas partes isoladas. Por isso, propriedades como *safety* só podem ser tratadas adequadamente de forma holística, levando em conta não apenas falhas físicas, mas também aspectos sociais e culturais.

22.3 A Mudança de Paradigma

Diante dessa análise, a resposta para os problemas complexos atuais está na teoria de sistemas — mas adotá-la exige uma mudança de paradigma: passar a tratar *safety*, *security*, ou qualquer propriedade emergente, como um problema de controle, e não de confiabilidade.

Em vez de se concentrar em prevenir falhas de componentes, o engenheiro de *safety* passa a trabalhar com o conceito de *control loop* (laço de controle), preocupando-se em impor restrições de *safety* sobre o sistema. Essa mudança se aplica a qualquer propriedade emergente — não apenas *safety*, mas também *security*, privacidade, escalabilidade, e assim por diante — situando-se em um contexto mais amplo, o da engenharia de sistemas.

É importante destacar que essa mudança de paradigma não invalida o que foi feito no passado. O valor de um novo paradigma depende justamente de sua capacidade de incorporar os sucessos e as lições aprendidas do paradigma anterior, oferecendo uma perspectiva mais ampla para interpretar o que já era conhecido.

22.4 Controlando Safety por Meio de Restrições

Uma vez adotada a mudança de paradigma, o objetivo passa a ser controlar a operação de sistemas complexos por meio de restrições de *safety*. Em um sistema complexo, os componentes interagem de formas diretas e indiretas, e um ou mais componentes atuam como *controladores*, impondo restrições de *safety* sobre o comportamento do sistema — seja de componentes individuais, seja das interações entre eles — por meio da emissão de ações de controle e da observação de *feedback*. Esse ciclo de ação e observação constitui o *control loop*: uma ação de controle muda o estado do sistema, o controlador recebe *feedback* sobre esse novo estado, e com base nele pode emitir novos comandos, mantendo continuamente a imposição das restrições de *safety*.

Restrições de *safety* costumam ser expressas como obrigações — em geral com a palavra "deve"(*must*). Alguns exemplos ilustram o conceito: a energia nunca deve estar ligada quando a porta de acesso a um equipamento perigoso estiver aberta (o princípio por trás dos sistemas de *interlock* usados em manutenção industrial); aeronaves ou veículos não devem violar uma separação mínima entre si; aeronaves devem manter sustentação suficiente para permanecer no ar; a integridade do casco de um submarino deve ser preservada; radiação ou elementos tóxicos não devem ser liberados de uma usina; trabalhadores não devem ser expostos a condições de perigo; o sistema de saúde pública deve prevenir a exposição da população a água ou produtos contaminados; e a pressão em uma plataforma de petróleo deve ser mantida sob controle.

Cada uma dessas restrições, ao ser identificada, se torna um dos primeiros requisitos — funcionais ou de *safety* — levados aos projetistas e arquitetos do sistema, que deverão considerá-los ao construir sistemas complexos capazes de atender às propriedades emergentes de *safety* e *security*.

Síntese do Capítulo

- Os modelos tradicionais de *safety*, baseados em confiabilidade e cadeia de eventos, não explicam adequadamente os acidentes em sistemas complexos atuais.
- Decomposição analítica (dividir para conquistar) supõe que a separação em partes não distorce o todo — suposição que falha em sistemas altamente acoplados e baseados em software.
- Estatística funciona bem em sistemas com alta aleatoriedade (complexidade não organizada), mas não nos sistemas sociotécnicos atuais.
- Teoria de sistemas trata da complexidade organizada, onde o todo é maior que a soma das partes e onde emergem propriedades como *safety* e *security*.
- A mudança de paradigma proposta é tratar *safety* como problema de controle — impor restrições de *safety* — em vez de problema de confiabilidade.
- Restrições de *safety* (expressas como "deve") tornam-se os primeiros requisitos levados a projetistas e arquitetos de sistemas.

Capítulo 23

Tratando Safety com o Laço de Controle

Tendo estabelecido a mudança de paradigma — tratar *safety* como problema de controle —, este capítulo mostra como o conceito de laço de controle (*control loop*) é usado, na prática, para explicar as causas de acidentes que a cadeia de eventos não conseguia capturar.

23.1 Revisão: Controle, Controlador e Control Loop

O conceito de controle aqui utilizado é o da engenharia de sistemas, e não o de comando hierárquico ou militar. Falhas de componentes e interações inseguras podem ser controladas em diferentes níveis: no nível de design (por exemplo, com *interlocks* e projetos *fail-safe*, que levam o sistema a um estado seguro), no nível de processo (como procedimentos de manutenção) e no nível social (legislação, regulamentações, questões jurídicas).

O *control loop* básico é composto por um **controlador**, que contém um modelo de processo e um algoritmo de controle, e por um **processo controlado**. O controlador atua sobre o processo controlado por meio de *control actions* (ações de controle, implementadas na engenharia através de atuadores) e recebe informação sobre o estado do processo por meio de *feedback* (implementado através de sensores). O controle, na teoria de sistemas, é hierárquico: componentes situados em níveis superiores controlam componentes de níveis inferiores.

O controlador toma decisões com base em seu modelo de processo — sua representação interna do estado do processo controlado — e em um algoritmo (ou procedimento, no caso de um controlador humano) que determina quais ações de controle emitir. Boa parte dos acidentes analisados por este modelo ocorre justamente quando o modelo de processo do controlador está dessincronizado do estado real do processo controlado.

Control Loop

Um laço de controle (*control loop*) é composto por um controlador — que mantém um modelo de processo e um algoritmo de controle — e um processo controlado, ligados por ações de controle (do controlador para o processo) e por *feedback* (do processo para o controlador). Diferentemente do modelo de cadeia de eventos, o *control loop* não se limita a capturar falhas de componentes: ele também permite capturar erros humanos, falhas de software, problemas de requisitos e falhas de design.

23.2 Explicando Acidentes com o Control Loop

A força do modelo de *control loop* está em explicar acidentes nos quais nenhum componente individual falhou, mas que ocorreram mesmo assim — casos que o modelo de cadeia de eventos não consegue explicar adequadamente. A seguir, revisitamos alguns exemplos já discutidos, agora sob a ótica do *control loop*.

23.2.1 Sonda Polar de Marte

Neste caso, o controlador é o software da aeronave e o processo controlado é a sonda. Sensores instalados nas pernas da sonda emitiram um *feedback* indicando pouso. Com base nesse *feedback*, o modelo de processo do software passou a indicar que a sonda havia aterrissado — e, segundo o algoritmo de controle, uma vez pousada, os motores (que geravam empuxo reverso para suavizar a descida) deveriam ser desligados. Os motores foram desligados, mas a sonda ainda estava a vários pés de altura: ela caiu em queda livre e se espatifou no solo. O problema não foi uma falha de componente, mas um modelo de processo incorreto.

23.2.2 Reversor de Empuxo em Varsóvia

Aqui o *control loop* envolve dois controladores — o piloto (humano) e o software — e o processo controlado é a aeronave. Em dia de chuva forte, o toque das rodas na pista foi tão suave que os sensores não o detectaram: o modelo de processo do software indicava que a aeronave ainda não havia pousado. Os pilotos, no entanto, viam visualmente a pista e sabiam que já haviam pousado; seu modelo de processo indicava pouso efetuado. Com base nisso, o piloto acionou o reversor de empuxo para ajudar na frenagem — mas o software ignorou o comando, pois seu algoritmo não permite ativar o reversor em voo. O resultado foi a inconsistência entre os modelos de processo dos dois controladores: o reversor não foi ativado, e a aeronave colidiu no final da pista.

23.2.3 Reversor de Empuxo em Moscou

O padrão se repete: o software, com base no toque leve das rodas, mantinha o modelo de processo de que a aeronave ainda estava no ar, enquanto o piloto, vendo a pista, sabia que já havia pousado. O piloto tentou acionar o reversor e, percebendo que a pista era curta, também aumentou a potência do motor para reforçar a frenagem. Só que, como o modelo de processo do software indicava que a aeronave ainda estava no ar, o aumento de potência apenas acelerou a aeronave no solo, agravando a situação. O resultado foi uma colisão no final da pista — um acidente causado pela combinação de modelos de processo divergentes entre os dois controladores, e não por falha de um componente isolado.

23.2.4 Fogo Amigo em Teste de Míssil

Neste exemplo, a estrutura de controle envolve vários controladores: a autoridade de comando, o piloto, o software de utilização do caça e a aeronave lançadora. A autoridade de comando ordenou um teste com míssil falso; o piloto executou o comando

de lançamento. Contudo, o algoritmo do software de utilização continha uma lógica adicional: se houvesse, na cena, um alvo em posição mais favorável para o disparo, o míssil correspondente àquela posição deveria ser lançado. Uma antena situada entre o caça e o alvo estava posicionada exatamente à frente do míssil falso. O algoritmo interpretou essa posição como mais vantajosa e substituiu o míssil a ser disparado, lançando um míssil real que atingiu o caça à frente. O piloto deu o comando correto (lançar o míssil falso), mas o algoritmo de utilização — parte do controlador automatizado — decidiu de forma diferente, com base em sua própria lógica interna.

Síntese do Capítulo

- Controle, na engenharia de sistemas, opera em múltiplos níveis: design, processo e social — não se restringe a comando hierárquico.
- O *control loop* é formado por controlador (modelo de processo + algoritmo) e processo controlado, ligados por ações de controle e *feedback*.
- Muitos acidentes ocorrem quando o modelo de processo do controlador diverge do estado real do sistema.
- Os casos da sonda de Marte e dos reversores de empuxo em Varsóvia e Moscou mostram acidentes causados por modelos de processo incorretos ou divergentes entre controladores, sem falha de componente.
- O caso do fogo amigo mostra como a lógica interna de um algoritmo de controle pode gerar uma ação de controle perigosa mesmo quando o comando humano foi correto.
- O *control loop* é um modelo mais poderoso que a cadeia de eventos: captura falhas de componentes, mas também erros humanos, falhas de software e de design.

Capítulo 24

Introdução ao Modelo STAMP

Este capítulo apresenta o modelo de causalidade de acidentes STAMP, sua origem, seu escopo e a forma como ele redefine a análise de *safety* em relação aos modelos tradicionais baseados em cadeia de eventos.

24.1 O que é STAMP

STAMP é a sigla para *System-Theoretic Accident Model and Processes*. Trata-se de um modelo de causalidade de acidentes baseado em teoria de sistemas, mais recente e mais poderoso que os modelos tradicionais construídos sobre cadeia de eventos. O modelo foi apresentado formalmente no livro *Engineering a Safer World* (2011/2012), da professora Nancy Leveson, embora seus fundamentos venham sendo pesquisados desde a década de 1990, com o livro *Software Safety*.

STAMP

STAMP (*System-Theoretic Accident Model and Processes*) é um modelo de causalidade de acidentes fundamentado em teoria de sistemas, e não em teoria de confiabilidade. Nele, acidentes e perdas são vistos como um problema de controle dinâmico — não como resultado de falhas de componentes. O objetivo passa a ser impor restrições de *safety* sobre o sistema analisado, por meio de laços de controle (*control loops*) que envolvem controladores, processos controlados, ações de controle e *feedback*.

Um conjunto de *control loops* trabalhando de forma coordenada forma uma **estrutura de controle**. O objetivo de uma estrutura de controle é garantir e impor restrições de *safety* sobre o processo controlado, evitando que ele entre em estados de alto risco. Formalmente, uma estrutura de controle hierárquica para *safety* impõe restrições sobre o comportamento do processo sendo controlado — tema que será aprofundado em capítulo posterior.

O modelo STAMP se aplica a sistemas muito complexos, que vão além de simples falhas de componentes de hardware: envolvem software, design de sistemas, fatores humanos, organizações e cultura. Sobre esse modelo é possível construir técnicas mais poderosas que as tradicionais, tanto para prevenir acidentes futuros quanto para analisar acidentes já ocorridos e aprender com seus fatores causais.

24.2 O que Está Incluído no Escopo de STAMP

Além dos cenários tradicionais capturados por métodos clássicos de análise de *hazards* — eventos de falha de componentes —, STAMP também lida com cenários associados à

interação entre componentes (propriedades que emergem dessas interações), problemas de software e de requisitos, erros de design de sistema, erros humanos e, de forma mais ampla, problemas oriundos de todo o contexto sociotécnico do sistema, não apenas de sua parte estritamente técnica.

Comparando os cenários capturados por STAMP com os capturados pelas ferramentas tradicionais de análise de *safety* (como FMEA, FMECA, FTA e HAZOP): as ferramentas tradicionais capturam um subconjunto de cenários relacionados a falhas de componentes. STAMP e suas técnicas capturam esse mesmo subconjunto, mas também vários outros cenários relacionados a software, erros de design e fatores humanos. Ou seja, o modelo e as técnicas construídas sobre STAMP são estritamente mais abrangentes.

24.3 Pensamento Orientado a Eventos versus Systems Thinking

Vale comparar o pensamento orientado a eventos — baseado em causalidade linear — com a prática da teoria de sistemas, conhecida como *systems thinking*.

No pensamento orientado a eventos, todo acidente pode, em tese, ser explicado por uma cadeia de causalidade entre eventos, e as causas raiz correspondem aos eventos que iniciam essa cadeia. Essa abordagem é sedutora, por ser simples e bem aceita socialmente — normalmente atribui-se a culpa às pessoas envolvidas no evento identificado como causa raiz. Como já discutido no exercício sobre o desastre de Bhopal, porém, a escolha do "evento raiz" é arbitrária e insuficiente para explicar as causas de acidentes em sistemas complexos.

No *systems thinking*, por outro lado, o pensamento é holístico: baseia-se em ciclos de *feedback*, ação e decisão, capturando relações diretas e indiretas entre componentes ou eventos. Aqui, o comportamento do sistema emerge da estrutura de seus *feedback loops*. As causas de um acidente, sob essa ótica, não são nós individuais isolados (um componente, um evento específico), mas forças que emergem da interação entre os elementos que compõem os *feedback loops* do sistema. Em resumo: o modelo STAMP captura mais tipos de causalidade do que o modelo de cadeia de eventos, que se limita à causalidade linear.

24.4 Acidentes que Exigem Causalidade Além da Linear

Três exemplos ilustram acidentes cuja explicação exige ir além da cadeia de eventos linear:

O desastre de Bhopal, já discutido em profundidade em exercício anterior, é considerado até hoje o pior acidente industrial da história. Ao analisá-lo apenas pela cadeia de eventos, perdem-se fatores causais sistêmicos essenciais — condições organizacionais da empresa, da cidade e da gestão — necessários para prevenir acidentes semelhantes.

O naufrágio do ferry *Herald of Free Enterprise*, em 6 de março de 1987, no trajeto entre Inglaterra e Bélgica, matou 187 pessoas: a embarcação tombou em alto-mar, os

veículos internos caíram e muitas pessoas morreram afogadas. O acidente é discutido em detalhe no Capítulo 2 do livro *Engineering a Safer World* (páginas 12 a 14), que apresenta as interações complexas envolvidas — interações que uma simples cadeia de eventos não capturaria.

A perda do ônibus espacial Columbia, em 1º de fevereiro de 2003, durante o reingresso na atmosfera, foi o segundo acidente fatal do programa espacial norte-americano, após o desastre da Challenger em 1986. O livro de Nancy Leveson discute o caso extensamente, com figuras que ilustram os *feedback loops* e fluxos causais envolvidos — mais uma vez, evidenciando a insuficiência do modelo de cadeia de eventos isolado.

Este capítulo, assim como os anteriores desta parte do livro, baseia-se nos tutoriais e apresentações da professora Nancy Leveson no STAMP Workshop — evento anual realizado em Cambridge, Massachusetts —, em particular na edição de 2021, e nos livros *Engineering a Safer World* e *STPA Handbook*.

Síntese do Capítulo

- STAMP (*System-Theoretic Accident Model and Processes*) trata acidentes como problema de controle dinâmico, fundamentado em teoria de sistemas, não em confiabilidade.
- Um conjunto de *control loops* coordenados forma uma estrutura de controle, responsável por impor restrições de *safety*.
- STAMP captura todos os cenários das técnicas tradicionais, além de cenários envolvendo software, design, fatores humanos e interações sociotécnicas.
- O *systems thinking* substitui a busca por uma causa raiz linear por uma visão holística de *feedback loops* e causalidade não linear.
- Bhopal, o naufrágio do *Herald of Free Enterprise* e a perda do ônibus espacial Columbia ilustram acidentes que a cadeia de eventos não explica sozinha.
- O modelo e os materiais desta parte do livro têm como referência central o trabalho da professora Nancy Leveson, apresentado no STAMP Workshop e em seus livros-texto.

Capítulo 25

Análise de Hazards e Acidentes com STAMP

25.1 Retomando o Control Loop

A análise de hazards e de acidentes sob o modelo STAMP tem como base o *control loop* básico: um controlador interage com um processo controlado por meio de *control actions*, que alteram o estado do processo, e de *feedback*, que informa ao controlador o estado e o comportamento resultante desse processo. Dentro do controlador, distinguem-se dois componentes internos: o algoritmo de controle e o modelo de processo. Com base no modelo de processo — a representação que o controlador mantém sobre o estado do sistema controlado — o algoritmo de controle decide quais ações emitir. Vale notar que o termo "algoritmo" é mais adequado a controladores automatizados; quando o controlador é humano, essa estrutura interna precisa ser repensada, como será discutido adiante.

Nos modelos STAMP, safety é tratado como um problema de controle: o objetivo é impor restrições de safety sobre o processo controlado, evitando que ele atinja um estado de risco. Por isso, o elemento central de qualquer análise de hazards ou de acidentes com STAMP é entender por que um controlador emite, ou deixa de emitir, uma control action sobre o processo que supervisiona.

25.2 O Papel Central das Control Actions

O conceito de control action é o eixo de qualquer análise de hazards conduzida com as técnicas construídas sobre STAMP, em especial a STPA (System-Theoretic Process Analysis). Nessa técnica, o analista busca inicialmente identificar quais ações de controle potenciais são inseguras: aquelas cuja emissão pode levar o sistema a um estado de risco, e aquelas cuja ausência — quando a ação seria necessária para manter o processo sob controle — também compromete a safety do sistema.

Quando a análise recai sobre um acidente já ocorrido, a técnica correspondente é a CAST (Causal Analysis based on System Theory). Além de identificar as ações de controle inseguras envolvidas no acidente, a CAST busca compreender por que essas ações foram fornecidas. Essa investigação das causas é indispensável: sem entender por que uma ação insegura foi emitida, não é possível eliminá-la ou, quando a eliminação não for viável, mitigar seus efeitos sobre o processo controlado. O objetivo final é sempre o mesmo — manter o comportamento do sistema fora do estado de alto risco que precede um acidente.

Há ainda uma segunda pergunta, tão importante quanto a primeira: quando uma ação de controle segura é de fato fornecida por um controlador, em que cenários ela

deixa de ser seguida, ou não chega a ser emitida a um controlador situado em um nível mais baixo da hierarquia? Responder a essa pergunta é essencial para mapear os cenários de causas que explicam falhas de controle ao longo da estrutura hierárquica, e não apenas na interação direta entre um controlador e seu processo.

Estrutura de Controle e Combinações de Controladores

Uma estrutura de controle é um conjunto de control loops que, combinados, impõem restrições de safety sobre o processo controlado, prevenindo que ele entre em um estado inseguro. Dentro dessa estrutura, os controladores em interação podem ser de naturezas distintas: software-hardware (por exemplo, um piloto automático emitindo comandos sobre atuadores físicos), humano-automação (a tripulação de voo interagindo com o piloto automático), humano-hardware (a tripulação interagindo diretamente com os freios de uma aeronave) e até humano-humano (uma agência reguladora, como a FAA, interagindo com fabricantes e companhias aéreas). Essa variedade de combinações mostra que a estrutura de controle de um sistema sociotécnico raramente se resume a uma única camada tecnológica: ela entrelaça componentes físicos, digitais, humanos e organizacionais.

25.3 STPA para Controladores Humanos

O modelo STAMP e a técnica STPA, formalizadas em 2011-2012 (ainda que fundamentadas em estudos anteriores, de meados da década de 1990), continuam evoluindo. Um exemplo relevante é a extensão proposta em 2017 pela então aluna de mestrado do MIT, Megan France, em sua dissertação "STPA for Humans".

No control loop tradicional, um controlador automatizado é internamente dividido em modelo de processo e algoritmo de controle. Como o termo "algoritmo" não descreve adequadamente a tomada de decisão humana, Megan France propôs uma divisão alternativa para controladores humanos, em três partes: seleção da ação de controle (*control action selection*), modelos mentais e atualização dos modelos mentais.

A lógica é a seguinte: a decisão de um ser humano sobre qual control action emitir depende do modelo mental que ele constrói sobre o estado do processo controlado, seu comportamento e o ambiente ao redor. Esse modelo mental precisa ser continuamente atualizado, formando uma espécie de "retrato" (snapshot) do processo em um dado instante. É esse snapshot, combinado a outras entradas, que orienta a seleção da ação de controle. A decisão humana está, portanto, sujeita a uma quantidade maior de variáveis do que a decisão de um controlador de software, e frequentemente o objetivo do operador humano diverge do objetivo previsto pelo projetista do sistema. Essa divergência torna difícil antecipar, em fase de projeto, todo o espectro de comportamentos possíveis de um controlador humano — tornando a análise de sistemas com humanos no loop significativamente mais complexa do que a análise de sistemas puramente automatizados.

25.4 Implicações para a Análise

A extensão de Megan France não abandona o control loop básico: ela apenas detalha internamente o componente "controlador" quando este é humano, permitindo capturar cenários de causas adicionais — ligados a modelos mentais desatualizados, incompletos ou incorretos — que ajudam a explicar por que ações de controle são ou não emitidas. Para o analista de safety, essa extensão reforça um princípio prático: ao investigar hazards ou acidentes envolvendo operadores humanos, é preciso examinar não apenas a ação emitida (ou omitida), mas também o modelo mental que a motivou e os processos de atualização desse modelo — muitas vezes o verdadeiro ponto de falha do sistema.

Síntese do Capítulo

- STAMP trata safety como problema de controle: o foco da análise está em por que control actions inseguras são emitidas ou control actions necessárias deixam de sê-lo.
- STPA identifica hazards antes do acidente; CAST investiga as causas de um acidente já ocorrido, incluindo por que ações inseguras foram fornecidas.
- Uma estrutura de controle combina múltiplos control loops e pode envolver interações software-hardware, humano-automação, humano-hardware e humano-humano.
- A dissertação de Megan France (STPA for Humans, 2017) estende o controlador humano em três partes: seleção da ação de controle, modelos mentais e atualização dos modelos mentais.
- Controladores humanos introduzem maior complexidade de análise, pois seus objetivos podem divergir dos previstos pelo projetista do sistema.

Capítulo 26

Estrutura Hierárquica para Controle de Safety

26.1 Sistemas como Estruturas Hierárquicas

A teoria de sistemas subjacente ao STAMP concebe um sistema como uma estrutura hierárquica, na qual cada nível impõe restrições sobre as atividades do nível imediatamente abaixo. A presença — ou a ausência — dessas restrições em um nível superior é o que permite ou controla o comportamento dos níveis inferiores. Processos de controle operam entre os níveis dessa hierarquia, e é por meio deles que as restrições de safety são impostas e mantidas.

Um acidente ocorre quando um processo de controle falha em impor controle adequado, e as restrições de safety acabam sendo violadas no comportamento dos componentes de nível inferior. Esse controle inadequado pode ter diversas origens: comandos de controle de safety mal formulados, comandos corretos que não são executados adequadamente nos níveis inferiores, feedback processado ou comunicado de forma inadequada, ou ainda restrições de comportamento mal impostas ao sistema como um todo.

Um exemplo concreto ilustra esses mecanismos de falha: considere um gerente de operações responsável por fornecer instruções de trabalho aos operadores sob sua supervisão. Mesmo que essas instruções imponham corretamente restrições de safety, nada garante que os operadores as sigam. Da mesma forma, o gerente pode não dispor de canais de feedback que lhe permitam identificar que instruções inseguras foram emitidas, ou que instruções relacionadas à prevenção de acidentes não estão sendo seguidas na prática.

26.2 A Estrutura Sociotécnica de Leveson

No livro *Engineering a Safer World*, Nancy Leveson apresenta uma estrutura de controle hierárquico para safety comum aos Estados Unidos, ilustrando o conceito de estrutura sociotécnica de controle. Essa estrutura se divide em duas partes que se comunicam constantemente: de um lado, a estrutura de controle associada ao desenvolvimento do sistema; de outro, a estrutura de controle associada à sua operação. A comunicação entre as duas partes é necessária porque questões de safety identificadas na operação retroalimentam decisões de desenvolvimento, e vice-versa.

A análise dessas estruturas segue uma lógica top-down: o analista parte de uma estrutura de controle em alto nível e vai detalhando progressivamente os componentes de interesse, até alcançar o nível de granularidade necessário para a análise em curso. Um exemplo típico parte da estrutura sociotécnica genérica — com sua camada de

operação — até chegar a uma estrutura de controle de baixo nível composta por três componentes específicos: um componente humano (o piloto), um componente automatizado (o software) e um componente físico (a aeronave).

26.3 Objetivos do Projeto de uma Estrutura de Controle

Ao projetar uma estrutura de controle de safety, o objetivo central é eliminar ou reduzir a ocorrência de eventos adversos. Para isso, é necessário estabelecer, em todos os níveis da hierarquia, uma definição clara de responsabilidades, de autoridade e de expectativas — e, igualmente importante, garantir que existam canais de feedback adequados em cada nível. Sem feedback funcional, um nível superior não tem como verificar se as restrições que impôs estão de fato sendo respeitadas nos níveis inferiores.

Estrutura de Controle e Restrições de Safety

A imposição de restrições de safety não é responsabilidade de uma única camada da estrutura de controle: toda a estrutura — projeto físico, operações, gestão, interações sociais e cultura organizacional — deve atuar em conjunto para garantir as propriedades emergentes de safety em nível de sistema. Como essas estruturas tendem a se tornar muito complexas, o uso de abstração é fundamental: ao examinar um hazard específico, apenas um subconjunto da estrutura de controle global costuma ser relevante, e o restante pode ser tratado como entrada ou ambiente para a subestrutura em análise, sem que seus detalhes internos precisem ser considerados. Ainda assim, a característica top-down da análise deve ser preservada: os hazards são identificados primeiro em nível sistêmico, e o processo de derivação de restrições de safety para as partes da estrutura deve seguir do geral para o específico — nunca o caminho inverso.

26.4 Complexidade e o Cuidado com o Caminho Bottom-Up

A abstração é uma ferramenta útil não apenas para conter a complexidade da análise, mas também para facilitar a comunicação sobre os controles necessários entre diferentes partes interessadas. Ainda assim, o analista deve resistir à tentação de iniciar a investigação pelos componentes de mais baixo nível. Partir de detalhes operacionais isolados, sem antes estabelecer os hazards em nível de sistema, compromete a coerência da estrutura de controle resultante e pode levar à omissão de restrições de safety que só fazem sentido quando vistas em um nível hierárquico mais alto.

Síntese do Capítulo

- Sistemas são estruturas hierárquicas em que cada nível impõe restrições sobre o nível inferior; acidentes decorrem de controle inadequado que viola essas re-

strições.

- Controle inadequado pode originar-se de comandos mal formulados, comandos mal executados, feedback deficiente ou restrições mal impostas.
- A estrutura sociotécnica de Leveson divide-se em desenvolvimento e operação, que se comunicam continuamente.
- Projetar uma estrutura de controle exige responsabilidades, autoridade, expectativas e feedback claramente definidos em cada nível.
- Toda a estrutura de controle — não apenas uma parte — deve atuar em conjunto para impor restrições de safety em nível de sistema.
- A análise deve ser sempre top-down: hazards identificados primeiro em nível sistêmico, com abstração usada para gerenciar a complexidade.

Capítulo 27

Técnicas STAMP: Eficácia e Aplicação na Indústria

27.1 Método, Técnica ou Ferramenta?

Antes de revisar as técnicas construídas sobre o modelo STAMP, vale uma observação terminológica. O próprio *STPA Handbook* não é consistente no uso dos termos "método", "técnica" e "ferramenta": em diferentes páginas, a STPA é chamada de método (por exemplo, por ser um processo iterativo), de técnica (como "técnica relativamente nova para análise de hazards") e de ferramenta ("as ferramentas mais amplamente utilizadas com base no modelo STAMP"). O mesmo ocorre com a CAST. Isso reflete o uso corrente entre os próprios criadores do modelo, incluindo Nancy Leveson e sua equipe, que não raramente alternam entre os três termos. Neste livro, adota-se de forma consistente o termo "técnica" para se referir a STPA, CAST, STPA-Sec e demais abordagens construídas sobre STAMP — mas o leitor deve estar ciente de que a literatura da área trata esses termos como sinônimos.

27.2 O Catálogo de Técnicas sobre STAMP

O modelo STAMP representa uma mudança de paradigma: safety passa a ser tratado como um problema de controle, cujo objetivo é impor restrições de safety sobre o sistema controlado, e não como um problema de confiabilidade voltado à prevenção de falhas de componentes individuais. Sobre esse modelo de causalidade sistêmica foram construídas diversas técnicas, e é esperado que outras surjam no futuro. Entre as mais conhecidas estão:

- **CAST** (Causal Analysis based on System Theory) — técnica para análise de acidentes já ocorridos.
- **STPA** (System-Theoretic Process Analysis) — técnica para análise de hazards, com o objetivo de identificar causas potenciais de acidentes antes que eles ocorram.
- **STPA-Sec** — extensão da STPA voltada para cyber security.

Além dessas, existem outras técnicas menos difundidas, como MODA (Model-Driven Engineering), System Engineering Leading Indicators e abordagens de análise de risco organizacional. Uma vez que se dispõe de um modelo subjacente e de técnicas construídas sobre ele, essas técnicas passam a ser aplicáveis em diferentes processos: engenharia de sistemas, gestão de riscos, operações, entre outros.

27.3 O Modelo e Suas Técnicas Funcionam? Evidências de Retorno de Investimento

Uma pergunta natural para qualquer engenheiro de safety é: essas técnicas realmente funcionam na prática? Hoje já existem centenas de avaliações e estudos comparativos entre STPA e abordagens tradicionais de análise de hazards, tanto em ambiente controlado quanto em aplicações reais na indústria. De forma consistente, esses estudos mostram que a STPA identifica mais requisitos críticos e mais falhas de projeto do que os métodos tradicionais, além de identificar mais acidentes reais — com uma demanda de recursos significativamente menor.

Quanto a retorno de investimento propriamente dito, os resultados ainda são mais escassos, mas já começam a se consolidar. Segundo a equipe do MIT responsável pelo desenvolvimento do modelo STAMP, contratantes da indústria de defesa relatam retorno de investimento na faixa de 15% a 20% ao adotar a STPA. Um estudo mais recente, de 2021, conclui que a STPA proporciona retorno de investimento positivo e significativo tanto em projetos pequenos quanto em projetos de grande porte.

27.4 Estudos de Caso em Sistemas Reais

Diversos estudos de caso documentam a aplicação da STPA em sistemas reais, com resultados relevantes para qualquer analista que avalie a adoção da técnica.

No sistema de defesa de mísseis balísticos MDA, um dos primeiros usos documentados da STPA em um sistema real de defesa (2005), o hazard analisado foi o lançamento não intencional de um míssil. A análise foi conduzida próximo à implantação e ao teste em campo, por duas pessoas sem familiaridade prévia com o sistema, ao longo de cinco meses. Foram identificados tantos caminhos possíveis para o lançamento não intencional que a implantação do sistema foi adiada em seis meses.

No sistema HUMS, de alerta em cockpits de helicópteros Black Hawk, os resultados da STPA foram comparados aos de uma equipe independente que utilizou técnicas tradicionais de safety. A STPA encontrou as mesmas causas de hazards identificadas pelas técnicas tradicionais, além de causas adicionais relacionadas a falhas de projeto, comportamento humano e interações entre componentes do sistema.

No sistema de posicionamento dinâmico de embarcações da Marinha — que havia sofrido duas colisões durante testes —, foram conduzidas análises CAST sobre os dois incidentes e uma análise STPA sobre o sistema como um todo. A STPA identificou cenários não encontrados por árvores de falha (FTA) e FMEA. Um almirante da Marinha rejeitou inicialmente os resultados, argumentando que a avaliação probabilística de risco vinha sendo usada com sucesso havia 40 anos; dois meses após o sistema entrar em operação, no entanto, ocorreu uma colisão com um submarino — exatamente um dos cenários previstos pela STPA.

Em um experimento controlado envolvendo o sistema de controle de uma usina nuclear, conduzido pelo Instituto de Pesquisa em Energia Elétrica, doze alunos de pós-graduação compararam, ao longo de duas semanas, as técnicas tradicionais FTA e FMEA com a STPA. Apenas a STPA foi capaz de identificar o acidente ocorrido em Three Mile Island — sem que os analistas tivessem conhecimento prévio desse evento histórico.

Na Embraer, um estudo sobre o sistema de controle de fumaça de uma aeronave identificou 43 situações de safety; dessas, 16 foram encontradas exclusivamente pela STPA. Já no caso do Range Extender System — um sistema que estende a autonomia de baterias de veículos elétricos por meio de um motor de combustão interna — a comparação entre um híbrido de técnicas tradicionais e a STPA mostrou que esta última exigiu cerca de três vezes menos esforço.

Evidência Empírica como Critério de Adoção

A adoção de uma nova técnica de engenharia de safety se justifica não apenas por argumentos teóricos, mas por evidência empírica acumulada. No caso da STPA, essa evidência combina dois eixos: capacidade de detecção (identificação de mais hazards, incluindo cenários não cobertos por FTA e FMEA, como nos casos da Marinha e de Three Mile Island) e eficiência de recursos (menor esforço e retorno de investimento mensurável, como nos casos do Range Extender System e dos contratantes de defesa). Para o analista de safety, esses dois eixos — o que a técnica encontra e a que custo — são os critérios centrais para justificar sua adoção em um projeto real.

27.5 Indústrias que Utilizam STAMP e Suas Técnicas

O modelo STAMP e suas técnicas, introduzidos em 2011 no livro *Engineering a Safer World*, são relativamente recentes, mas já se disseminaram por uma ampla gama de setores. Segundo a própria Nancy Leveson, hoje eles são aplicados em aviação, indústria automobilística, setor de defesa, transportes, hospitais, segurança do trabalho, indústria petroquímica, indústria espacial, setor de energia, indústria nuclear, indústria marítima, manufatura, comunicações, ferrovias, robótica, veículos não tripulados, controle de processos e mineração, entre outros. Essa disseminação, documentada em apresentações e tutoriais do STAMP Workshop, reforça que STAMP não é uma técnica de nicho, mas uma abordagem geral aplicável a qualquer sistema sociotécnico complexo em que safety seja uma preocupação central.

Síntese do Capítulo

- Os termos "método", "técnica" e "ferramenta" são usados como sinônimos na literatura sobre STAMP; este livro padroniza o uso de "técnica".
- As técnicas mais conhecidas construídas sobre STAMP são CAST (análise de acidentes), STPA (análise de hazards) e STPA-Sec (extensão para cyber security).
- Estudos indicam retorno de investimento de 15% a 20% na indústria de defesa, com resultados positivos tanto em projetos pequenos quanto grandes.

- Casos reais — MDA, HUMS, Marinha, indústria nuclear, Embraer e Range Extender System — mostram que a STPA encontra hazards não detectados por FTA e FMEA, muitas vezes com menor esforço.
- O caso da Marinha ilustra o risco de rejeitar resultados de STPA com base apenas em tradição metodológica: o cenário previsto se concretizou dois meses depois.
- STAMP e suas técnicas já são aplicados em dezenas de setores industriais, da aviação à mineração, confirmando seu caráter de abordagem geral de safety.

Parte VI

A Técnica STPA

Capítulo 28

Visão Geral da Técnica STPA

Este capítulo abre a parte do livro dedicada à técnica STPA (*System-Theoretic Process Analysis*), construída sobre o modelo de causalidade de acidentes STAMP. O objetivo aqui é apresentar, a 10.000 pés de altura, os quatro passos que compõem a técnica, para que os capítulos seguintes possam aprofundar cada um deles.

28.1 Os Quatro Passos da STPA

A STPA prescreve quatro passos sequenciais e iterativos:

1. Definir o objetivo da análise.
2. Modelar a estrutura de controle.
3. Identificar as ações de controle inseguras (*Unsafe Control Actions* — UCAs).
4. Identificar os cenários de perdas.

STPA

STPA é uma técnica de análise de hazards fundamentada no modelo STAMP, que trata safety como um problema de controle — e não apenas de confiabilidade de componentes. A técnica é executada em quatro passos, cada um produzindo artefatos rastreáveis entre si, e pode ser aplicada não somente a safety, mas a qualquer propriedade emergente de interesse, como security, privacidade ou performance.

28.2 Passo 1: Definir o Objetivo da Análise

Como em qualquer análise de hazards, o primeiro passo consiste em delimitar o que se pretende alcançar. Nesse momento, o analista busca responder três perguntas: quais são os tipos de perdas inaceitáveis que a análise deve prevenir; quais são os hazards de nível de sistema associados a essas perdas; e qual é o sistema sendo analisado, incluindo os limites que o separam do ambiente.

A STPA vai além das análises de safety tradicionais, que costumam mirar apenas a prevenção de perda de vida humana. É preciso decidir, já no Passo 1, se a técnica será aplicada com esse escopo mais restrito ou em sentido mais amplo, cobrindo qualquer propriedade emergente relevante — security, privacidade, performance, entre outras.

28.3 Passo 2: Modelar a Estrutura de Controle

Concluído o Passo 1, a STPA prescreve a modelagem da estrutura de controle: o conjunto de *control loops* responsável por impor as restrições de safety sobre o processo analisado. Essa estrutura hierárquica captura os relacionamentos funcionais entre os componentes do sistema, sendo formada por controladores, processos controlados, ações de controle (*control actions*) e realimentação (*feedback*).

A modelagem segue uma lógica *top-down*: começa-se em um nível mais genérico e, à medida que a análise avança, a estrutura é refinada para capturar mais detalhes. Esse passo permanece inalterado independentemente da propriedade emergente sob análise — safety, security ou qualquer outra.

28.4 Passo 3: Identificar as Ações de Controle Inseguras

O terceiro passo busca identificar, dentro dos *control loops* modelados no passo anterior, quais ações de controle são inseguras. Essas ações — as UCAs — mantêm rastreabilidade direta com os hazards e as perdas definidos no Passo 1: cada UCA identificada pode ser associada a um ou mais hazards de nível de sistema.

As UCAs identificadas neste passo servem de base para a criação de requisitos funcionais e de restrições sobre o sistema. Assim como o Passo 2, o Passo 3 também permanece inalterado independentemente da propriedade emergente analisada.

28.5 Passo 4: Identificar os Cenários de Perdas

O último passo tem como objetivo identificar os cenários de perdas: as razões pelas quais um controle inseguro pode ocorrer no sistema. Esses cenários explicam, por exemplo, por que um controlador emite ou deixa de emitir uma ação de controle seguindo fatores como *feedback* incorreto, requisitos pouco claros ou erros de projeto em diversos componentes.

Os cenários também explicam o caso complementar: quando uma ação de controle segura é fornecida, mas não é seguida ou executada de forma apropriada, levando ainda assim a uma perda inaceitável.

28.6 Requisitos e Rastreabilidade ao Longo dos Passos

Um traço distintivo da STPA é que requisitos são identificados ao longo de todos os passos, não apenas no início da análise. No Passo 2, requisitos emergem da modelagem dos *control loops*, suas ações de controle e *feedback*. No Passo 3, requisitos e restrições surgem da identificação das UCAs. E o Passo 4 permite identificar requisitos adicionais de safety, além de recomendações de projeto, direcionamentos de arquitetura, revisão de decisões de design já tomadas e identificação de lacunas em análises anteriores — especialmente quando a STPA é aplicada após o design já estar finalizado.

Os cenários do Passo 4 também subsidiam a definição de casos e planos de teste, bem como a identificação de indicadores de que o sistema está migrando para um estado de maior risco — temas que serão retomados ao longo desta parte do livro.

Este capítulo, assim como os quatro que seguem, foi elaborado a partir do Capítulo 2 do *STPA Handbook*. Os próximos capítulos detalham cada um dos quatro passos apresentados aqui.

Síntese do Capítulo

- A STPA é composta por quatro passos: definir o objetivo da análise, modelar a estrutura de controle, identificar ações de controle inseguras e identificar cenários de perdas.
- O Passo 1 delimita perdas inaceitáveis, hazards de nível de sistema e os limites do sistema analisado.
- O Passo 2 modela a estrutura de controle hierárquica de forma iterativa, do mais genérico ao mais específico.
- O Passo 3 identifica as UCAs, sempre rastreáveis aos hazards e perdas definidos no Passo 1.
- O Passo 4 identifica os cenários causais que explicam por que ações de controle inseguras ocorrem ou por que ações seguras não são executadas corretamente.
- Todos os artefatos produzidos pela STPA mantêm rastreabilidade entre si, e a técnica se aplica tanto a safety quanto a outras propriedades emergentes, como security.

Capítulo 29

Passo 1 da STPA: Definir o Objetivo da Análise

Este capítulo detalha o primeiro passo da técnica STPA: a definição do objetivo da análise. O passo se subdivide em quatro partes — identificação das perdas inaceitáveis, identificação dos hazards de nível de sistema, definição das restrições de nível de sistema e, opcionalmente, o refinamento desses hazards em sub-hazards. Cada parte consome artefatos das anteriores e produz saídas próprias, todas rastreáveis entre si.

29.1 Visão Geral do Passo 1

No Passo 1, o analista de safety identifica as perdas que a análise busca prevenir, os hazards de nível de sistema associados a essas perdas, e define o sistema e sua fronteira — separando o que está sob controle do projetista daquilo que pertence ao ambiente. As quatro partes do passo formam uma cadeia: a identificação de hazards depende da definição da fronteira do sistema; a definição de restrições depende dos hazards já identificados; e o refinamento, opcional, aprofunda os hazards em sub-hazards de maior granularidade.

29.2 Identificação das Perdas Inaceitáveis

Perda

Uma perda envolve algo de valor para os *stakeholders*: perda de vidas humanas ou ferimentos, danos à propriedade, poluição ambiental, perda de missão, perda de reputação, vazamento de informação confidencial, ou qualquer outra consequência inaceitável do ponto de vista de um *stakeholder* relevante.

Diferentes indústrias usam termos distintos para esse conceito — acidente, *mishap*, evento adverso —, mas este livro adota o termo genérico "perda", já que o objetivo da STPA é prevenir perdas de qualquer natureza considerada inaceitável, não apenas as tradicionalmente associadas a safety.

Antes de iniciar a análise, os *stakeholders* devem identificar as perdas que servirão de foco. Todo artefato produzido ao longo da STPA mantém rastreabilidade até uma ou mais dessas perdas, o que permite priorizar e ranquear os resultados da análise conforme a relevância das perdas associadas.

Uma abordagem prática para identificar perdas consiste em: (1) identificar os *stakeholders*; (2) identificar os valores e objetivos que eles esperam do sistema; e (3) traduzir cada valor ou objetivo em uma perda inaceitável correspondente. Exemplos

típicos incluem ferimento ou perda de vidas (L1), perda ou dano a um veículo (L2), dano a objetos externos ao veículo, perda de missão, perda de satisfação de clientes, perda de informação confidencial, perda ambiental e perda de geração de energia.

Algumas diretrizes ajudam a evitar erros comuns nesta etapa:

- Uma perda pode ser qualquer consequência inaceitável para um *stakeholder*, não somente perda de vida ou ferimento.
- Perdas não devem referenciar componentes individuais do sistema nem causas específicas, como "erro humano" ou "falha de freio".
- Perdas podem envolver aspectos do ambiente que não estão sob controle direto do projetista — essa é uma das diferenças centrais entre perdas e hazards.
- É boa prática documentar suposições e exclusões explícitas feitas durante a identificação das perdas.

29.3 Identificação dos Hazards de Nível de Sistema

A segunda parte do Passo 1 identifica os hazards de nível de sistema e os associa às perdas já definidas. Um hazard é um estado do sistema que, combinado com um conjunto particular de piores condições do ambiente, levará a uma perda. Um hazard isolado, sem essa combinação com condições ambientais desfavoráveis, não leva a uma perda por si só.

Para identificar hazards é necessário primeiro delimitar o sistema: um conjunto de componentes que agem como um todo para atingir um objetivo comum, podendo conter subsistemas ou fazer parte de um sistema maior. O sistema possui uma fronteira que separa o que está sob controle do projetista — e portanto dentro do sistema — do que pertence ao ambiente.

Essa fronteira é o critério que distingue perdas de hazards: perdas podem envolver aspectos do ambiente sobre os quais o projetista tem controle parcial ou nenhum controle; hazards são condições ou estados que o projetista ou operador pode efetivamente controlar. O objetivo central da engenharia de safety é eliminar ou mitigar os efeitos dos hazards — e, como o modelo STAMP trata safety como problema de controle, algum nível de controle sobre essas condições é sempre necessário.

Um exemplo ajuda a fixar essa distinção: em uma usina nuclear, três fatores contribuem para uma potencial perda de vida — o vazamento de material radioativo, a proximidade de uma cidade e a direção do vento. Apenas o vazamento é hazard de nível de sistema, pois é o único fator sob controle da engenharia; a localização da cidade e a direção do vento pertencem ao ambiente.

Outros exemplos de hazards de nível de sistema, sempre rastreáveis a uma ou mais perdas: uma aeronave que viola padrões de separação mínima durante o voo; perda da integridade do chassi (*airframe*) de uma aeronave; uma aeronave que deixa a pista, o *taxiway* ou a rampa designada; uma aeronave que se aproxima muito de outros objetos no solo; um satélite incapaz de coletar dados científicos; um veículo que não mantém distância segura do solo e de obstáculos; um veículo aéreo não tripulado que não completa uma missão de monitoramento; e uma usina nuclear que libera materiais perigosos.

29.3.1 Critérios para Definição de Hazards

Alguns critérios orientam a definição correta de hazards de nível de sistema:

- Hazards são estados do sistema, não estados externos do ambiente fora do controle do projetista.
- Hazards não devem descrever causas detalhadas em nível de componente, como bateria insuficiente ou vazamento hidráulico — essas causas serão tratadas em passos posteriores.
- É sempre necessário um pior caso do ambiente para que um hazard leve a uma perda; um hazard isolado não é suficiente.
- Hazards descrevem estados a serem prevenidos, nunca a negação do próprio objetivo do sistema — por exemplo, "uma aeronave voando" não é um hazard, pois voar é o propósito da aeronave.

29.3.2 Erros Comuns e Dicas Práticas

O erro mais frequente é confundir hazards com suas causas — por exemplo, apontar "operador desatento", "falha de freio" ou "vazamento hidráulico" como hazard, quando na verdade são causas que levam a um hazard de nível de sistema. Para evitar essa armadilha, verifique se o hazard identificado se refere ao sistema como um todo, e não a um componente isolado.

Outro erro comum é o excesso de hazards identificados. Não existe um limite rígido, mas dificilmente uma análise deveria ultrapassar de 7 a 10 hazards de nível de sistema (no máximo uma dúzia). Se isso ocorrer, considere combinar hazards muito detalhados em descrições mais genéricas, deixando o refinamento para uma etapa posterior.

Um terceiro erro é o uso de palavras ambíguas ou recursivas na definição do hazard, como o adjetivo "inseguro". A definição "um automóvel trafegando de forma insegura" é vaga demais; é preferível descrever a condição concreta, como "um automóvel trafegando colado ao veículo da frente".

Por fim, lembre-se que a STPA é iterativa: não é necessário identificar todos os hazards já na primeira tentativa. Passos posteriores frequentemente revelam novos hazards, e a lista inicial pode — e deve — ser atualizada ao longo da análise.

29.4 Definição das Restrições de Nível de Sistema

A terceira parte do Passo 1 define as restrições de nível de sistema associadas a cada hazard identificado. Uma restrição de nível de sistema especifica comportamentos ou condições que precisam ser satisfeitos para prevenir o hazard e, conseqüentemente, a perda inaceitável correspondente.

Definir uma restrição costuma ser simples: basta negar o estado descrito no hazard. Se o hazard H1 afirma que "uma aeronave viola padrões de separação mínima durante o voo", a restrição correspondente é "uma aeronave deve satisfazer os padrões de separação mínima de outras aeronaves e objetos durante o voo". Da mesma forma, se

o hazard H2 é a "perda da integridade do *airframe*", a restrição é que "a integridade do *airframe* deve ser mantida mesmo sob condições de pior caso".

A rastreabilidade entre perdas, hazards e restrições não é necessariamente um-para-um: um hazard pode estar associado a várias perdas, e um hazard pode gerar várias restrições. As restrições também podem especificar como minimizar as consequências caso o hazard efetivamente ocorra, seguindo o padrão: "se o hazard ocorrer, então [ação necessária para prevenir ou minimizar a perda]". Por exemplo: "se uma aeronave violar os padrões de separação mínima, a violação deve ser detectada e medidas devem ser tomadas para prevenir a colisão". Note que a restrição não especifica uma solução de implementação particular — isso será tratado em passos posteriores da análise.

A partir desse ponto, o restante da análise STPA identifica sistematicamente os cenários que podem violar essas restrições, levando aos hazards e, consequentemente, às perdas inaceitáveis.

29.5 Refinamento Opcional dos Hazards em Sub-hazards

A última parte do Passo 1, opcional, refina os hazards de nível de sistema em sub-hazards de maior granularidade. Esse refinamento é particularmente útil em análises de sistemas complexos, pois orienta passos futuros, como a modelagem da estrutura de controle no Passo 2.

Uma forma de derivar sub-hazards é perguntar: "o que precisamos controlar para prevenir este hazard?". Tomando como exemplo o hazard H4 ("aeronave se aproxima muito de outros objetos no solo"), a resposta aponta para o controle de desaceleração, aceleração e direção da aeronave em solo. A partir dessas atividades, é possível derivar sub-hazards como "desaceleração insuficiente após aterrissagem, decolagem ou durante o táxi", "manobra de desaceleração assimétrica que direciona a aeronave a outros objetos", "aceleração excessiva durante o táxi" ou "direcionamento insuficiente da aeronave durante fases em solo".

Assim como para os hazards de nível de sistema, também é possível definir restrições de safety para cada sub-hazard — por exemplo, especificando o tempo e a taxa mínima de desaceleração exigidos após aterrissagem, ou determinando que a aceleração não deve ser fornecida após o ponto de decisão V1 durante a decolagem.

Este capítulo, assim como o anterior, foi elaborado a partir do Capítulo 2 do *STPA Handbook*.

Síntese do Capítulo

- O Passo 1 se subdivide em quatro partes: identificação de perdas, identificação de hazards de nível de sistema, definição de restrições de nível de sistema e refinamento opcional em sub-hazards.
- Perdas são consequências inaceitáveis para um *stakeholder* e podem envolver aspectos do ambiente fora do controle do projetista.

- Hazards são estados do sistema sob controle do projetista que, combinados ao pior caso do ambiente, levam a uma perda; não devem ser confundidos com causas ou falhas de componentes.
- O número de hazards de nível de sistema deve permanecer pequeno (em geral até 10), e definições vagas ou recursivas (como o adjetivo "inseguro") devem ser evitadas.
- Restrições de nível de sistema costumam ser obtidas pela negação do estado descrito no hazard, podendo também especificar ações de mitigação caso o hazard ocorra.
- O refinamento em sub-hazards é opcional, mas útil para orientar a modelagem da estrutura de controle em sistemas complexos.

Capítulo 30

Passo 2 da STPA: Modelar a Estrutura de Controle

O primeiro passo da STPA define perdas, hazards e restrições de segurança em nível de sistema. Com esses objetivos estabelecidos, o analista avança para o Passo 2: modelar a estrutura de controle do sistema. Este capítulo apresenta o conceito de estrutura de controle hierárquica, os equívocos mais comuns em sua construção e o processo iterativo de modelagem.

30.1 O Que É uma Estrutura de Controle

Uma estrutura de controle hierárquica é o modelo do sistema composto por loops de controle e feedback interligados, que impõe restrições ao comportamento do sistema como um todo.

O conceito parte do control loop básico: um controlador interage com um processo controlado por meio de ações de controle (control actions) e feedback, operando com base em um algoritmo de controle e em um modelo de processo. Uma estrutura de controle é um conjunto desses control loops sobrepostos, e problemas podem surgir em qualquer ponto — por exemplo, quando o modelo de processo do controlador fica inconsistente com a realidade, levando a ações de controle inseguras (unsafe control actions): um controlador que acredita que a aeronave está descendo quando na verdade está subindo. Falhas de sensor também produzem feedback incorreto, por ausência ou atraso; e o algoritmo de controle pode ser mal implementado, mal projetado desde a origem, ou tornar-se inadequado com mudanças no sistema.

A STPA identifica sistematicamente esses problemas nos loops de controle, antecipando interações complexas entre software e seres humanos que podem levar a perdas. Quando o controlador é humano, os termos mudam, mas o conceito permanece: em vez de “modelo de processo”, fala-se em *modelo mental*; em vez de “algoritmo de controle”, em *procedimentos de operação* ou *regras de tomada de decisão*.

Estrutura de Controle Hierárquica

Modelo funcional do sistema formado por múltiplos loops de controle sobrepostos, organizados em níveis de autoridade. Elementos no topo da hierarquia controlam elementos posicionados abaixo. Seus componentes típicos são: controladores, ações de controle (setas descendentes), feedback (setas ascendentes) e o processo controlado. Atuadores e sensores podem ser adicionados posteriormente, tipicamente durante o Passo 4.

Um exemplo simples ilustra a hierarquia na aviação: no topo, o controlador de tráfego aéreo; no meio, a tripulação de voo; na base, os processos físicos da aeronave,

com a automação atuando como controlador intermediário. Um mesmo elemento pode ser controlador e processo controlado, dependendo do ponto de vista: a automação da aeronave controla os processos físicos, mas, sob a perspectiva da tripulação, ela própria é um processo controlado.

30.2 Equívocos Comuns na Modelagem

Alguns pontos de confusão recorrentes merecem atenção. Primeiro, a estrutura de controle é um modelo *funcional*, não físico: as conexões representam comandos e feedback, não ligações físicas — a interação entre tripulação e controle de tráfego aéreo, por exemplo, não é física, mas funcional. É também um modelo não executável, especialmente por incluir seres humanos: indica responsabilidades (como o controle de tráfego aéreo ter a responsabilidade de enviar instruções à tripulação) sem garantir que a interação ocorra exatamente dessa forma — ainda que a STPA possa produzir requisitos precisos que alimentem modelos executáveis em etapas posteriores.

Segundo, controle não deve ser confundido com obediência: o envio de uma ação de controle não garante que ela será seguida, e a presença de feedback na estrutura não garante que ele será efetivamente enviado quando necessário. Esses elementos indicam apenas o mecanismo de comunicação projetado, não como os controladores se comportarão na prática.

Terceiro, estruturas de controle empregam abstração para gerenciar complexidade. Em vez de detalhar cada subsistema, o analista pode iniciar em nível mais abstrato — representando, por exemplo, a automação da aeronave e os processos físicos que ela controla em apenas dois níveis. O mesmo vale para ações de controle e feedback: em vez de listar cada botão do cockpit, começa-se com ações amplas, refinando-as progressivamente.

30.3 Como Modelar a Estrutura de Controle

A modelagem é um processo iterativo que parte de uma estrutura abstrata e de alto nível, adicionando detalhes progressivamente. Uma abordagem eficaz é iniciar identificando os subsistemas básicos necessários para impor as restrições de segurança e prevenir os hazards definidos no Passo 1.

No exemplo do sistema de freios de aeronaves, a restrição de segurança 6.1 estabelece que a desaceleração deve ocorrer dentro de um tempo definido após pouso ou decolagem abortada, a uma taxa mínima especificada. Essa restrição aponta para subsistemas capazes de impô-la: o subsistema de freios das rodas e o subsistema de propulsão reversa, por exemplo.

Uma vez identificados, o analista dá zoom em cada subsistema até atingir o nível de granularidade necessário, perguntando se será controlado manualmente pela tripulação, por unidades automatizadas, ou por ambos. No exemplo do freio de rodas, o subsistema foi refinado em duas partes: a unidade de controle do sistema de freios (BSCU) e os freios físicos.

Em seguida, define-se as *responsabilidades* de cada controlador — o refinamento das restrições de sistema em nível operacional. Para a restrição 6.1, os freios físicos recebem a responsabilidade de desacelerar as rodas quando acionados pelo BSCU ou

pela tripulação; o BSCU, a de ativar os freios quando solicitado; e a tripulação, a de decidir quanto frear é necessário e configurar o BSCU e o freio automático antes da frenagem.

As responsabilidades servem de entrada para derivar as control actions de cada controlador: da responsabilidade da tripulação de frear quando necessário deriva-se “frear manualmente”; da de configurar o sistema antes da frenagem, “armar e configurar o BSCU”.

O feedback, por fim, é derivado das responsabilidades e das control actions, identificando os modelos de processo que cada controlador necessita para decidir. Se a responsabilidade do BSCU é ativar os freios quando solicitado, seu modelo de processo deve indicar se o comando de frear foi solicitado; o feedback correspondente pode ser a aplicação (ou não) do pedal de freio.

Em síntese, o fluxo de modelagem segue a sequência: identificar subsistemas e controladores; derivar responsabilidades a partir das restrições de segurança; derivar control actions a partir das responsabilidades; e derivar feedback a partir das responsabilidades, control actions e modelos de processo.

30.4 Questões Frequentes sobre a Estrutura de Controle

A estrutura precisa estar completa antes de prosseguir? Não necessariamente. Antes do término do desenvolvimento, ela pode permanecer incompleta e ser refinada iterativamente; o mínimo para avançar é um controlador, uma ação de controle e um processo controlado. A STPA pode até ser aplicada a um projeto já existente para identificar falhas, como feedback necessário mas ausente.

Qual o nível de especificidade exigido nos rótulos? Termos vagos como “comando”, “feedback” ou “computador controlador” devem ser evitados: as etiquetas devem indicar o tipo de comando (“abrir” ou “fechar” válvula) ou a informação transmitida (“velocidade da roda”). Pelo mesmo motivo, atuadores e sensores específicos ainda não precisam constar na estrutura — são informações de design que aparecerão apenas na criação de cenários.

A estrutura enfatiza relacionamentos funcionais, úteis para identificar falhas de projeto, de requisitos, erros humanos e de software; os processos físicos ficam no nível mais baixo da hierarquia, com controladores funcionais acima. Essa hierarquia não precisa ser linear: muitos sistemas incluem múltiplos controladores no mesmo nível, sem relação de controle entre si.

Identificar quem controla quem costuma ser evidente — um supervisor instruindo um subordinado, um computador comandando uma válvula —, mas pode ser menos direto, como gerentes atribuindo prioridades. Vale reforçar: controle não é obediência, e ações de controle podem, por boas razões, ser desconsideradas para evitar uma perda em situação inesperada. Por fim, é boa prática documentar informações adicionais sempre que aspectos relevantes não estiverem visíveis no diagrama.

30.5 Revisando a Estrutura de Controle

Ao revisar uma estrutura de controle, alguns cuidados ajudam a prevenir erros comuns: verificar se os rótulos descrevem informação funcional; confirmar que todo processo físico está sob pelo menos um controlador; revisar as responsabilidades em busca de conflitos ou lacunas; e checar se as ações de controle e o feedback necessários para cada responsabilidade foram identificados. O feedback é opcional em fases conceituais iniciais, mas sua ausência deve ser tratada como lacuna a resolver depois.

Este material foi elaborado a partir do Capítulo 2 do STPA Handbook.

Síntese do Capítulo

- A estrutura de controle hierárquica modela o sistema como um conjunto de loops de controle e feedback sobrepostos, com elementos de maior autoridade no topo.
- Trata-se de um modelo funcional e não executável: as conexões indicam comandos e feedback, não conexões físicas nem garantia de obediência.
- A modelagem é iterativa: parte de subsistemas em alto nível, deriva responsabilidades a partir das restrições de segurança, depois ações de controle e feedback a partir das responsabilidades.
- Rótulos vagos (“comando”, “feedback”) devem ser evitados em favor de descrições funcionais específicas.
- Atuadores, sensores e hierarquias estritamente lineares não são pré-requisitos — esses elementos podem ser refinados em passos posteriores.
- Revisar a estrutura quanto a controladores ausentes, responsabilidades conflitantes e feedback ou ações de controle faltantes é essencial antes de avançar para o Passo 3.

Capítulo 31

Passo 3 da STPA: Identificar Ações de Controle Inseguras (UCAs)

Com a estrutura de controle modelada no Passo 2, a STPA avança para o Passo 3: identificar as ações de controle inseguras, ou UCAs (*Unsafe Control Actions*). Este capítulo define o conceito, apresenta a ferramenta tabular que sistematiza a análise, discute as dúvidas mais frequentes, trata da identificação de UCAs com controladores humanos e mostra como derivar restrições de controlador a partir delas.

31.1 Definição e Ferramenta Tabular

Uma ação de controle insegura é uma ação de controle que, em um contexto particular e no pior caso do ambiente, leva a uma condição de perigo (hazard). O termo “insegura” remete aos hazards identificados na STPA, que podem envolver perda de vida humana e ferimentos — foco tradicional das análises de safety — mas também perda de missão, de performance, perdas ambientais e outras formas de perda mais amplas.

O exemplo condutor é a unidade de controle do sistema de freios de aeronaves (BSCU): a tripulação de voo pode armar, configurar, desarmar ou acionar o BSCU, que em cascata emite a control action “frear” para o controlador hidráulico. A análise trabalha com o hazard H4 (a aeronave se aproximar demais de outros objetos no solo), refinado em sub-hazards como 4.1, 4.2 e 4.3, relacionados à desaceleração da aeronave.

Tipos de Ação de Controle Insegura

A análise do Passo 3 utiliza uma tabela com quatro colunas, uma para cada tipo de UCA: (1) ação de controle *não fornecida* quando necessária; (2) ação de controle *fornecida* quando não deveria; (3) ação de controle fornecida *no momento errado* — muito cedo, muito tarde ou fora de ordem; (4) ação de controle *interrompida cedo demais ou aplicada por tempo excessivo* (aplicável apenas a ações contínuas). Para cada control action da estrutura de controle, o analista percorre as quatro colunas identificando os contextos em que aquele tipo de comportamento leva a um hazard.

Tomando a control action “frear” (Brake) emitida pelo BSCU: a UCA 1 descreve o não fornecimento — o BSCU não fornece o comando de frear durante o *landing roll* (a corrida de pouso, trajeto da aeronave desde o toque no solo até a desaceleração completa) quando está armado. A UCA 2 ilustra o fornecimento indevido — o BSCU fornece o comando de frear durante uma decolagem normal —, e é possível haver mais de uma UCA por coluna: a UCA 5 descreve frenagem com nível insuficiente durante o landing roll, e outra variante descreve frenagem assimétrica no mesmo contexto. Para o tipo temporal, a UCA 3 descreve o comando de frear fornecido com atraso excessivo

após o toque em solo; para o último tipo, a UCA 4 descreve o comando de frear interrompido cedo demais, antes de a aeronave atingir a velocidade de táxi.

O elemento central de toda UCA é o *contexto*: o estado do sistema que torna aquela ação de controle insegura. O contexto é crítico porque nenhuma ação de controle é insegura em absoluto — se fosse, os engenheiros jamais a incluiriam no projeto. Qualquer condição relevante pode compô-lo: condições ambientais, estágios do processo controlado, estados do controlador, ações de controle prévias; palavras como “quando”, “enquanto” e “durante” ajudam a articulá-lo na redação da UCA.

Uma UCA completa reúne cinco elementos: o controlador de origem, o tipo de UCA, a control action em si, o contexto, e o hazard de sistema ao qual está associada.

31.2 Dúvidas Comuns sobre UCAs

- **Uma UCA garante que o hazard sempre ocorrerá?** Não. A UCA 2 (BSCU frear na decolagem normal), no melhor caso, é detectada e revertida a tempo pelos pilotos; no pior caso, ocorre após a velocidade de decisão V1 e a aeronave sai da pista. A STPA sempre assume o pior caso, não o caso médio ou mais provável.
- **É preciso identificar UCAs mesmo havendo salvaguardas?** Sim. Back-ups e proteções redundantes podem falhar, ser insuficientes ou não terem sido verificados para a situação em questão; a existência de uma proteção não justifica omitir a UCA correspondente.
- **Qual a diferença entre os dois tipos temporais?** O terceiro trata do momento errado (cedo, tarde ou fora de ordem); o quarto aplica-se somente a ações contínuas, com duração, quando interrompidas cedo demais ou mantidas por tempo excessivo. Se a control action for modelada como par de comandos discretos, o quarto tipo não se aplica.
- **É necessário exatamente uma UCA por tipo?** Não. Cada categoria pode conter zero, uma ou várias UCAs; todos os quatro tipos devem ser considerados, mas nem todos serão aplicáveis em todo sistema.
- **Existem mais de quatro tipos?** Não, mas há subcategorias. No segundo tipo, por exemplo, cabe considerar se a control action nunca é segura, se possui parâmetros incorretos (insuficientes, excessivos, direção errada), ou se já havia sido fornecida antes.
- **E se uma UCA relevante não se associar a nenhum hazard existente?** Sinal de que um hazard pode estar faltando. Toda UCA deve ser rastreável a um ou mais hazards; sem correspondência, considere adicionar um novo hazard ou revisar o conjunto existente.
- **Deve-se especificar falhas do modelo de processo no contexto da UCA?** Não. O contexto descreve o estado real que torna a ação insegura, não uma suposição do controlador — confundir a UCA com sua causa é erro comum; as causas são objeto do Passo 4.

31.3 UCAs com Controladores Humanos

A identificação de UCAs não muda quando o controlador é humano. Para a control action da tripulação de voo que liga/desliga o BSCU, a mesma tabela de quatro tipos se aplica: a UCA 1 descreve a tripulação não desligando o BSCU quando ocorre comportamento anormal do sistema de freios; a UCA 2 descreve a tripulação desligando o BSCU quando a antiderrapagem é necessária e o sistema funciona normalmente; o tipo temporal identifica a tripulação desligando o BSCU cedo demais, antes de a atuação do freio automático ou do antiderrapagem estar completa. Para o quarto tipo, nesse caso específico, não há UCA aplicável — resultado também válido da análise. Controladores humanos são tratados como qualquer outro componente do sistema (o STPA Handbook, páginas 158 a 160, traz exemplos adicionais para a tripulação de voo).

31.4 Prevenindo Erros Comuns

Algumas diretrizes evitam falhas recorrentes na identificação de UCAs: garantir que cada UCA especifique claramente o contexto que a torna insegura, descrevendo estados reais e não crenças hipotéticas do controlador; manter os contextos bem definidos, sem substituí-los por efeitos futuros; documentar a rastreabilidade entre cada UCA e os hazards correspondentes; revisar criticamente toda classificação “não aplicável”; para ações contínuas com parâmetros, considerar variações como excesso, insuficiência e direção errada; e documentar suposições ou raciocínios especiais por trás de cada UCA.

31.5 Restrições de Controlador

Identificadas as UCAs, o passo seguinte é definir as *restrições de controlador* — diferentes das restrições de segurança em nível de sistema do Passo 1 — especificando o comportamento que cada controlador precisa satisfazer para prevenir as UCAs identificadas. Derivá-las é direto: basta negar a UCA correspondente. Da UCA 1 (o BSCU não fornece o comando de frear durante o landing roll quando armado) deriva-se: o BSCU deve fornecer o comando de frear durante o landing roll quando armado. Da UCA 2 (fornece o comando durante decolagem normal) deriva-se: o BSCU não deve fornecer o comando de frear durante decolagem normal. Da UCA 3 (atraso excessivo após o toque em solo) deriva-se: o BSCU deve fornecer o comando dentro do tempo definido após o toque em solo.

O Passo 3 usa como entrada os hazards do Passo 1 e a estrutura de controle do Passo 2; suas saídas são as UCAs e as restrições de controlador, insumos do Passo 4.

Síntese do Capítulo

- Uma UCA é uma ação de controle que, em determinado contexto e no pior caso do ambiente, leva a um hazard.

- A análise usa uma tabela com quatro tipos: não fornecida, fornecida indevidamente, fornecida no momento errado, e interrompida cedo ou aplicada por tempo excessivo.
- O contexto é o elemento crítico de cada UCA — sem ele, a ação de controle não seria insegura.
- A STPA é uma técnica de pior caso: UCAs devem ser identificadas mesmo na presença de salvaguardas, pois estas podem falhar.
- Controladores humanos seguem o mesmo processo de identificação que controladores automatizados.
- Cada UCA pode ser invertida para gerar a restrição de controlador correspondente, que alimentará a busca por cenários de perda no Passo 4.

Capítulo 32

Passo 4 da STPA: Identificar Cenários de Perdas

O Passo 4 encerra a sequência da técnica STPA: identificar os cenários de perdas. Enquanto o Passo 3 revelou *quais* ações de controle são inseguras, o Passo 4 investiga *por que* elas ocorrem — e também por que ações de controle seguras podem falhar na execução, levando a hazards mesmo sem UCA emitida. O exemplo condutor permanece o mesmo: a unidade de controle do sistema de freios de aeronaves (BSCU) e o hazard de a aeronave se aproximar demais de outros objetos no solo.

32.1 O Conceito de Cenário de Perda

Um cenário de perda descreve os fatores causais que levam a ações de controle inseguras e, por consequência, a hazards. A STPA considera dois tipos: por que UCAs ocorrem, e por que ações de controle são executadas de forma inadequada ou simplesmente não executadas, mesmo quando originalmente seguras. Nesta etapa, é útil refinar a estrutura de controle incluindo atuadores e sensores — os mecanismos pelos quais control actions e feedback são implementados na prática.

Cenário de Perda

Conjunto de fatores causais — não um fator isolado — que, combinados, explicam por que um controlador emite uma ação de controle insegura ou por que uma ação de controle segura falha ao ser executada sobre o processo controlado, resultando em hazard. Um cenário sempre articula uma cadeia de causas, nunca uma falha pontual e desacompanhada de contexto.

32.2 Cenários que Levam a Ações de Controle Inseguras

Duas causas gerais explicam por que um controlador emite uma UCA: comportamento inseguro do próprio controlador, ou feedback e outras entradas inadequadas.

32.2.1 Comportamento Inseguro do Controlador

Quatro motivos costumam explicar por que um controlador fornece — ou deixa de fornecer — uma ação de controle insegura.

O primeiro é *falha do controlador*, tipicamente relevante para controladores físicos: falhas de hardware, perda de energia, entre outras. No exemplo do Handbook, a UCA

em que o BSCU não fornece o comando de frear durante o landing roll quando armado pode ter como cenário causal o controlador físico do BSCU falhando durante o landing roll, resultando em desaceleração insuficiente na aterrissagem.

O segundo é *algoritmo de controle inadequado*. Para a UCA em que a tripulação não desliga o BSCU diante de comportamento anormal do sistema de freios, o cenário pode ser: o indicativo de falha é apresentado à tripulação, mas os procedimentos operacionais não especificam que o BSCU deve ser desligado após esse indicativo. Outra falha recorrente ocorre quando o controlador assume, sem confirmação, que uma ação de controle prévia foi executada com sucesso: o BSCU pode deixar de frear por assumir, incorretamente, que um comando anterior já colocou a aeronave em frenagem. Em análises de security, soma-se a possibilidade de um adversário introduzir deliberadamente essa falha no algoritmo.

O terceiro motivo é *entrada de controle insegura* vinda de outro controlador: as próprias UCAs de controladores individuais, já identificadas no Passo 3, tornam-se causas de comportamento inseguro em controladores subsequentes na hierarquia.

O quarto é *modelo de processo inadequado*, decorrente de o controlador receber feedback incorreto, interpretá-lo mal, não recebê-lo a tempo, ou de o feedback sequer existir na estrutura de controle. Exemplo do Handbook: o BSCU está armado e a aeronave inicia o landing roll, mas não fornece o comando de frear por acreditar, erroneamente, que a aeronave já está parada — crença originada por um feedback que momentaneamente indicou velocidade zero durante uma operação de antiderrapagem, sem que a aeronave tivesse de fato parado.

32.2.2 Feedback ou Informação Inadequados

O segundo grande fator causal está na informação recebida pelo controlador: feedback não recebido, ou recebido de forma inadequada. Cenários de security cabem aqui também, considerando como um adversário poderia injetar, interceptar ou revelar feedback incorreto.

32.3 Cenários de Execução Inadequada da Ação de Controle

O segundo tipo geral de cenário não trata de UCAs, mas de ações de controle que, mesmo corretas quando emitidas, são executadas de forma inadequada ou simplesmente não são executadas — hazards podem ocorrer mesmo sem que nenhuma ação de controle insegura tenha sido emitida. Dois conjuntos de fatores são relevantes aqui: fatores que afetam o caminho de controle e fatores que afetam o processo controlado.

32.3.1 Fatores no Caminho de Controle

Cenários envolvendo o caminho de controle tratam de ações não executadas ou executadas de forma inadequada entre a emissão pelo controlador e a chegada ao processo. A abordagem recomendada é partir de uma control action, identificar o que significaria sua não execução ou execução inadequada, e verificar como o caminho de controle contribui para esse comportamento.

Para “BSCU envia comando de frear”: a não execução pode ser os freios simplesmente não respondendo, por falha do atuador; a execução inadequada, frenagem insuficiente por resposta lenta do atuador — em ambos, desaceleração insuficiente. Há também o caso inverso: o BSCU não envia o comando, mas os freios são aplicados por falha na válvula hidráulica, comprometendo a aceleração na decolagem. Em security, o comando pode ser injetado, interceptado ou bloqueado por um adversário — um ataque de negação de serviço bloqueando o comando de frear, com o mesmo resultado.

32.3.2 Fatores no Processo Controlado

Mesmo quando a ação de controle chega integralmente ao processo, ela pode não ser efetiva ou ser sobrescrita por outro controlador. Os cenários aqui também se dividem entre ação não executada e executada inadequadamente, com foco em fatores do processo em si — pressão hidráulica insuficiente impedindo os freios, ou pista molhada causando aquaplanagem mesmo com frenagem correta, ambos resultando em desaceleração insuficiente. Em security: um adversário injeta um comando que coloca o sistema em modo de frenagem alternativo, com o mesmo resultado.

32.4 Prevenindo Erros na Identificação de Cenários

O erro mais comum neste passo é identificar fatores causais isolados em vez de cenários completos. Considerar apenas falhas de componentes individuais equivale a aplicar a lógica da técnica FMEA (Failure Modes and Effects Analysis), que a STPA busca superar. Um cenário válido combina múltiplos fatores — controlador, algoritmo, feedback, caminho de controle e processo controlado —, nunca um elemento isolado.

32.5 Entradas, Saídas e Rastreabilidade

O Passo 4 utiliza como entrada os hazards, a estrutura de controle e as UCAs do Passo 3; sua saída, em ambos os tipos de cenário, são os artefatos de *cenários de perda inaceitáveis*.

A rastreabilidade entre os artefatos produzidos ao longo dos quatro passos é central ao modelo STAMP e à STPA. As restrições de controlador remontam às UCAs, que remontam aos hazards, que remontam às perdas. Cenários com UCA associada seguem a cadeia cenário–UCA–hazard–perda; cenários sem UCA seguem diretamente cenário–hazard–perda.

32.6 O Uso das Saídas da STPA

As saídas da STPA — perdas, hazards, restrições, UCAs e cenários — têm aplicações que vão além do relatório de análise: podem direcionar decisões de arquitetura, alimentar requisitos, apontar recomendações de design, indicar mitigações e salvaguardas, orientar planos de teste, subsidiar sistemas de gestão de safety e servir de entrada para indicadores antecipados de risco (leading indicators).

32.7 Considerações sobre a Técnica

Ao final dos quatro passos, fica evidente por que a STPA se diferencia das técnicas tradicionais de safety: em vez de cadeias lineares de falhas de componentes, ela trata o sistema como uma estrutura de controle hierárquica, na qual perdas emergem de interações inadequadas entre controladores, processos e feedback — mesmo sem falha de nenhum componente individual. Essa perspectiva sistêmica permite lidar tanto com falhas físicas e erros humanos quanto com ameaças deliberadas de segurança.

Síntese do Capítulo

- Um cenário de perda articula os fatores causais — nunca isolados — que levam a UCAs ou a hazards.
- Existem dois tipos de cenário: os que levam um controlador a emitir uma UCA, e os que levam a uma ação de controle correta ser executada de forma inadequada ou não executada.
- O comportamento inseguro do controlador decorre de quatro causas: falha do controlador, algoritmo de controle inadequado, entrada de controle insegura e modelo de processo inadequado.
- Ações de controle corretas podem falhar por fatores no caminho de controle (atuadores, comunicação) ou no processo controlado (pressão hidráulica, condições ambientais).
- Cenários de security seguem a mesma estrutura, substituindo falhas espontâneas por ações deliberadas de um adversário.
- A rastreabilidade entre perdas, hazards, UCAs, restrições e cenários é o que sustenta a aplicabilidade prática da STPA ao longo de todo o ciclo de vida do sistema.

Considerações Finais

Ao longo deste livro, percorremos o caminho da engenharia de requisitos e da engenharia de sistemas até os modelos modernos de análise de segurança. Vimos os fundamentos do SWEBOK e do INCOSE, os conceitos básicos de sistema, controle e complexidade, o estudo de caso do desastre de Bhopal, os limites dos modelos tradicionais de causalidade de acidentes baseados em cadeias de eventos, e a mudança de paradigma proposta pelo modelo STAMP, que trata safety como um problema de controle. Por fim, detalhamos os quatro passos da técnica STPA para identificação de hazards, ações de controle inseguras e cenários de perda.

Que este material sirva como referência de consulta sempre que for necessário relembrar, aprofundar ou aplicar esses conceitos ao longo da trajetória acadêmica e profissional.